Databases

Thomas Weise (汤卫思)

February 11, 2025

Abstract

This book is an introduction into databases for undergraduate and graduate students.

Contents

Contents i

1 Introduction 1
- 1.1 Features that we want from a Database . . . 3
- 1.2 History . . . 6
- 1.3 Software . . . 8

I Getting Started 10

2 Installing PostgreSQL 12
- 2.1 Installing PostgreSQL under Ubuntu Linux . . . 12
- 2.2 Installing PostgreSQL under Microsoft Windows . . . 18

3 Installing LibreOffice 32
- 3.1 Installing LibreOffice under Ubuntu Linux . . . 33
- 3.2 Installing LibreOffice under Microsoft Windows . . . 34

4 Installing Python, PyCharm, and Psycopg 43
- 4.1 Installing Psycopg . . . 43

II A Simple Example: The Factory Database 46

5 Creating a User and the Data 47

6 Creating Tables and Filling them with Data 49

7 Selections and Views 56

8 Connecting from Python 59

9 Cleanup After the Example 61

Backmatter 63

Glossary 64

Acronyms 68

SQL Commands 69

Bibliography 70

Preface

This book is an introduction into databases for undergraduate and graduate students.

This book is intended to be read on an electronic device. Please do not print it. Help preserving the environment.

This book is work in progress. It will take years to be completed and I plan to keep improving and extending it for quite some time.

This book is freely available. You can download its newest version from https://thomasweise.github.io/databases. This version may change since this book is, well, work in progress. The book is released under the *Attribution-NonCommercial-ShareAlike 4.0 International license* (CC BY-NC-SA 4.0). You can freely share it. You can also copy text or figures, as long as you cite the book as the original source [147], e.g., by using the following BibTEX:

```
@book{databases,
 author = {Thomas Weise},
 title = {Databases},
 year = {2025},
 publisher = {Institute of Applied Optimization,
              School of Artificial Intelligence and Big Data,
              Hefei University},
 address = {Hefei, Anhui, China},
 url = {https://thomasweise.github.io/databases}
}
```

The text of the book itself is also available in the repository https://github.com/thomasWeise/databases. There, you can also submit issues, such as change requests, suggestions, errors, typos, or you can inform me that something is unclear, so that I can improve the book. Such feedback is most welcome. The book is written using LATEX and this repository contains all the scripts, styles, graphics, and source files of the book (except the source files of the example programs).

Copyright © 2025

Prof. Dr. Thomas Weise (汤卫思教授)
at the Institute of Applied Optimization (应用优化研究所, IAO)
of the School of Artificial Intelligence and Big Data (人工智能与大数据学院)
of Hefei University (合肥大学),
in Hefei, Anhui, China (中国安徽省合肥市)

Contact me via email to tweise@hfuu.edu.cn with CC to tweise@ustc.edu.cn.

[book pdf] [course website]

https://thomasweise.github.io/databases

This book was built using the following software:

```
Alpine Linux 3.21.2
Linux 6.8.0-1021-azure x86_64

python: 3.12.9
PostgreSQL client: 16.6
psycopg: 3.2.4

latexgit_py: 0.8.27
pycommons: 0.8.64
pdflatex: pdfTeX 3.141592653-2.6-1.40.26 (TeX Live 2024)
biber: 2.20
makeglossaries: 4.55 (2024-11-01)
ghostscript: 10.04.0 (2024-09-18)

date: 2025-02-11 23:27:57 +0000
```

Chapter 1

Introduction

There are many different kinds of data.Since these kinds differ very much in their nature, the ways in which we store and retrieve them vary as well.

For example, there exists unstructured data, like texts, graphics, or this book. Such unstructured pieces of data are usually stored in single document files. An organization, e.g., a company or a university, produces heaps of such documents. Every year, the students of our university write Bachelor's and Master's theses. The different schools of our university submit reports, presentations, budget plans, and so on. The university itself issues regulations and notices. There exist common structures for the different document types in our university, such as templates for theses. However, beyond such common structures, the data in the documents can vary enormously and there is no way to unify it. Therefore, we "store" such data as singular documents and maybe organize these documents in catalogs, e.g., by student ID, year, school, department, title, keyword, and so on. If we want to retrieve the data from the theses, we can maybe search them by title or keyword. Once we found the right document, we arrive at the end of what traditional technology can offer us and we have to read them. Nowadays, maybe we can generate an AI summary, but in the end, we still have unstructured information to digest.

Then, there is data that is tightly structured and self-contained. For data that can be stored in a single table, simple text formats like Comma-Separated Values (CSV) [124] or Excel tables [23, 70] are common. If the data is more complex, maybe hierarchically structured, formats like XML [29, 41, 99], JSON [28, 134], or YAML [40, 57, 99] may be more suitable. All of these formats have in common that they store data in singular documents. They offer clear and strict rules how the data can be defined, structured, stored, and retrieved. They are open standards and vendor-independent. With the exception of Excel tables, they are also text-based formats.[1] However, they are mainly suitable for data that, well, can be stored in single files efficiently. They are not suitable for manipulating huge datasets. They are also not suitable for modelling more complex relationships between different datasets. For example, if we want to represent all the personnel, schools, students, and assets of our university, we could try to do that in a single XML document. This document would need to store which teacher belongs to which school, which student belongs to which school, which student is supervised by which teacher, and so on. Maybe a single person could write such a document. But as soon as multiple people need to work on such a document together, everything will collapse. Clearly, another way to work with *relational* data is needed.

Welcome to the course *Databases*. Here, we learn about exactly such a way. Before we begin to do that, let us clarify some simple definitions:

Definition 1 (Database) A database (DB) is a computerized collection of interrelated stored data of potentially many types, maybe accessed by many users and applications concurrently.

There exist many types of databases. There are databases for documents, databases for complex objects, and databases geographical data. We, however, will mainly focus on databases that store data in tables which may be coupled by relationships:

[1] Today, Excel tables are basically compressed collections of XML documents.

Definition 2 (Relational Database) A relational database is a database that organizes data into rows (tuples, records) and columns (attributes), which collectively form tables (relations) where the data points are related to each other [39, 74, 75, 128, 130, 149].

Definition 3 (Record) A record is a group of related data items treated as a unit by an application program.

For example, a record with student information could store the student's name, date of birth, ID number, and mobile phone number. Records are the basic units of data stored in relational databases. A second key element of relational databases are the relationships between records. These relationships guard the correctness and integrity of the data.

Before, we mentioned that a database for our university could store "which student is supervised by which teacher." This would mean that each student record can be linked to teacher records. Does it have to be linked to one? Maybe not, because when the student enrolls into the university, they may not yet have a supervisor. So maybe it is OK if a student record is not linked to any supervisor record. But if it is linked to a supervisor record, then it must be ensured that this record exists and is valid. Of course, it may also be permissible that a student record is linked to more than one supervisor record. Maybe the student has a primary supervisor and a secondary supervisor. As you can see, figuring out how the records can be related alone can already be a bit challenging.

So now we have arrived at the kind of data that we want to talk about. We talk about data that comprises entities of different types, say, student records, teacher records, course records, maybe even teaching room records, schedule elements, grade records, and so on. Between such records, relations exist that must not be violated. There could be arbitrarily many such records. Indeed, DBs usually grow over time. And there often are many people (or programs) who work with the DBs. It is easy to see that we will need some kind of software to manage all of that.

Definition 4 (Database Management System) The database management system (DBMS) is a software system for manipulating databases. It offers the ability to create, modify, and delete databases, tables, and records, to add rows to tables, and so on. It also manages and controls the access rights to the database.

Database management systems can be arbitrarily complex pieces of software. If you think about what things we would like to have when dealing with important data, it becomes immediately clear why.

First, there may be lots and lots of data and we want to access it. We want to find, add, remove, or change certain records. And we want to do this reasonably quickly. So first, a DBMS provides efficient access to the data. Thus, it needs to efficient and therefore complicated datastructures and access algorithms. If you have learned `Python` programming [148], then you will know that finding records in a sequential `list` is much slower than finding them in a hash table (a `dict`)..... but these data structures exist in the RAM of a computer, and DBMSs need to work with files on the disk. Therefore, the datastructures they implement become even more complicated.

Second, database management systems also allow us to specify the possible relationships between data. And they need enforce that the data never violates the relationships.

Third, they also need to offer a user management and access rights management. In a databases for the human resources department (HR) in a big organization, for example, not every HR staff member is allowed to access (and much less to modify) all the data. The same holds for the applications that access the data and, maybe, present some of it on the website.

Fourth, a DBMSs also must take care to properly manage concurrent access, i.e., in situations where multiple users and multiple programs work on the same database at the same time, it must preserve data integrity and consistency. It also has to offer functionality to create backups, i.e., copies of the databases that are preserved in case of errors, hardware faults, or other situations that could destroy the original databases. In some scenarios, databases can become very huge, too big to be stored on a single computer. Today's database management systems often allow us to partition the databases and distribute them over a cluster of computers.

These are just some examples of the features that a DBMS must provide. We discuss more of them in Section 1.1. From the fact that database management systems need to take care of many complex

tasks, it follows that using, configuring, and maintaining them over many years is probably challenging, too. Working with and maintaining databases is a job that can require a lot of expertise. In many organizations, there are specific positions for database experts:

Definition 5 (Database Administrator) A database administrator (DBA) is the person or group responsible for the effective use of database technology in an organization or enterprise.

And *many organizations* is an understatement. The HR departments for organizations like universities and companies use relational databases to manage the members of the organization. The financial departments keep track of salary payments, of budgets for projects, of funding, and benefits in relational databases. Asset management departments maintain lists of, well, assets, like computers, furniture, cars, tools, machines, expensive equipment, in relational databases. Banks manage accounts using relational databases. The government manages tax payments using relational databases. Relational databases store the information about the inventory and cashflow of online shops. Basically every organization with a certain size uses multiple relational databases.

1.1 Features that we want from a Database

So far, we developed some rough ideas about what a database is. We developed these ideas by contrasting the requirements of organizations with the limitations of simple document files. To get maybe some further ideas of what we want to accomplish with databases, let us reiterate what requirements organizations face when dealing with data. Let us work through this list of desired features by imagining that we are building a database for a bank. We can imagine that the database is supposed to store customers, accounts, transactions, and bank employees. Without getting too specific, we can explore our expectations based on this idea. Our goal will be to compile some sort of wishlist of features.

1.1.1 Data Modelling and Representation

First of all, the database management system must give us the tools into our hands to create and implement a model for our data. Matter of fact, we can distinguish several facets of modeling and representing data. We here focus on the *relational* data model, based on tables on the relations between them. There can also be databases using other models, such as graphs, hierarchies, objects, and such and such.

Based on the data model, we must be able to define a conceptual or logical schema for our database. For example, we want to specify which kind of tables exist, e.g., a table with customer information, a table with bank employee information, a table for the accounts that exist, and a table with all the transactions. We can define which information we want to store about customers, say their name, ID number, mobile phone number, etc. This includes some basic validity constraints, e.g., names cannot be empty, ID numbers must follow the standard Chinese ID number scheme, and so on. We must also be able to specify the relationships between these tables. For example, each bank account must be associated with exactly one customer, but one customer may have multiple accounts. Ideally, this can be done using a simple programming language.

This logical schema can be separate from the physical schema of the database, namely how the data is actually organized on the disk. The database management systems should allow the database designers to, for instance, define indexes for tables to speed up the access.

Nevertheless, the concept of *Data Independence* is important: There are different levels of abstraction. The application programs that access the data based on the logical schema should be independent from the physical organization of the data. It should not really matter to them whether records are stored in a hash, a B-tree, or as simple sequences in a file. If we create an easy-to-use application as interface for our banking database, this interface would interact with the logical view of the data via the database management system. How the data is actually stored does not matter to our application. This is called *physical data independence*.

Let's say we create two applications: We create an application for the bank employees to manage accounts and customers. We also create a web application through which the customers can log into their bank accounts and make transfers. The first application may offer much more information to the bank employees. Customers have do not have access to the accounts of other customers, for example. The web application for customers can also not access information about the bank employee

who is managing their accounts. Therefore, it should work independently from such information. If the structure of these information (that it cannot see) is changed later, then this should have no impact on that program. This is called *logical data independence*.

1.1.2 Data Availability

The database management system makes a collection of data is available to users and applications in a meaningful format at a reasonable performance. We expect that the speed of reading, writing, changing, deleting, and adding of data to the database be high. A simple and reasonably abstract programming language should be provided for querying and updating the database. There should also be tools available that further simplify the modeling, editing, and sorting of data as well as the generation of reports from data.

1.1.3 Data Integrity

The correctness and validity of the data is ensured. This includes several aspects: First, the modeled relationships must be enforced. If it is stated each bank account is associated with exactly one customer, then there must never be a bank account record that is not associated with an existing customer. Also, there can never be two bank accounts with the same account ID. This property is called *Consistency*.

Let's say that for each account, we store the current balance. If a customer deposits money, then we can simply add the amount to their account's balance. This is one operation where not much can ge wrong. However, if money is transferred from one account of our bank to another account, then this "looks like" two operations. But it actually is a single *transaction*. A database should be consistent before and after each transaction.

Atomicity means that such a transaction will either complete successfully in its entirety or fail. If the transaction is completed successfully, i.e., *commits*, then all the changes become visible. If the transaction is aborted, no change is performed. No transaction can complete partially, it is all-or-nothing. This means that either the money is correctly subtracted from one account and added to another *or* nothing happens. It must never happen that money is subtracted from one account but does not arrive in another due to an intermediate error.

1.1.4 Concurrency Support and Isolation

Closely related to data integrity is the support for concurrency. Multiple users and multiple processes can access and modify the database at the same time. The database management system preserves the integrity of the database and of all the views that the users and process have on the data.

Different concurrent updates to the same account must take place atomically and in isolation. This means that if money is transferred from account A to account B while simultaneously money is transferred from account B to account C, the two transactions should not influence each other. Both transactions should be isolated.

Another aspect of concurrency is that in some cases, we want to distribute databases over cluster of computers. Maybe the databases are too big. Maybe we want to increase the performance further by dividing the data into several sets which are almost independent. Either way, an ideal database management system should offer us the functionality for doing that.

1.1.5 Durability and Data Safety

A basic feature that any database needs to support is *Durability*: Once a transaction is committed, its changes are permanent. The changes caused by the transaction are saved to the database permanently.

The database management system has to ensure that data can be preserved in case of unforeseen situations. Indeed, durability extends also to system crashes and failures. This includes the support of checkpoints and recovery after restarts.

Of course, there are limits to this: At some lower layers, databases are stored as files in the filesystem. If the underlying hard disk fails, these files can be destroyed. There is no magical way a database management system can guard against this. No system can be entirely safe from hard disk failures and crashes.

Therefore, database management systems need to offer methods to back up the data, i.e., to create copies of the databases and store them on other devices. Then, as long as reasonable backup strategies

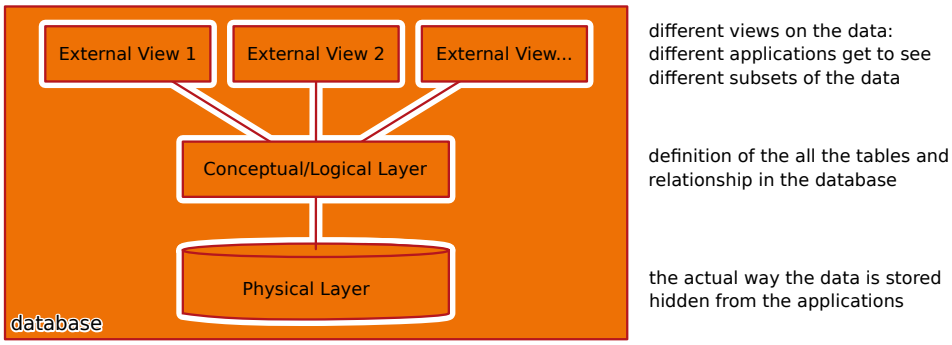


Figure 1.1: The Three Schema Architecture of databases [4, 30, 122, 137].

are employed, it must be possible to re-create the data after the original database was lost. It is clear that for a bank, the data about its and its customers' accounts is its business. If that data is lost, the bank is lost. So ensuring that the database is robust and that it can be recreated from backups is of paramount importance.

The features *A*tomicity, *C*onsistency, *I*solation, and *D*urability together are often called *ACID* properties.

1.1.6 Data Privacy and Security

The database management system must enable the protection of data against access from unauthorized access from both within and outside of the organization. It must be possible to specify roles for users and processes that define which data they can access or modify. For example, the database administrator of the bank's database needs to be able to modify the structure of the database, maybe add and modify tables. A normal bank employee should not be able to do that. But they may be allowed to enter new records into certain tables of the the database, for instance, to add customers or to create new accounts. Thinking this through even further, a bank employee may be assigned as manager to some customer's accounts. The employees should only be able to see the information in these accounts. All of this can be done by assigning roles to users and processes and access rights to roles.

Another aspect of this is the so-called Three-Schema Architecture [4, 30, 122, 137] illustrated in Figure 1.1: It should be possible to provide different views on the data for different users and applications. As already mentioned before, the application for bank employees should be able to "see" other data than an application for customers. In other words, at the bottom layer of the database is the physical schema managing how data is stored. On top of that sits the conceptual (or logical) layer, where the structure of the tables and the relationships between them are defined. On the highest level, different views on the data can be provided for different applications. This way, users can only see the data that they need to be able to see and nothing more.

This goes hand-in-hand with security measures. Examples could be that all access to the database is password protected. Communication with the database happens over encrypted connections.

A database management system should also support accounting and an audit trail. It must be possible to log information regarding which user changed which dataset and when. This is required to pass certain government, financial, or ISO certification audits.

1.1.7 Summary

It is quite obvious that the basic filesystem only offers very few of the properties discussed above. Using simple data formats like CSV or even Excel tables will not be a good choice either. While one might realize concurrency by placing multiple interrelated such documents on a shared folder, there certainly would not be any *ACID* guarantees. Multiple people editing them at the same time will wreak havoc. Structured formats like JSON, XML, or YAML would allow us to represent complex data in single documents, but this would make the multi-user access situation even worse. And none of these solutions could provide any means to protect data integrity, let alone access control.

Instead, we require a software layer between the database and the user processes. Honestly, we do not even really care of this software layer actually stores the data, how the data is organized internally,

or how *ACID* is implemented. As long as this software, the database management system, offers us the above features, we will be happy.

1.2 History

We will now take a brief look at the history of databases [3, 66, 107, 125, 154]. People began storing and processing information a very long time ago. A driving force to collect and analyze data must have been the management of limited resources. Some of the earliest discovered writings are Sumerian accounting and tax records on clay tablets from Mesopotamia, dating back four thousand years [46, 98, 139] The collection and analysis of information never stopped from then onwards.

When much data is stored, three big questions emerge: *How do we store the information?*, *How can I find a specific record?*, and *How can we condense and extract representation information from all of this data?*

Examples for the data organization include the Dewey system for organizing books in a libray, which emerged in the 1870s [36]. Not much later, we enter the earliest stage of data processing with machines. Data storage happened using physical tools [78]. The punched cards system by Hollerith, patented in the late 1880s [81, 82], was used in the 1890s US Census. The automatic processing of these cards allowed the census to finish under budget and ahead of time [133]. Hollerith's Tabulating Machine Company eventually merged with three other companies into International Business Machines (IBM). Punch cards made up 20% of the revenue of IBM as late as the mid-1950s [133].

In addition to punch cards, reels of punched tape emerged as data storages and later magnetic tapes. The way data can be retrieved depends on how the data is stored. Punch cards can be sorted, stacked, and otherwise cleverly be arranged to access information quickly. Tape-based storages requires us to sequentially spool through the tape to find the data we are looking for. Either way, efficient organization of data and media became more and more important. In , the Electronic Recording Machine Accounting (ERMA) Mark 1 was developed as automated system for organizing banking records [15]. It had features of file system, although the term "files" was meant literally – they organized paper-based documents, using a structure reminiscent of the aforementioned hierarchical, number-based Dewey categorization method.

In 1956, the IBM 305 RAMAC came out: the first to use a random-access disk drive [115]. It could store five to ten megabytes. Its true innovation was that from now on, it was no longer necessary to store and access data in a sequential order [33]. Instead, the data on its disks could be accessed in any order.

The first file systems for computers appeared in the 1960s. The Atlas system in the UK had offered a rudimentary file system functionality already in 1961 [93]. The Operating System (OS) *Compatible Time-Sharing System* (CTSS) [43] at the MIT had a flat filesystem only one or two years later [111]. The hierarchical file system for the *Multics* OS [44], published in 1965, already had surprisingly many advanced features that we know from today's file systems: fine-grained access control for data privacy, backup ability, links, and IO queue management. File systems are very good for organizing documents and heterogeneous data. They are not very suitable to main the sort of relational data and to achieve the features that would like databases to have.

The first version of the Integrated Data Store (IDS) was developed by Bachman in 1961/62 at General Electric [8, 9]. IDS offered the first direct access database, holding data in virtual memory. It may have been the first real DBMS and Bachman won the 1973 A.M. Turing Award for this work [73]. IDS was based on a network model, further advancing data management by structuring complex relationships in data. The programmer acted as navigator through the data. The idea to step-by-step navigate through data, updating it if needed, is more complicated and slower compared to the relational approach of today's databases. IDS and similar systems encode elements of the views of the data as part of the database structure, which make IDS less flexible. The Database Task Group of the Conference on Data Systems Languages (CODASYL), a standardization body for the data processing industry, took over many of Bachman's ideas for IDS in the late 1960s. CODASYL is best known for its creation of the COBOL programming language [73].

Only slightly later than IDS, another database management systems that could handle the structured storage of records obeying datatype constraints appeared. IBM developed the Information Management System (IMS) for the Apollo space program [95]. The system was launched 1965/1967 [19]. IMS is still sold as a product and still exists today. Like IDS, it was not a database management system for *relational databases*. Instead, it offered hierarchically structured records. This system, too, had

a set of disadvantages [100]: If your data is not structured hierarchically, some records may need to be duplicated. For example, if we have a database that assigns students to courses, then a student would appear in each of the records of the courses that she attends. The programming language is also relatively low-level and, for example, the search strategy has to be implemented. Finally, if changes to the logical schema will require us to perform cascading changes in the code accessing them.

Both IDS and IMS offered a strikingly new concept: The application code and the code for physical data storage and retrieval were separated. Between them, the Data Language One (DL/I) was located as interface in IMS. IDS offered the Data Description Language (DDL) to define types of logical records and the Data Manipulation Language (DML) to manipulate and navigate the data. The database management system controls how the data is stored and loaded. Application programs can navigate through the data using the much simpler interface languages. As we briefly mentioned before: DBMSs can become arbitrarily complicated pieces of software, and one major part of this is the code to efficiently store and retrieve data.

Just think about it: All data is eventually mapped/stored to sequential files on a hard disk or other storage medium. You must be able to add new records and delete old records. You must also be able to find them. This alone is not easy to implement, but all of that has to be efficient, so (today) one would probably like to use data structures like hashes, tress, and sorted lists . . . all of which are actually located in sequential files. Before IDS and IMS, this had to be part of your application's code. Actually, it had to be part of the code of *all* applications that accessed the data, if multiple such applications exist [19]. Now this is part of IDS and IMS, and the user can ignore this complexity and just use a simple programming language where she can define which record to load, change, delete, or add.

> Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation).
>
> — *Edgar Frank "Ted" Codd [39], 1970*

In 1970, the seminal paper *"A Relational Model of Data for Large Shared Data Banks"* by Codd appeared, starting with the above quote. He noticed the shortcomings of the IDS network model, namely that the programmers accessing the data still need a lot of information about how the data is actually represented and organized internally. Even Bachman himself mentioned that issues with IDS arose when users did not consider how data is internally sorted [9]. Codd wanted to protect programmers against such errors. He models data as a relational view by using tables, that he calls *relations*. Each column has a type that defines its permitted values. Each row must be distinct. One set of columns per table (usually a single column), the *primary key*, is used to uniquely identify the row. Rows in one table can references other rows of either the same table or another table. Therefore, some column(s) of the table (called *foreign key*) then store the primary key of the row to be reference. The data in the database is then a collection of tables. He further describes a set of operations that can work on the tables to extract information, which we will discuss later on. By the end of the 1970s, over a dozen database management systems had been implemented based on these new concepts [94]. Codd won the A.M. Turing Award for his work.

The *semantic* division between the storage representation of data and the conceptual layout and access was a very important step in the development of databases, the separation was also a *physical* one. Computer networks as distributed systems began to emerge in the 1960s [101]. Baran had the idea of routing messages over multiple network switches in 1960 [10–14]. In 1961, Kleinrock proposed packet switched networks, where messages are split into multiple packages that each individually travel their corresponding optimal paths [97]. This allows multiple users in a network to share the same data path at the same time. This idea of packet switching was also independently developed by Baran [12] and Davies [51–53]. Licklider proposed standardizing computer and networking languages to increase interoperability and to allow researchers to build upon each other's work in 1963 [106]. This could be considered as the first description of the idea of the internet. In 1965/66, the TX-2 computer at MIT Lincoln Lab in Massachusetts and the Q-32 in Santa Monica, California were connected, forming the first wide-area network [121]. In 1967, ARPANET, the precursor of the internet was conceived by the Advanced Research Projects Agency as a network connecting 35 computers at 16 sites in the USA [101, 121]. In 1969, the first four nodes of the network became operational and in 1972, ARPANET was demonstrated publically [101, 121]. Canaday et al. proposed putting a database management system on a dedicated back-end computer in 1974 [31]. Applications and users would access the DBMS from another computer via a communication link. The first Ethernet was developed at the Xerox Palo Alto Research Center (PARC) in 1976 by Metcalfe and Boggs [42], TCP/IP became deployed 1983, and

in the 1990s, the internet took off [101] The term clients was coined in 1978 by Israel, Mitchell, and Sturgis [88]. From now on, the notation of the client-server architecture [22, 60, 112, 116, 120], in which most DBMSs are implemented today, was formalized. From then on, the development was unstoppable.

Another very important step forward for DBMSs was the SEQUEL language developed by Chamberlin and Boyce in 1974 [35] as part of the IBM System R project. The language was based on Codd's relational algebra, but wrapped it into an easy-to-understand language for data queries. Two years later, it was extended by more features, such as methods for inserting, deleting, and updating records [34]. In 1977, the SEQUEL name was shortened to SQL, an acronym for Structured Query Language (SQL) [33]. At the same time where SEQUEL was developed, the Interactive Graphics and Retrieval System (INGRES) group at the University of Berkeley in California, too, conducted research on databases [129]. The open environment of publishing latest research and collaboration between the researchers contributed significantly to their success. They jointly received the ACM Software System Award in 1988 [33].

Oracle databases, the first commercial SQL-based product, was released in 1979 [33] by the SDL company, which later renamed itself to Oracle. It was an immediate success, because it was portable and could run cheaper hardware. IBM released its SQL databases DB2 in 1983 [33, 72]. SQL became a US ANSI standard in 1986 and an international ISO standard in 1987 [48, 49]. The standard continues to evolve, with its latest version being released in 2023 [86].

In the 1990s, several open source implementations of database management systems became available for free [33]. We will discuss them in the next section.

1.3 Software

A wide range of database management systems for relational databases exist. A few we already mentioned before.

1.3.1 Open Source Relational Database Management Systems

A very big chunk of the relational databases in many scenarios is managed by open source database management systems. These systems are available for free and their source code is readily available in the internet. Large communities exist around them that can offer all kinds of advise. This makes them very attractive for developers and users. Several of these systems are around since the 1990s, so they are mature and tested products [33].

`MySQL` is a DBMS for relational databases [25, 58, 119, 132, 150]. It was originally developed by Michael Widenius and David Axmark at the Swedish company `MySQL` AB and released in 1996 [33]. It soon gained widespread use as part of the `LAMP Stack`, i.e., a system setup for web applications based on the `Linux` operating system, the Apache webserver, the `MySQL` database, and the server-side scripting language PHP [32, 80]. `MySQL` AB was acquired by Sun Microsystems in 2008, which in turn was acquired by Oracle in 2010 [33].

After the acquisition by Oracle, some of the original developers of `MySQL`, including Michael Widenius, created a fork of `MySQL`: `MariaDB` [6, 7, 17, 58, 117]. They promise that `MariaDB` will stay open source forever.

`PostgreSQL` [61, 63, 110, 132] is an object-relational database management system, meaning that it supports concepts from object-oriented programming (OOP), such as inheritance relationships between tables. It also supports many additional types like JavaScript Object Notation (JSON) objects and geometric types. `PostgreSQL` emerged from the POSTGRES project, the successor of the INGRES project at the University of Berkeley in California [33]. It may be the most fully-featured of the existing open source SQL database systems

The SQL database with the widest distribution is `SQLite` [33, 67, 152]. It breaks with the common approach to offer access to the database in a client-server scheme. Instead, it is directly loaded as a library in the process that uses. Today, it is installed on nearly every smartphone, computer, web browser, television, and automobile [33, 67, 152]. It was first released in 2000 and its core designer Richard Hipp received the SIGMOD Systems Award in 2017 [33].

The concept of database management systems that can work on stand-alone files is also implemented in the open source office suite LibreOffice [1, 68, 123]: LibreOffice `Base` [62, 123]. Base is more than a DBMS working on single file. Instead, it offers a powerful user interface that can also connect to

databases such as MySQL, MariaDB and PostgreSQL. The user interface allows you to conveniently design tables, relations, forms, and reports for the databases it connects to. LibreOffice Base is a free alternative to the commercial product Microsoft Access [18, 37, 140], which offers a similar functionality.

Many of the examples used in this book will be implemented based on PostgreSQL and we will also play around with LibreOffice Base.

Part I

Getting Started

The goal of this course is to introduce databases. However, this is a practical course. So we will work on practical examples and create "real" databases on real database management systems. This means that we will look into several interesting topics:

- We will work with an actual database management system. So you need to install an actual DBMS. We choose PostgreSQL [61, 63, 110, 132].
- DBMSs are usually just servers to which you can send SQL commands via a client terminal. However, there exist also nicer tools that offer rich graphical user interfaces (GUIs) that allow you to design forms and reports. We need such a tool as well. We choose LibreOffice Base [62, 123].
- The DBMS is usually just the backend of a rich landscape of tools in an enterprise or organization. Rarely will one work only with a DBMS. Instead, there may be several applications that connect to databases through unified Application Programming Interfaces (APIs). Since we will also take a look at how that works, we need to install a programming language and corresponding "DBMS-access library." We chose the Python programming language [5, 83, 96, 104], because we also provide a free book on it in [148]. As library to connect to the PostgreSQL DBMS, we pick psycopg [144].

As you can see, we only consider free open source software. Learning about databases with this book should be free.

Before we get into the necessary installation and setup steps for the software that we need to really learn about databases, we face a small problem: Today, devices with many different OS are available. For each OS, the installation steps and software availability may be different, so I cannot possibly cover them all. Personally, I strongly recommend using Linux [16, 77, 136] for programming, work, and research. If you are a student of computer science or any related field, then it is my personal opinion that you should get familiar with this operating system. Maybe you could start with the very easy-to-use Ubuntu Linux [38, 80]. Either way, in the following, I will try to provide examples and instructions for both Ubuntu and the commercial Microsoft Windows [26] OS.

Chapter 2

Installing PostgreSQL

For our course, we will use the open source DBMS PostgreSQL [61, 63, 110, 132]. Here, we briefly discuss how to install it on your machine. PostgreSQL follows the client-server architecture. It provides the DBMS implemented as server program. It manages the databases and stores and provides the data. Other programs and the users can connect to it to access the databases. PostgreSQL also offers a client program, `psql`, with which human users can communicate with the DBMS server in a normal terminal. We want to install both pieces of software. You can find more information about this process at the PostgreSQL download page https://www.postgresql.org/download. Here, we provide step-by-step guides for Ubuntu Linux and Microsoft Windows.

2.1 Installing PostgreSQL under Ubuntu Linux

The installation PostgreSQL on Ubuntu Linux is rather simple. First, we open a Bash terminal, i.e., a window into which we can directly type commands, with Ctrl+Alt+T. Both the client and server program are installed via `sudo apt-get install postgresql-16`, as shown in Figure 2.1.1. Notice that we chose to install PostgreSQL version 16, denoted by the `-16` suffix, but you could probably also pick other versions or just install `postgresql` without version specification. Either way, this installation method requires super user privileges, which is why we have to start the command with sudo. In Figure 2.1.2, we therefore need to provide our superuser password.

When this is entered and confirmed with Enter, the package manager checks what needs to be installed. In Figure 2.1.3 it informs us about the package itself and the dependencies that are needed, as well as how much download and space that requires. It asks us whether we are OK with that, which we confirm by typing y+Enter in Figure 2.1.4. Then, the download of the required packages starts. Once the download completes, the packages are installed. After this completes in Figure 2.1.5, PostgreSQL is installed and running.

We now want to double-check whether everything went well. First, we need to investigate whether the PostgreSQL DBMS server is indeed installed and running properly. We can do this by invoking `systemctl status postgresql` in Figure 2.1.6. The output on my laptop computer, given in Figure 2.1.7, indicates that the service is up and running. Notice `active` means that the DBMS is started everytime your computer boots and runs all the time (until you shutdown your computer, that is). Running this program opens some sort of paginated mode, which we can leave by hitting q+Enter in Figure 2.1.8.

If you do not want that, then you disable the service by the command `sudo systemctl disable postgresql` in the terminal, which again requires sudo privileges. The PostgreSQL server will then no longer start automatically. If you want to access it, you then would start it manually via `sudo systemctl start postgresql`. After you are done with it, you can stop it by typing `sudo systemctl stop postgresql`. If you ever want it to start automatically again, this can be achieved by `sudo systemctl enable postgresql`.

Either way, after a successful installation, the PostgreSQL server service is activated and running. We now also want to check whether the client program `psql` was installed correctly. The client can be run in the terminal and allows us to communicate with the DBMS server. We therefore type `psql --version` in Figure 2.1.9, which will print the version of this program after we hit Enter. The result in Figure 2.1.10 shows that the `psql` client version on my system: 16.6.

Let us now connect to the PostgreSQL server via the client for the first time. The goal of this

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ sudo apt-get install postgresql-16
```

(2.1.1) Installing PostgreSQL using the `apt-get install` command in a terminal opened with Ctrl+Alt+T.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ sudo apt-get install postgresql-16
[sudo] password for tweise:
```

(2.1.2) This command requires the super user password, which we type in and then press Enter.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ sudo apt-get install postgresql-16
[sudo] password for tweise:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  postgresql-client-16 postgresql-client-common postgresql-common
Suggested packages:
  postgresql-doc-16
The following NEW packages will be installed:
  postgresql-16 postgresql-client-16 postgresql-client-common
  postgresql-common
0 upgraded, 4 newly installed, 0 to remove and 2 not upgraded.
Need to get 17.0 MB of archives.
After this operation, 50.1 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

(2.1.3) We get asked whether we really want to install the required packages.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ sudo apt-get install postgresql-16
[sudo] password for tweise:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  postgresql-client-16 postgresql-client-common postgresql-common
Suggested packages:
  postgresql-doc-16
The following NEW packages will be installed:
  postgresql-16 postgresql-client-16 postgresql-client-common
  postgresql-common
0 upgraded, 4 newly installed, 0 to remove and 2 not upgraded.
Need to get 17.0 MB of archives.
After this operation, 50.1 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

(2.1.4) We answer with y+Enter.

Figure 2.1: Installing PostgreSQL under Ubuntu Linux, checking its status, and setting a secure password.

```
tweise@weise-laptop: ~
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/16/main ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... Asia/Shanghai
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$
```

(2.1.5) The installation proceeds and finishes.

```
tweise@weise-laptop: ~
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/16/main ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... Asia/Shanghai
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$ systemctl status postgresql
```

(2.1.6) We want to check the status of the fresh `PostgreSQL` installation. We can do this by typing in `systemctl status postgresql` and hit Enter.

```
tweise@weise-laptop: ~
selecting default shared_buffers ... 128MB
selecting default time zone ... Asia/Shanghai
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
lines 1-8/8 (END)
```

(2.1.7) The output shows us that the `PostgreSQL` service is now running. It will always start when we boot our system.

Figure 2.1: Installing `PostgreSQL` under `Ubuntu Linux`, checking its status, and setting a secure password.

```
tweise@weise-laptop: ~
selecting default shared_buffers ... 128MB
selecting default time zone ... Asia/Shanghai
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$
```

(2.1.8) We press q + Enter to leave the status output page.

```
tweise@weise-laptop: ~
selecting default shared_buffers ... 128MB
selecting default time zone ... Asia/Shanghai
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
```

(2.1.9) `psql` is the client program that can connect to the PostgreSQL server and that gets installed as well. We can check its version by typing `psql --version` and hitting Enter.

```
tweise@weise-laptop: ~
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabl ed; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$
```

(2.1.10) At the time of this writing, I got version 16.6.

Figure 2.1: Installing PostgreSQL under Ubuntu Linux, checking its status, and setting a secure password.

```
tweise@weise-laptop: ~
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$ sudo -u postgres psql
```

(2.1.11) In order to set a proper password for the PostgreSQL main user `postgresql`, we need to log into `psql` using sudo privileges, but under the newly created system user `postgres`. We do this by writing `sudo -u postgresql psql` and hit Enter.

```
tweise@weise-laptop: ~
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok
Processing triggers for man-db (2.12.0-4build2) ...
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$ sudo -u postgres psql
[sudo] password for tweise:
```

(2.1.12) We may or may not need to enter our super user password again...

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$ sudo -u postgres psql
[sudo] password for tweise:
psql (16.6 (Ubuntu 16.6-0ubuntu0.24.04.1))
Type "help" for help.

postgres=#
```

(2.1.13) Starting `psql` like this connects us to the local PostgreSQL server

Figure 2.1: Installing PostgreSQL under Ubuntu Linux, checking its status, and setting a secure password.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$ sudo -u postgres psql
[sudo] password for tweise:
psql (16.6 (Ubuntu 16.6-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# ALTER USER postgres PASSWORD '[redacted]';
```

(2.1.14) We can now communicate with the database management system using SQL. We use the `ALTER USER postgres PASSWORD 'XXX';` command, where `XXX` is replaced with a secure password. In the screenshot, the password is covered by a red rectangle after I replaced it with many Xs.

```
tweise@weise-laptop: ~
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$ sudo -u postgres psql
[sudo] password for tweise:
psql (16.6 (Ubuntu 16.6-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# ALTER USER postgres PASSWORD '[redacted]';
ALTER ROLE
postgres=#
```

(2.1.15) After we hit Enter, the system confirms the change by showing the command `ALTER ROLE` back to us. From now on, the server main user has a secure password.

```
tweise@weise-laptop: ~
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; prese>
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$ sudo -u postgres psql
[sudo] password for tweise:
psql (16.6 (Ubuntu 16.6-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# ALTER USER postgres PASSWORD '[redacted]';
ALTER ROLE
postgres=#\q
```

(2.1.16) We quit `psql` by typing \ + q + Enter.

Figure 2.1: Installing PostgreSQL under Ubuntu Linux, checking its status, and setting a secure password.

```
tweise@weise-laptop: ~
     Active: active (exited) since Fri 2025-01-17 15:00:28 CST; 1min 34s ago
   Main PID: 63621 (code=exited, status=0/SUCCESS)
        CPU: 1ms

1月 17 15:00:28 weise-laptop systemd[1]: Starting postgresql.service - PostgreS>
1月 17 15:00:28 weise-laptop systemd[1]: Finished postgresql.service - PostgreS>
tweise@weise-laptop:~$ psql --version
psql (PostgreSQL) 16.6 (Ubuntu 16.6-0ubuntu0.24.04.1)
tweise@weise-laptop:~$ sudo -u postgres psql
[sudo] password for tweise:
psql (16.6 (Ubuntu 16.6-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# ALTER USER postgres PASSWORD '[redacted]';
ALTER ROLE
postgres=#\q
tweise@weise-laptop:~$
```

(2.1.17) We are finished.

Figure 2.1: Installing PostgreSQL under Ubuntu Linux, checking its status, and setting a secure password.

exercise will be to set a proper password for the administrative user of our instance the PostgreSQL DBMS. The installation process created both a Linux system user named *postgres* as well as a user of the same name in the DBMS. The DBMS needs to have its own user and rights management implemented, because we need to be able to grant different programs and users different read and write privileges for a database (remember Section 1.1.6). Thus, we would like the administrative account `postgres` of the DBMS to have some secure password.

We therefore connect the `psql` client to the PostgreSQL server as superuser impersonating the `postgres` system user. This sounds very strange. I am not even sure whether I explained it correctly. Either way, we type `sudo -u postgres psql` into the terminal and hit Enter in Figure 2.1.11. We may get asked to provide the superuser password for our computer again in Figure 2.1.12, in which case we simply provide it and hit Enter.

For the first time, we are now in the `psql` client and are connected to the PostgreSQL server. We see the prompt `postgres=#` in Figure 2.1.13.

To set the password for the user (or role) `postgres`, we type in `ALTER USER postgres PASSWORD 'XXX';` in Figure 2.1.14. (This is equivalent to `ALTER ROLE postgres PASSWORD 'XXX';` in PostgreSQL.) This is actually an SQL command and we may (or may not) learn later what this exactly means (once and if I get to write such a chapter). This would set the password to `XXX`. Obviously, this is not the secure password that we are going to use. Instead, you will not type `XXX` but a secure password of your choosing. A password that you shall remember well. We hit Enter.

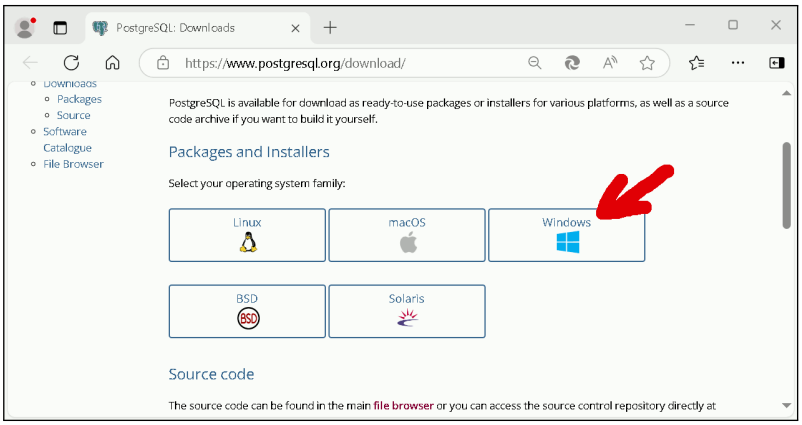
In Figure 2.1.15, we see that this prints `ALTER ROLE` back to us. It shows us the command that was performed. We did alter the role (or user) `postgres`. The new password is set. You will learn how to use that another time.

For now, we will happily exit the `psql` client by typing \ + q + Enter. In Figure 2.1.16, we quit the client this way. In Figure 2.1.17, we then are back in our normal Bash terminal. We have finished and validated the PostgreSQL installation. And we have set a proper password for the administrator account `postgres`.

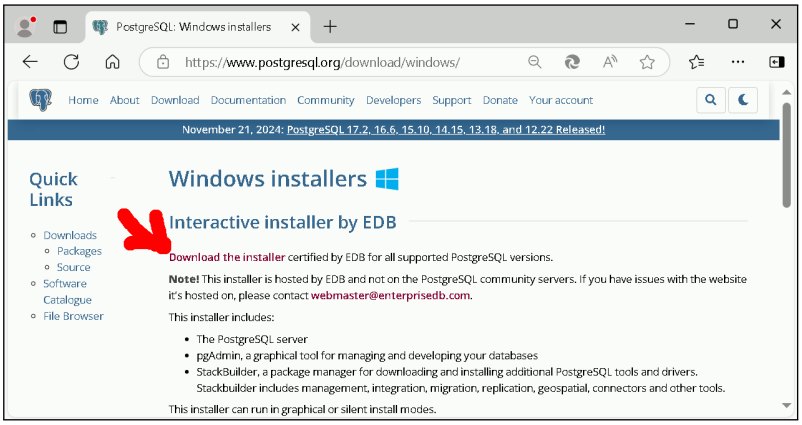
2.2 Installing PostgreSQL under Microsoft Windows

We want to install and properly configure PostgreSQL on a Microsoft Windows system. To do this, we first visit the official download page at `https://www.postgresql.org/download`, as shown in Figure 2.2.1. There, we click on the large *Windows* menu. This leads to a page with more text and the link *download the installer* (Figure 2.2.2). This, in turn, takes us to `https://www.enterprisedb.com/downloads/postgres-postgresql-downloads`, as shown in Figure 2.2.3.

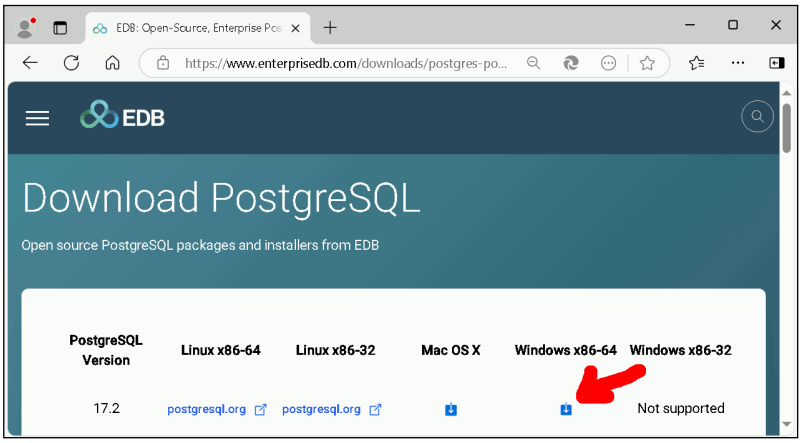
On this website, a wide variety of different versions of PostgreSQL for different OSs can be downloaded. We choose the newest version for Windows, which can be found in the top-most row of the list presented to us. At the time of this writing, this is version 17.2. We click the corresponding download



(2.2.1) We visit the website `https://www.postgresql.org/download` and click on Windows.

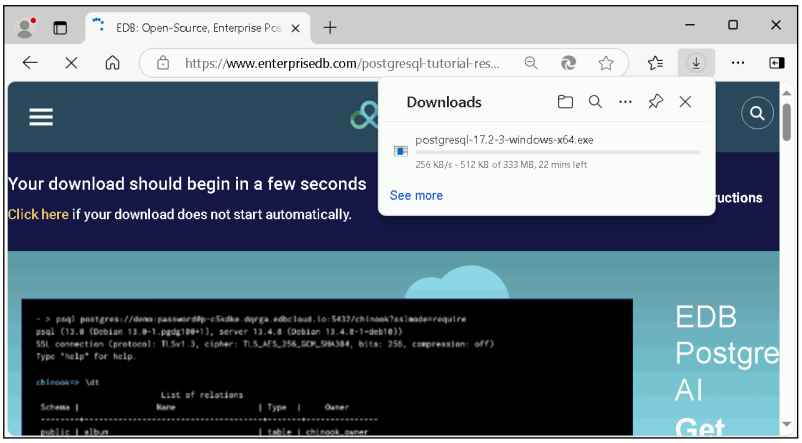


(2.2.2) We click on the link *download the installer*, which takes us to `https://www.enterprisedb.com/downloads/postgres-postgresql-downloads`.

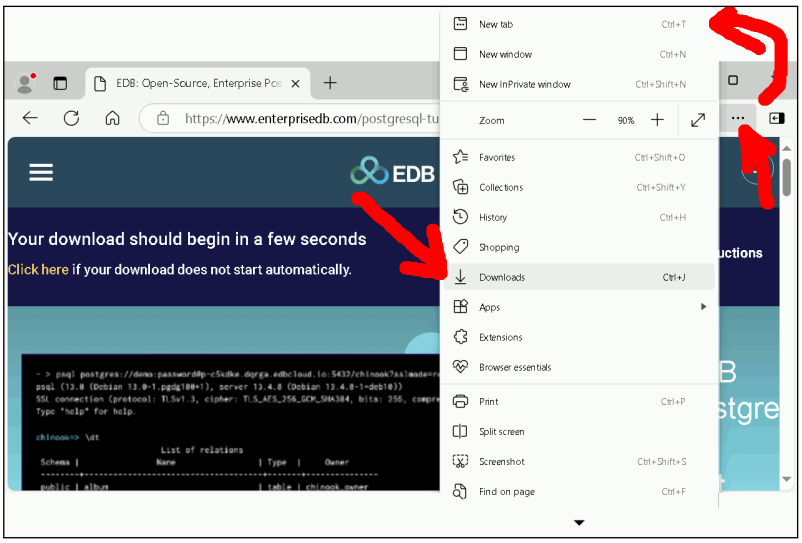


(2.2.3) A wide variety of versions for different OSs are available. We choose the newest (top-most) one for `Windows`, at the time of this writing, this is version 17.2. We click the download icon.

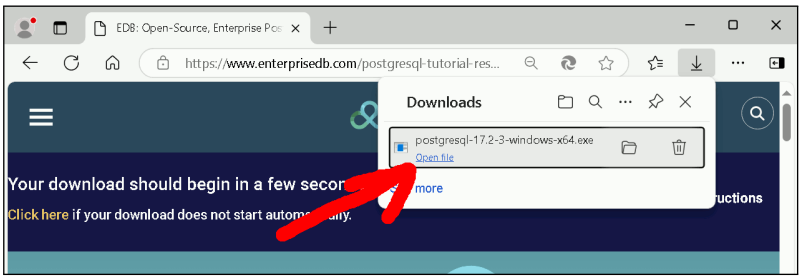
Figure 2.2: Installing and configuring `PostgreSQL` under Microsoft `Windows`.



(2.2.4) The download begins.



(2.2.5) After the download completes, we need to find it. . .



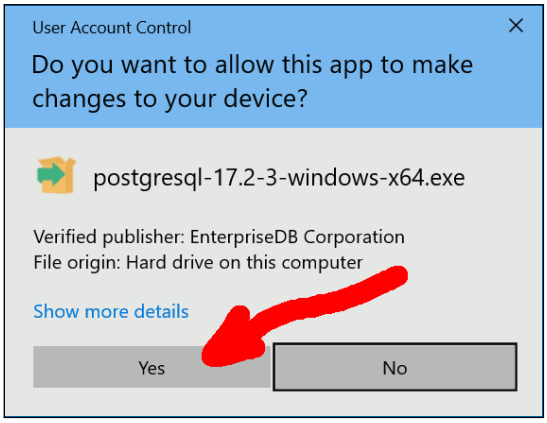
(2.2.6) . . . and execute it by clicking on Open file.

Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.

icon.

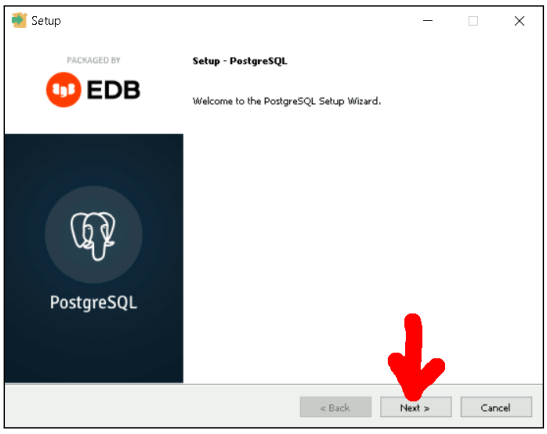
The download begins, as shown in Figure 2.2.4. Once it completes, we need to find the downloaded file, as shown in Figure 2.2.5. I downloaded the software using the Microsoft Edge browser, so I need to click on the ... button and then on Downloads, or press Ctrl + J. Regardless how you downloaded the installer file, once you found it, you need to execute it. As shown in Figure 2.2.6, this can be accomplished by clicking on Open file in my case. The OS will now ask us whether we want to permit the downloaded program to make changes on our device. We certainly want that, because we want to use it to install PostgreSQL. So we click Yes in Figure 2.2.7.

Then, the installer begins its work with a small splash screen shown in Figure 2.2.8. In the following welcome screen, we simply click Next (Figure 2.2.9). We now work our way through setting up the installation options step-by-step. First, we can select the directory in which PostgreSQL should be

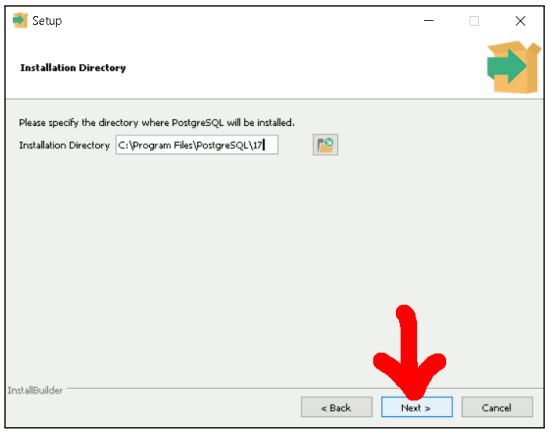


(2.2.7) When asked whether we want to allow the downloaded program to make changes on our device, we click Yes.

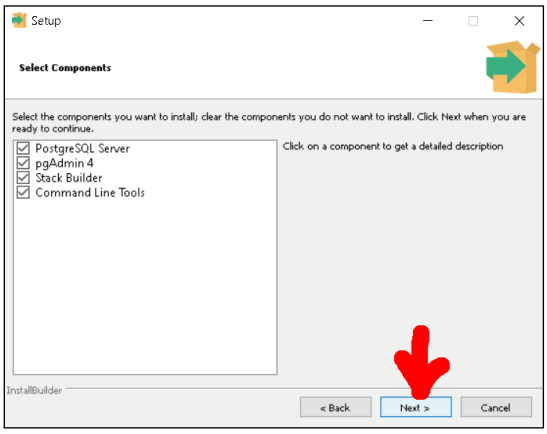
(2.2.8) Then, the installer begins its work.



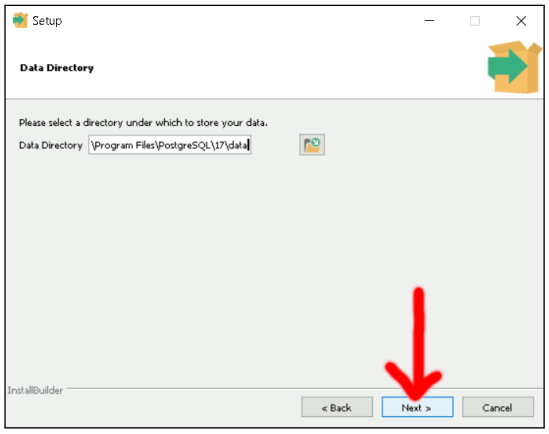
(2.2.9) In the welcome screen, we simply click Next.



(2.2.10) We can select the directory in which `PostgreSQL` should be installed. Let's leave it at the default setting and click Next.

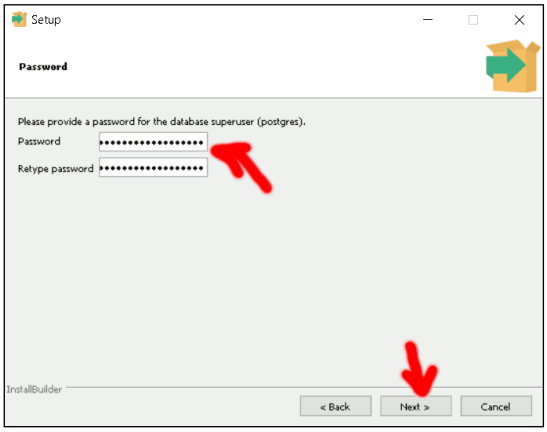


(2.2.11) We now get to the selection of what to install. Let's leave it at the default setting and click Next.

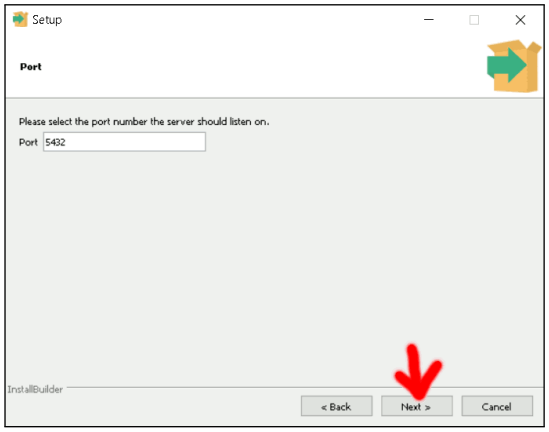


(2.2.12) We also leave the directory where the databases will be stored at the default setting and click Next.

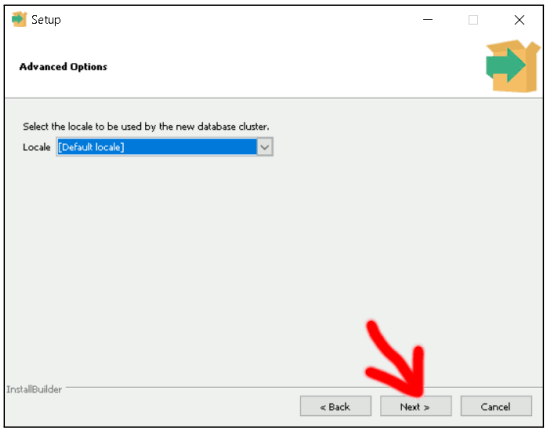
Figure 2.2: Installing and configuring `PostgreSQL` under Microsoft `Windows`.



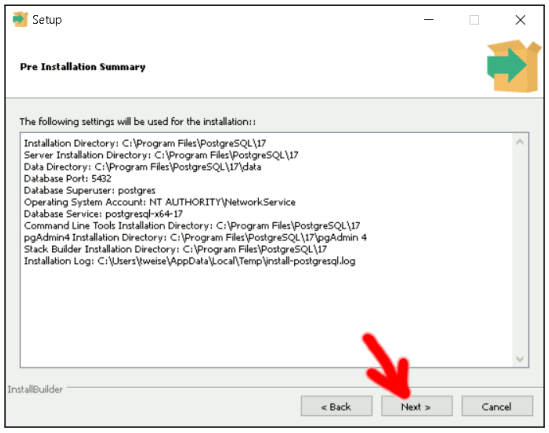
(2.2.13) We need to provide a secure password for the PostgreSQL DBMS. Please carefully choose one, enter it into both form fields, and click Next.



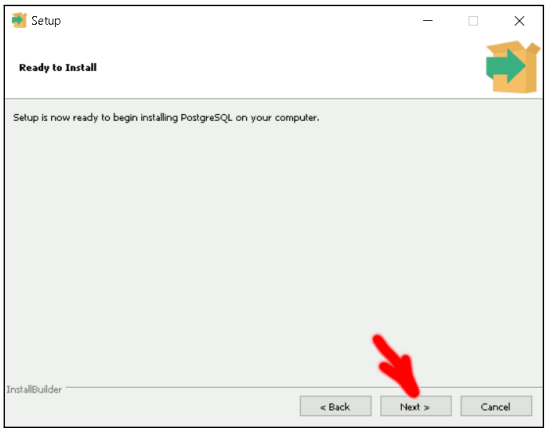
(2.2.14) We can select the port at which the PostgreSQL server will listen for incoming connections. We leave it at the default 5432 and click Next.



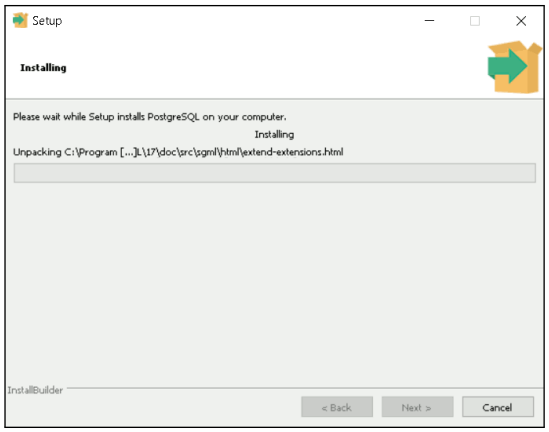
(2.2.15) The locale allows us to select country- or culture-specific default settings, e.g., for currency or number formatting. We leave it at the system default and click Next.



(2.2.16) We get informed about the components that will be installed. We click Next.

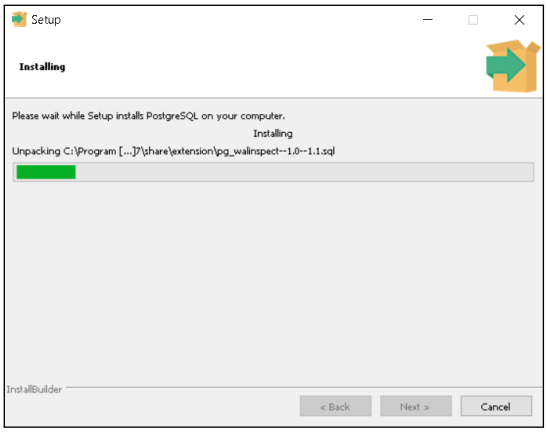


(2.2.17) We get asked whether we are ready to install. We are, so we click Next.

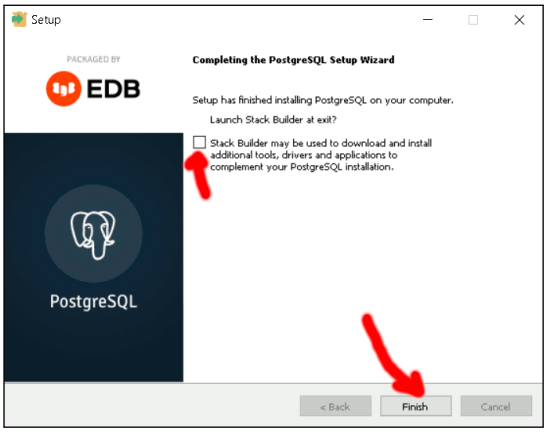


(2.2.18) The installation begins.

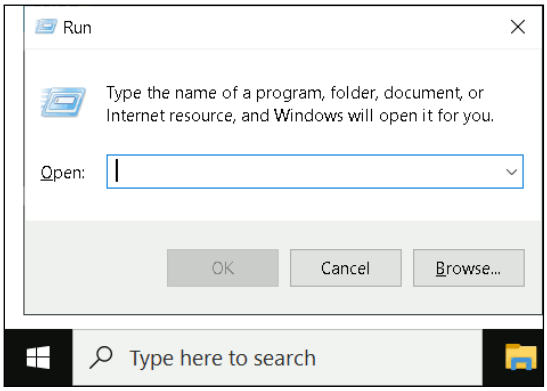
Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.



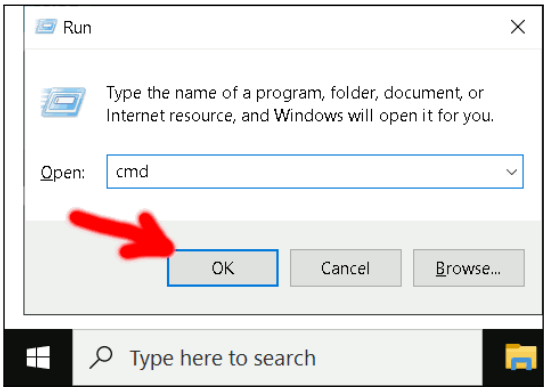
(2.2.19) The installation proceeds.



(2.2.20) Once the installation completes, we get asked whether we want to use the *Stack Builder* software to set up additional components. We do not want that, so we make sure that the checkbox is *not* checked. Then we click Finish. `PostgreSQL` is now installed.



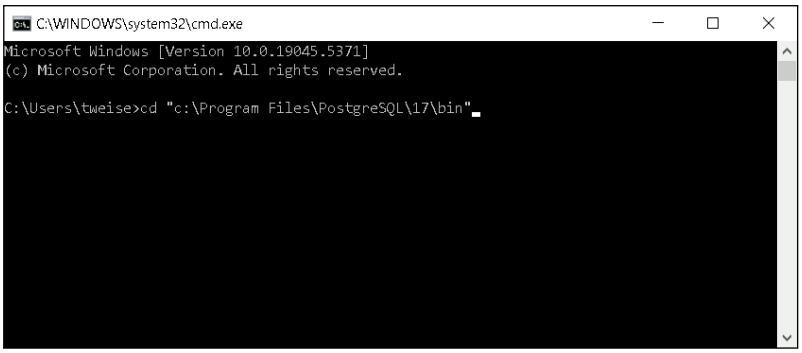
(2.2.21) To validate the installation, we need to open a terminal. We therefore press ⊞+R.



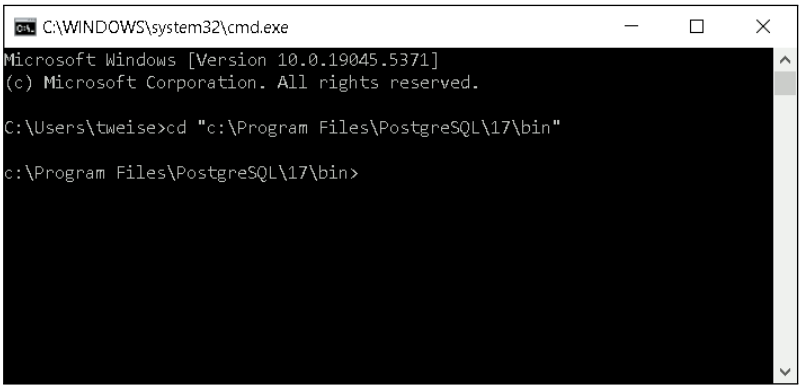
(2.2.22) We type in `cmd` and hit Enter.

(2.2.23) A new terminal window opens up.

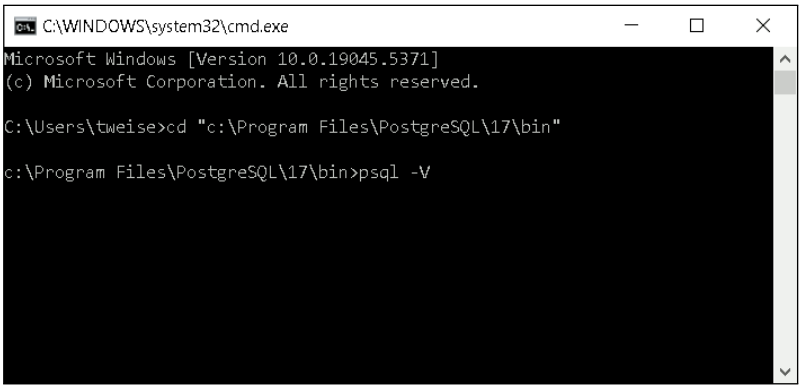
Figure 2.2: Installing and configuring `PostgreSQL` under Microsoft `Windows`.



(2.2.24) We need to enter the `bin` folder in the directory into where the PostgreSQL DBMS was installed. If we used the default settings, we can do that by typing `cd "C:\Program Files\PostgreSQL\bin"` and hitting Enter.



(2.2.25) We are now in that directory.



(2.2.26) First, we want to print the version of `psql`, the client program of PostgreSQL. We can do this by typing `psql -V` and pressing Enter.

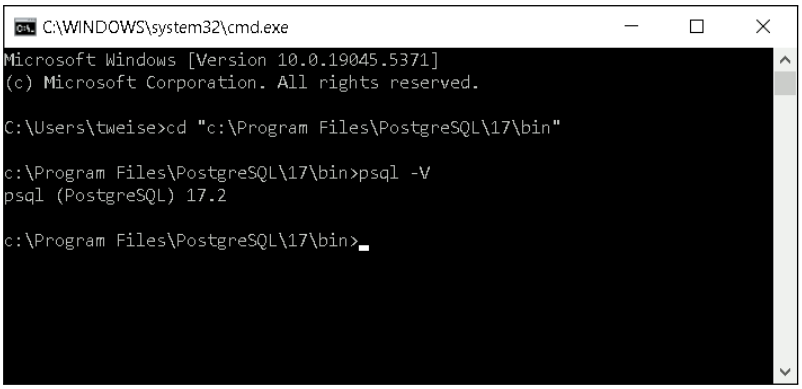
Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.

installed in Figure 2.2.10. I chose to just leave it at the default setting and click Next. This is what I will do for most of the rest of the installation, too.

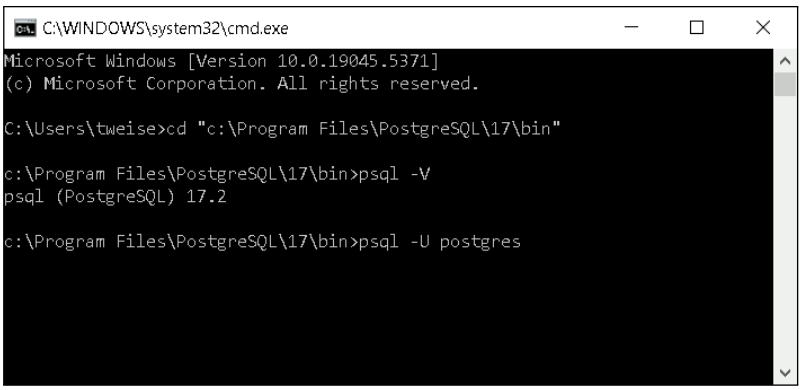
We now get to the selection of what to install. I left this at the default setting, too, and simply click Next in Figure 2.2.11. Then, in Figure 2.2.12, we can also leave the directory where the databases will be stored at the default setting and click Next.

In the following screen, we do need to provide a secure password for the PostgreSQL DBMS. Here we need to carefully choose a password *and remember it well*. We enter it into both form fields, and click Next in Figure 2.2.13.

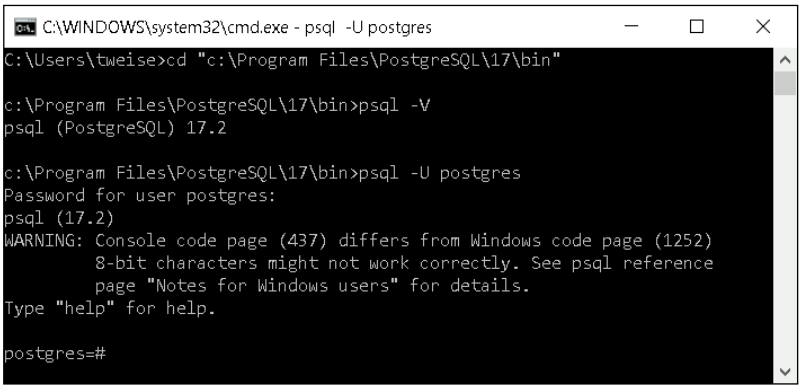
We can now select the port at which the PostgreSQL server will listen for incoming connections. If we imagine a computer network as a transportation system, then the network protocol would be the means of transportation, e.g., bus, airplaine, or train. The network address would identify the station, e.g., Hefei Station or Beijing Station. Then the *port* would be something like the platform



(2.2.27) In my case, the output shows that version 17.2 was installed.



(2.2.28) We now want to also see which version of the PostgreSQL server was installed and, by doing so, also confirm that the server is up and running correctly. We therefore launch `psql -U postgres`, i.e., start `psql` using the user (or role) `postgres`.

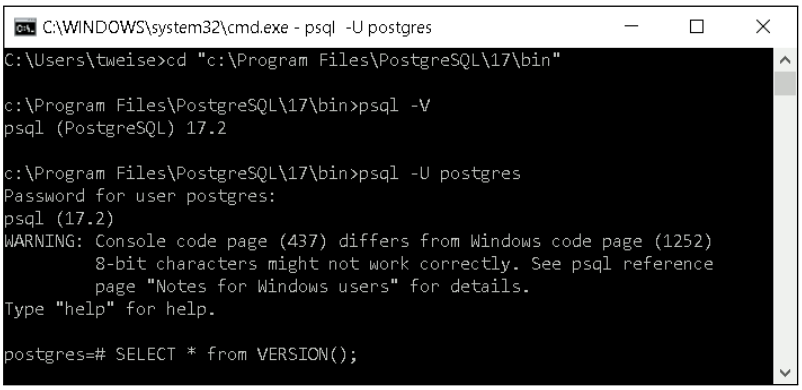


(2.2.29) When the program starts, it requires us to enter the password for the user `postgres`. This is the password that we specified during the installation in Figure 2.2.13. We enter it and press Enter. We are now in the `psql` console, and see the `postgres=#` prompt.

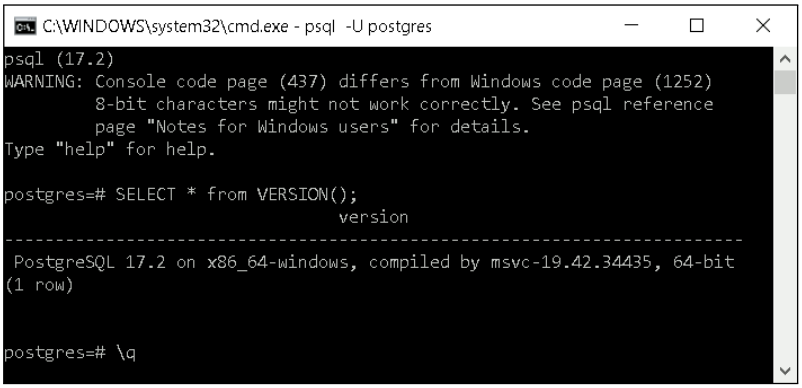
Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.

inside the station. Well, that is a very loose and very wrong analogy. Basically, the port is a number that identifies a communication partner relative to the network protocol and network address. Here, we specify the port at which the PostgreSQL server will expect incoming connections. We do not touch this parameter. We leave it at the default 5432 and click Next in Figure 2.2.14.

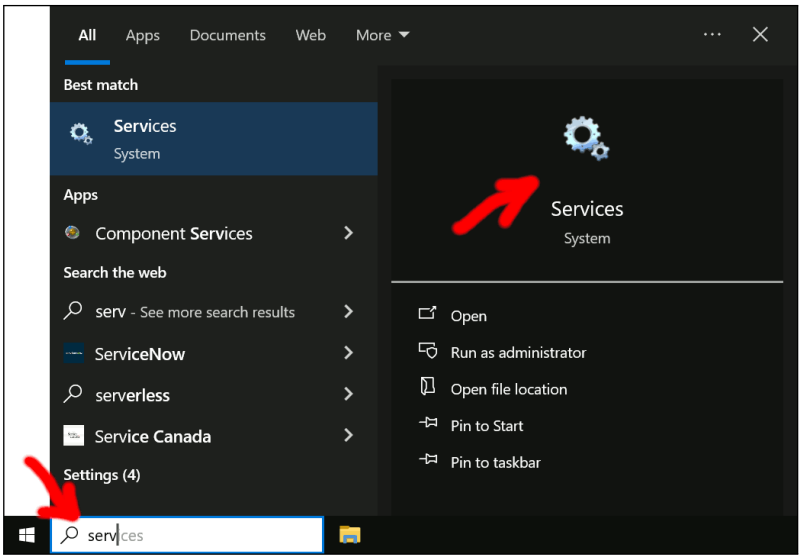
We directly encounter the next odd configuration parameter. We can choose the so-called locale. A locale identifies, basically, a region or country with specific cultural properties. This identifier is used to decide how numbers or currency or dates should be displayed. For instance, there is a crucial difference in British English and American English in how dates are written in numerical form: the former uses the month/day/year scheme, whereas the latter uses day/month/year. If PostgreSQL is supposed to print dates in the form $\cdot\cdot/\cdot\cdot/\cdot\cdot\cdot\cdot$, it has to know whether it is running on an American or British



(2.2.30) We enter the SQL command `SELECT * FROM VERSION();` and press Enter.

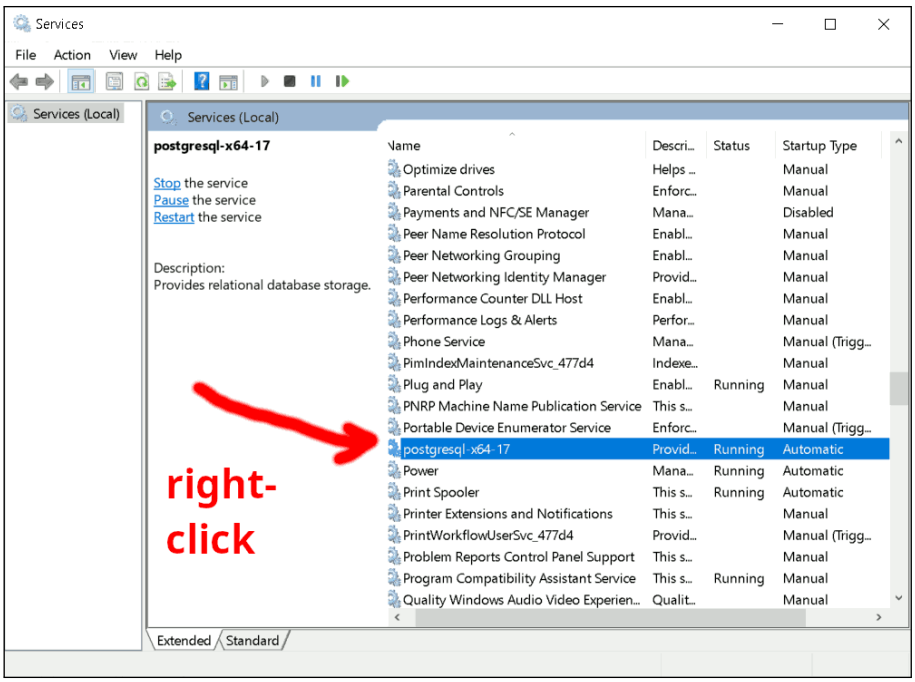


(2.2.31) The result gets printed. In my case, it shows that the PostgreSQL server also has version 17.2.We now type in \+q+Enter, which will exits `psql`.

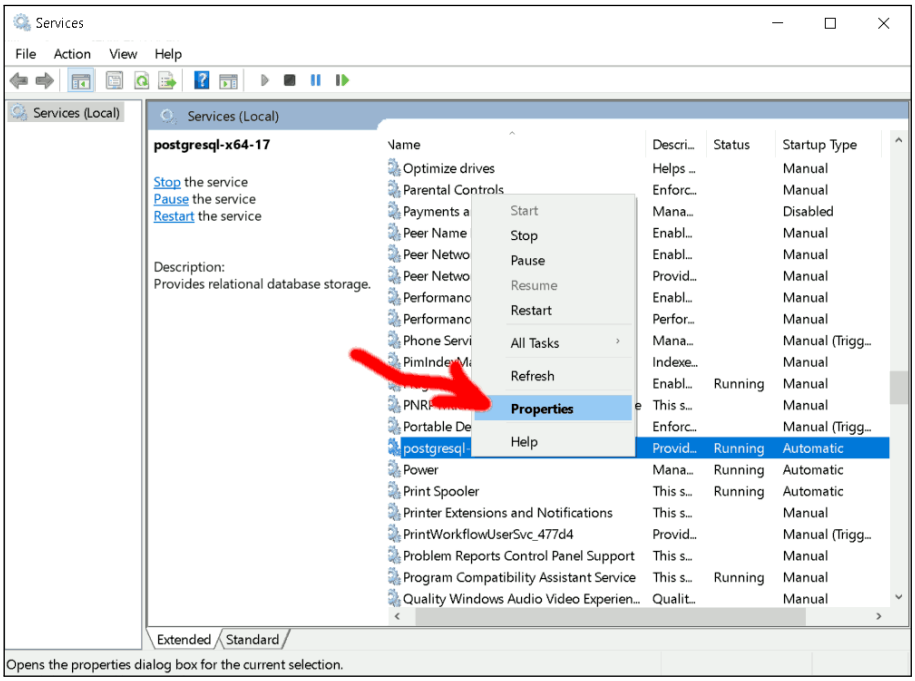


(2.2.32) Let us now explore how the PostgreSQL server is run on our Windows machine: It is executed as a service. We press ⊞, type in `services`, and click on the "gears" symbol named *Services* that appears.

Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.

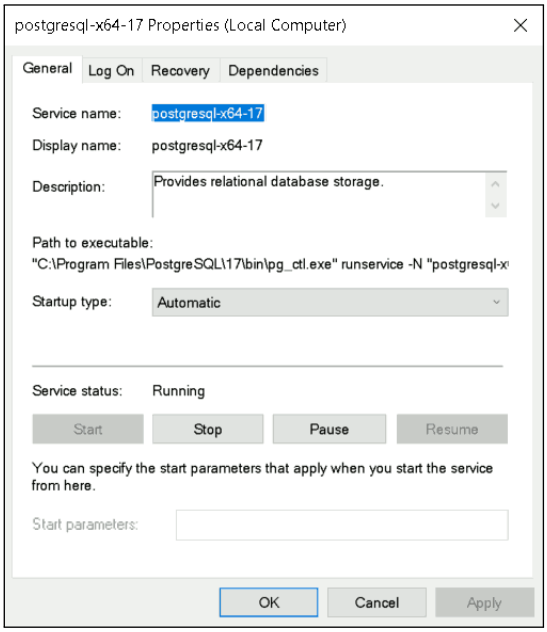


(2.2.33) The *Services* system setup window opens. Search for a service named something like `PostgreSQL`, in my case, it is `postgresql-x64-17`. Right-click on it.

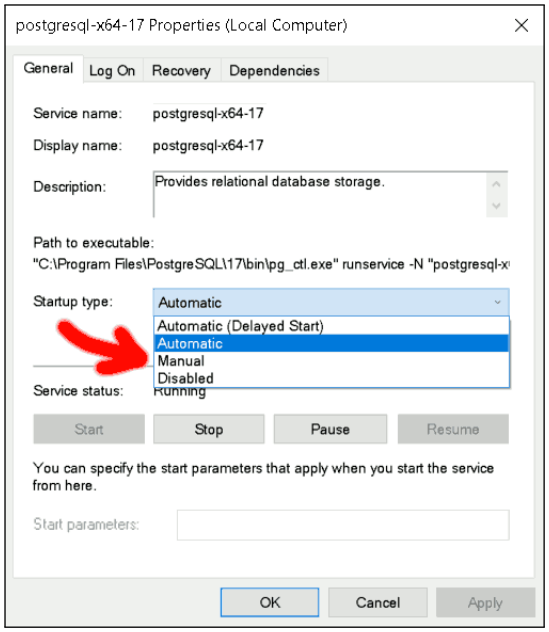


(2.2.34) In the Pop-up menu that appears, click on Properties.

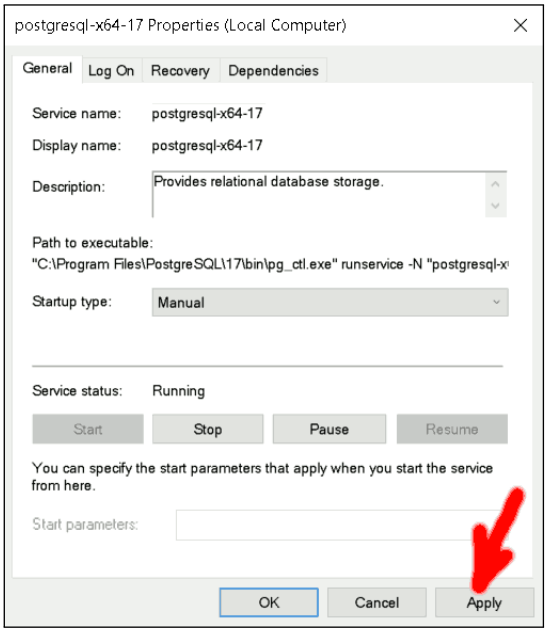
Figure 2.2: Installing and configuring `PostgreSQL` under Microsoft `Windows`.



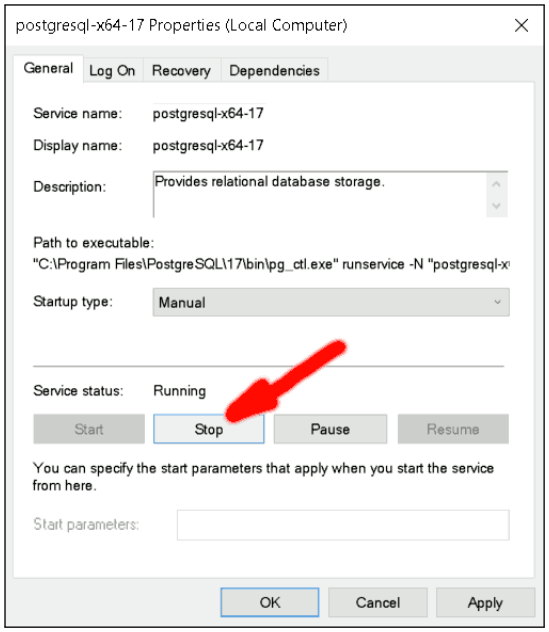
(2.2.35) The service properties dialog appears.



(2.2.36) Click on the drop-down box for Startup type:. It will be set to Automatic, which means that the PostgreSQL DBMS server will be started automatically whenever your system starts.

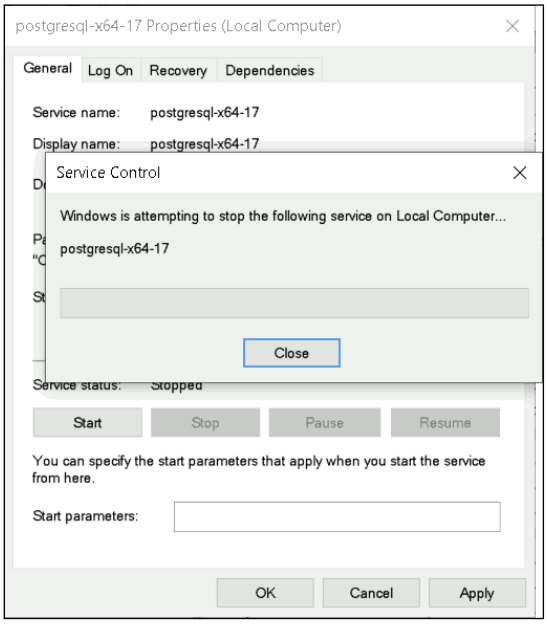


(2.2.37) The following is completely optional. You do not have to do that. If you do *not* want that, because it may make booting slower and maybe you only need the service studying for this course, you can turn off this automated startup. You would therefore select Manual as Startup type: and click the Apply button.

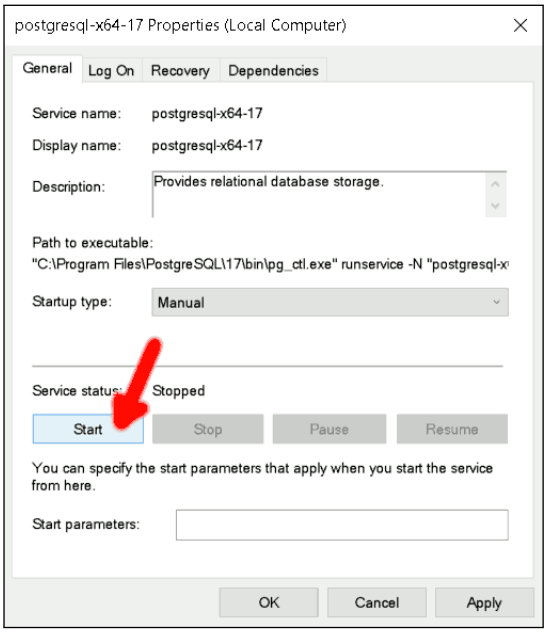


(2.2.38) The service is currently still running after that change, but it will not start automatically anymore. If you want it to start automatically again, just select Automatic as Startup type:. If you want to stop the currently-running service, click the Stop button.

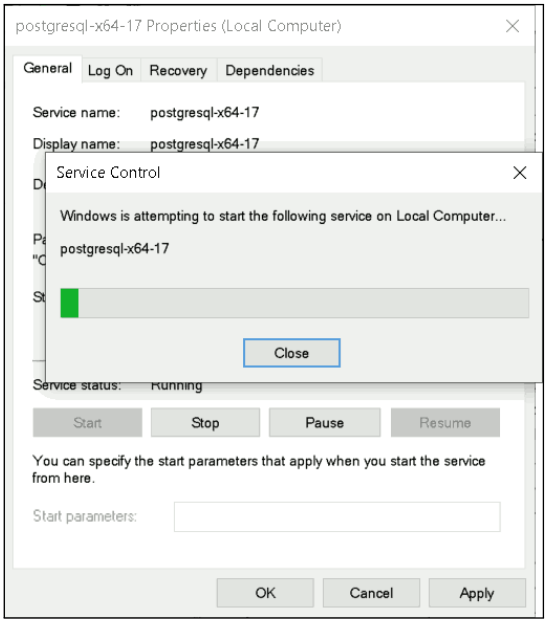
Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.



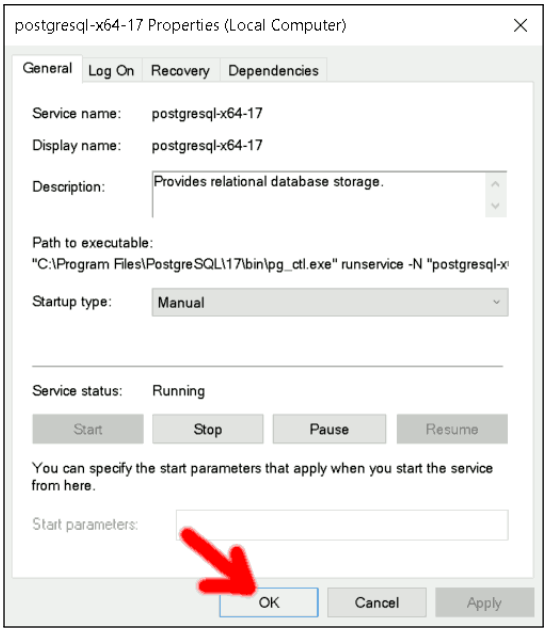
(2.2.39) Then, the service will be stopped.



(2.2.40) It is now no longer running. Let's start it again by clicking the Start button.



(2.2.41) The service is now starting again.



(2.2.42) We click OK to leave the dialog.

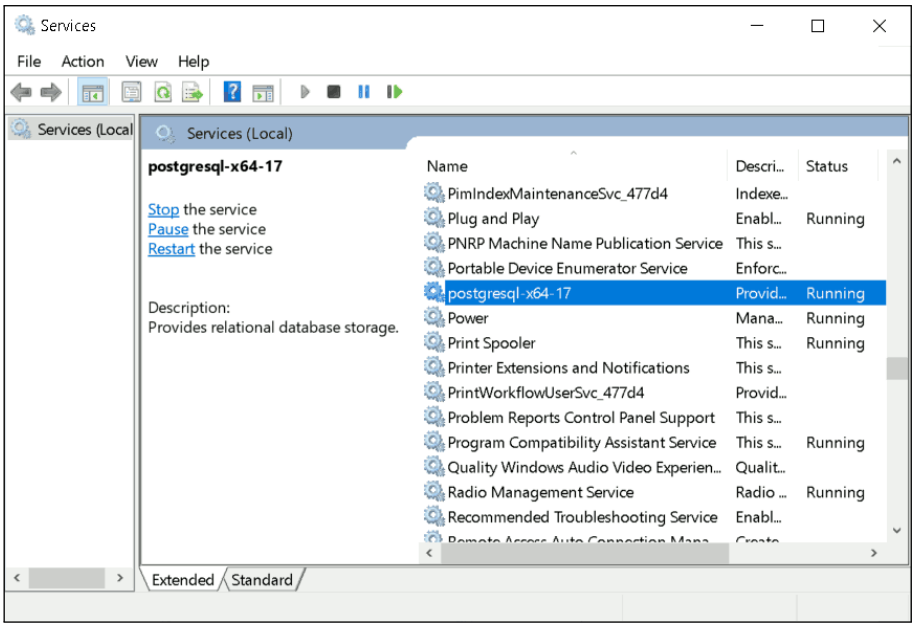
Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.

English PC. Well, we leave it at the system default, which should be OK for use on our own PC, and click Next in Figure 2.2.15.

Now we get informed about the components that will be installed. We click Next in Figure 2.2.16. We get asked whether we are ready to install. We are, so we click Next in Figure 2.2.17. The installation begins and proceeds, as sketched in Figures 2.2.18 and 2.2.19.

Once the installation completes, we get asked whether we want to use the *Stack Builder* software to set up additional components. We do not want that, so we make sure that the checkbox is *not* checked. Then we click Finish in Figure 2.2.20. At this point, both the DBMS server and the client `psql` of PostgreSQL are installed.

To validate the installation, we need to open a terminal. We therefore press ⊞+R, type in `cmd`,



(2.2.43) We see that the service is *Running* and in mode *Manual* (if we changed it to that mode). When you shutdown your system, the DBMS is stopped. It does not start automatically when you boot (unless you set *Automatic* as *Startup type*). So if you need it, you would again enter the *Services* system setup program and manually start it.

Figure 2.2: Installing and configuring PostgreSQL under Microsoft Windows.

and hit ↵ in Figures 2.2.21 and 2.2.22. A new terminal window opens up in Figure 2.2.23. We here can write text-based commands and execute programs in this window.

To work with our new PostgreSQL installation, we first need to change into its installation folder and then into the folder where its executables are located. In other words, we need to enter the `bin` folder in the directory into where the PostgreSQL DBMS was installed. If we used the default settings, we can do that by typing `cd "C:\Program Files\PostgreSQL\bin"` and hitting Enter in Figure 2.2.24. We are now in that directory in Figure 2.2.25.

First, we want to print the version of `psql`, the client program of PostgreSQL. `psql` is the tool that we will use to communicate with the DBMS. We can get its version by typing `psql -V` and pressing Enter in Figure 2.2.26. In my case, the output shows that version 17.2 was installed, as illustrated in Figure 2.2.27.

We now want to also see which version of the PostgreSQL server was installed. By getting this information, we will also confirm that the server is up and running correctly. We therefore launch `psql -U postgres`, i.e., start `psql` using the user (or role) `postgres` in Figure 2.2.28. `postgres` is the username for the administrative account of our DBMS.

When the program starts with these parameters, it requires us to enter the password for this user (`postgres`). This is the password that we specified during the installation in Figure 2.2.13. We enter it and press Enter. We are now in the `psql` console, and see the `postgres=#` prompt in Figure 2.2.29.

We enter the SQL command `SELECT * FROM VERSION();` and press Enter in Figure 2.2.30. This is actual a command in the SQL language mentioned in the history section before. We will learn lots and lots of that later on. For now, you do not need to understand it. You only need to know that this line will print the version of the server for us. And the version gets indeed printed: In my case, it shows that the PostgreSQL server also has version 17.2. We now type in \ + q + Enter, which will exits `psql` in Figure 2.2.31. This will take us back to the normal Windows terminal. We no longer have any use for it and can close it.

We now have a PostgreSQL DBMS server running on our system. What does that mean? How is it running? What if I restart my computer? Will it still be running? Will it always be running? Let us investigate these questions.

The PostgreSQL server is executed as a service in our OS. So we will switch over to the *Services* configuration window of Windows. We therefore press ⊞, type in `services`, and click on the

"gears" symbol named *Services* that appears in Figure 2.2.32. The *Services* system setup window opens. We scroll down and search for a service named something like PostgreSQL, in my case, it is postgresql-x64-17. We right-click on this service, i.e., we click with the mouse button on the right, in Figure 2.2.33. In the Pop-up menu that appears, click on Properties in Figure 2.2.34.

This opens the service properties dialog for PostgreSQL in Figure 2.2.35. We click on the drop-down box for Startup type: in Figure 2.2.36. Here we see the different options regarding whether and how a service is started. Right now it is set to Automatic, which means that the PostgreSQL DBMS server will be started automatically whenever your system starts.

The following is completely optional. You do not have to do that. If you do *not* want that the service always starts automatically, because it may make booting slower and maybe you only need the service studying for this course, you can turn off this automated startup. Maybe you want that the server is normally turned off. Whenever you need it, you want to turn it on by yourself. If that is your goal, then you would probably want to select Manual as Startup type: and click the Apply button, as shown in Figure 2.2.37. This means that, the next time your system boots, PostgreSQL will not be started automatically anymore.

The service is currently still running after we made that change, but it will not automatically start anymore. If you want it to start automatically start again, just select Automatic as Startup type:. If you want to stop the currently-running service, click the Stop button, as shown in Figure 2.2.38. Then, the service will be stopped, as you can see in Figure 2.2.39. After that, it is no longer running

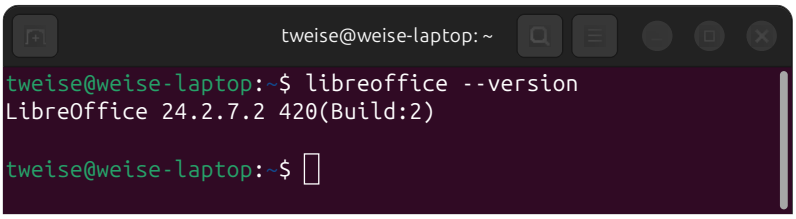
Let's start it again by clicking the Start button in Figure 2.2.40. The service is now starting again, as illustrated in Figure 2.2.41. We click the OK button to leave the dialog in Figure 2.2.42. We see that the service is *Running* and in mode *Manual* (if we changed it to that mode) in Figure 2.2.43. When you shutdown your system, the DBMS is stopped. If it is in *Manual* mode, the DBMS does not start automatically when you boot. It you set it back to *Automatic* as *Startup type*, then it will start automatically again. So if you need it, you would again enter the *Services* system setup program and manually start it.

With this we depart from the magical world of PostgreSQL server installation under Windows. Our work here is done.

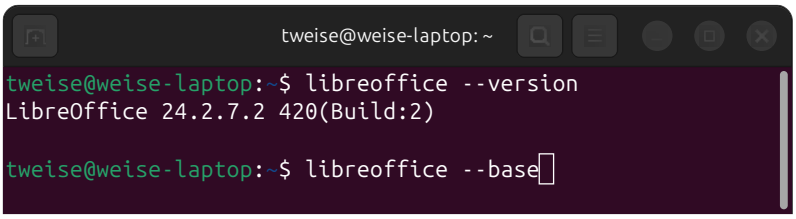
Chapter 3

Installing LibreOffice

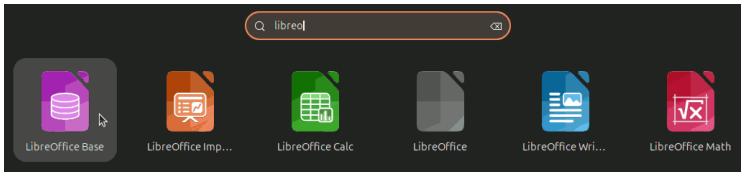
In our excursion on the available database management system in Section 1.3, we mentioned that the open source office suite LibreOffice provides program named LibreOffice Base. LibreOffice Base can be used as either a stand-alone DBMS or as a nice user interface that can connect to other databases [62, 123]. It has a functionality similar to Microsoft Access [18, 37, 140], but it is free. We will therefore use this software to connect to PostgreSQL. Let us therefore discuss how LibreOffice [1, 68, 123] can be installed.



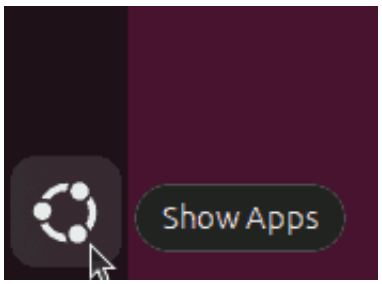
(3.1.1) Open a terminal via with Ctrl + Alt + T, then type `libreoffice --version` and hit Enter. On my system, LibreOffice 24.2.7.2 is installed.



(3.1.2) We can start LibreOffice Base from the terminal by typing `libreoffice --base` and hitting Enter.

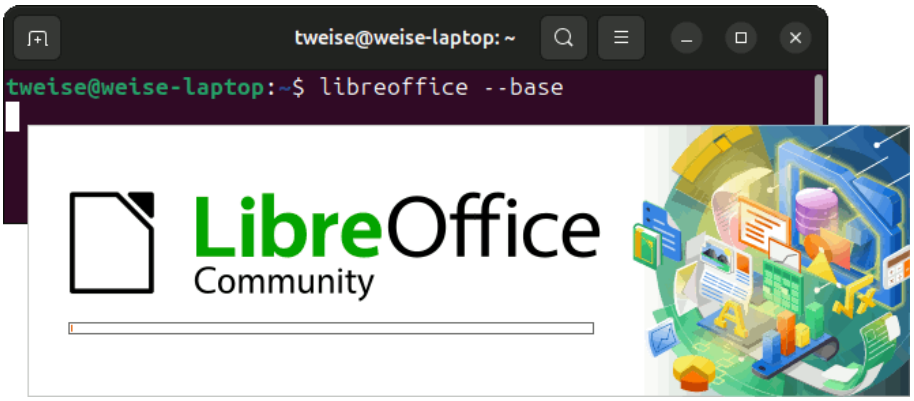


(3.1.3) Or we can open the dash by hitting ⊞, type `libreoffice`, and then clicking on the symbol for LibreOffice Base.

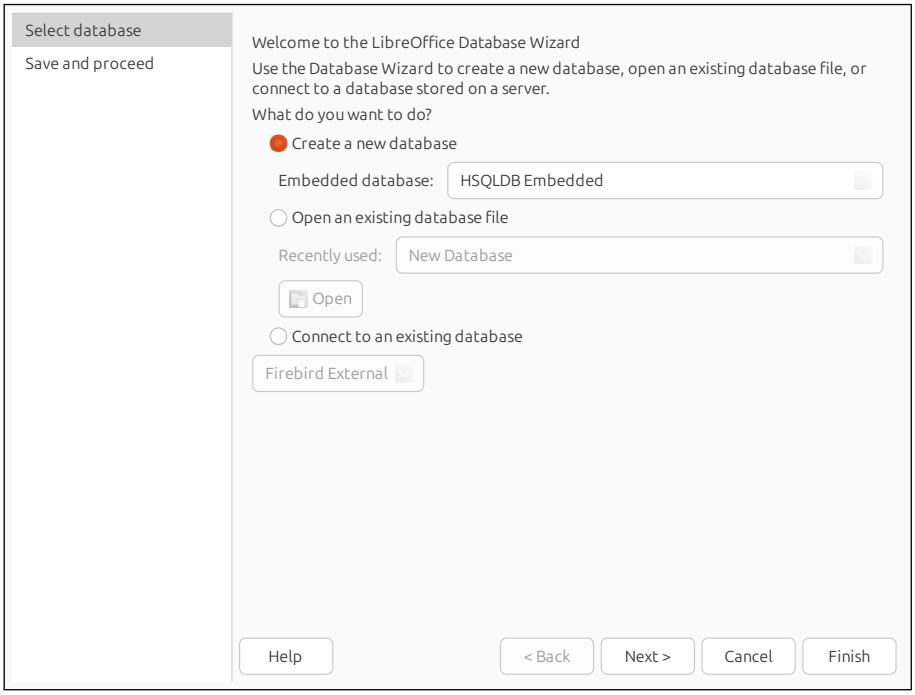


(3.1.4) We can also open the dash by clicking on the little Ubuntu symbol at the bottom-left corner of the screen.

Figure 3.1: Starting LibreOffice Base under Ubuntu if it is already installed.



(3.1.5) The startup screen appears.



(3.1.6) The initial form of LibreOffice Base appears. We are done for now and close the program.

Figure 3.1: Starting LibreOffice Base under Ubuntu if it is already installed.

3.1 Installing LibreOffice under Ubuntu Linux

If you are a user of Ubuntu Linux, then LibreOffice and, hence, LibreOffice Base, come already pre-installed on your machine. You do not actually need to do anything.

To confirm that LibreOffice is indeed installed, we open a terminal via with Ctrl + Alt + T. We type `libreoffice --version` and hit Enter. On my system, the result in Figure 3.1.1 shows that LibreOffice 24.2.7.2 is installed.

We can also start LibreOffice Base directly from the terminal by typing `libreoffice --base` and hitting Enter, as show in Figure 3.1.2. Alternatively, we can open the dash by hitting ⊞, type `libreoffice`, and then clicking on the symbol for LibreOffice Base, as illustrated in Figure 3.1.3. If you do not want to use a hotkey to open the dash, you can also click on the little Ubuntu symbol at the bottom-left corner of the screen (see Figure 3.1.4). Regardless of whether you opened LibreOffice Base via a terminal or the dash, the startup screen given in Figure 3.1.5 will appears. Then, the initial form of LibreOffice Base is displayed as shown in Figure 3.1.6. We are done for now and close the program.

In the unlikely case that LibreOffice was not installed on your system, the above will obviously not work. However, Figure 3.2.1 shows that we can easily install it by typing `sudo apt-get install libreoffice` into a terminal window that we opened with Ctrl + Alt + T

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ sudo apt-get install libreoffice
```

(3.2.1) In the unlikely case that LibreOffice was not installed, we can install it by typing `sudo apt-get install libreoffice` into a terminal window that we opened with Ctrl+Alt+T and hit Enter.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ sudo apt-get install libreoffice
[sudo] password for tweise:
```

(3.2.2) This requires sudo privileges, so we need to enter the super user password.

```
tweise@weise-laptop: ~
tweise@weise-laptop:~$ sudo apt-get install libreoffice
[sudo] password for tweise:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libreoffice is already the newest version (4:24.2.7-0ubuntu0.24.
04.1).
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
tweise@weise-laptop:~$
```

(3.2.3) Now LibreOffice could be installed. On my system, it is already installed. So nothing happens.

Figure 3.2: Installing LibreOffice under Ubuntu.

and hit Enter. This requires sudo privileges, so we need to enter the super user password in Figure 3.2.2. Now you would probably be told how much data will need to be downloaded, then asked whether you are OK with that, and then LibreOffice would be installed. Since it is already installed on my system, nothing happens. as shown in Figure 3.2.3.

The gist is that LibreOffice is usally already installed on Ubuntu Linux or if not, can easily be installed.

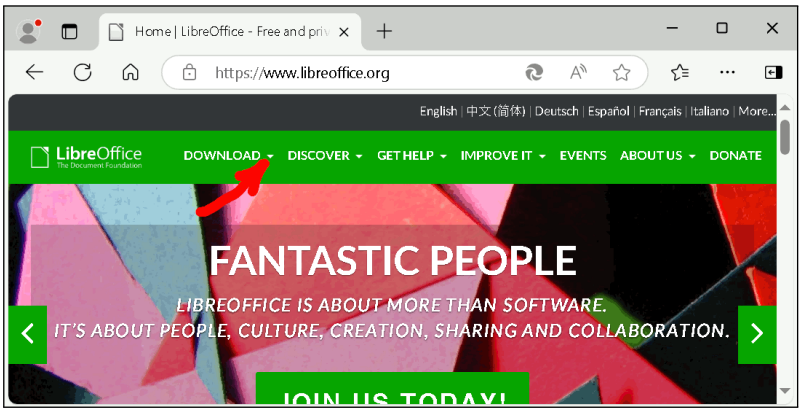
3.2 Installing LibreOffice under Microsoft Windows

Installing LibreOffice under Microsoft Windows requires us to first download the software and then to install it. We therefore open a web browser and go to https://libreoffice.org in Figure 3.3.1. There, we click on DOWNLOAD. A drop-down menu opens in Figure 3.3.2. We click on Download LibreOffice. This takes us to the download page, where we, again, click on DOWNLOAD in Figure 3.3.3. On this page, we could probably select the operating system of our choice. Unless you are using some other operating system, the default choice, Windows (64-bit), is probably correct and we can leave it as is. This takes us to yet another download page in Figure 3.3.4, where we click the big button for downloading LibreOffice. Finally, the download starts in Figure 3.3.5.

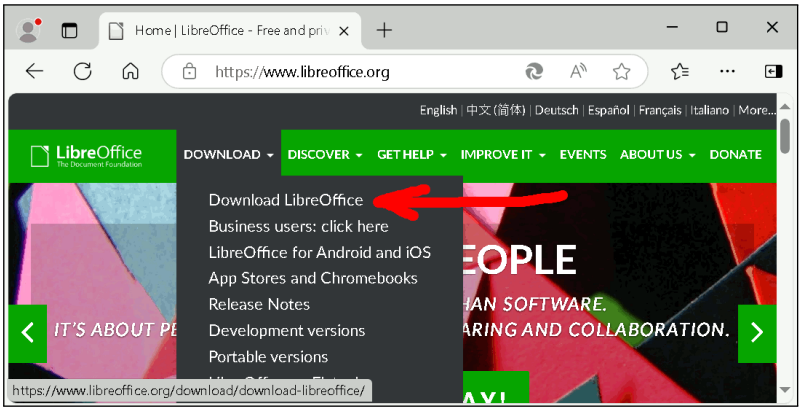
After the download completes, we click Open file in Figure 3.3.6. A Windows loading screen appears in Figure 3.3.7. You see, we downloaded a new program from the internet and not the Microsoft Store. So this worries our Windows installation, and it asks whether we *really* want to install this program in Figure 3.3.8. In Figure 3.3.9, we click on Install anyway.

The installation wizard window appears and we click Next in Figure 3.3.10. On the next screen, we can choose whether we want to customize the installation of LibreOffice or, instead, would prefer a "typical". We indeed prefer the "typical installation" and click Next in Figure 3.3.11. In the next screen in Figure 3.3.12 we confirm that we are also OK with a shortcut on our desktop and click Install.

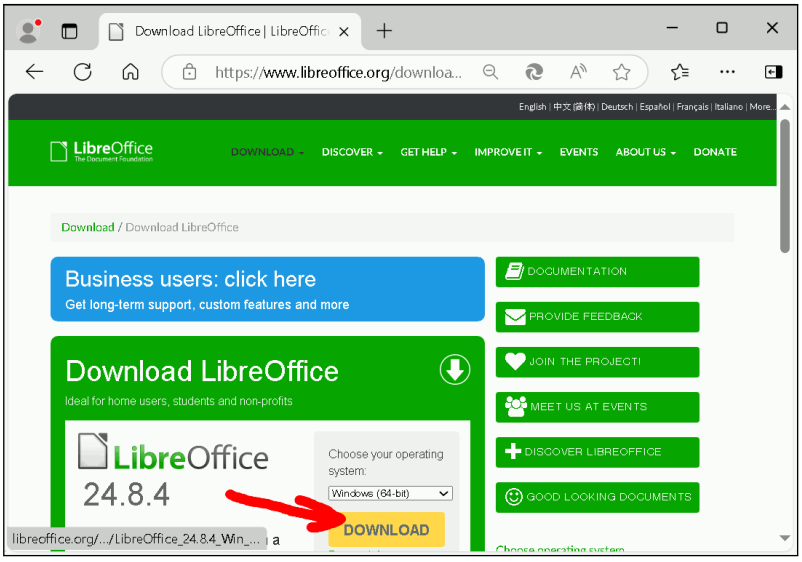
Now The installation begins (see Figure 3.3.13). Since the installer tries to modify our system, Windows asks us whether we would like to permit the installer to make changes to our system. We indeed are OK with that, so we click Yes in Figure 3.3.14. The installation continues in Figure 3.3.15.



(3.3.1) We open a web browser and go to `https://libreoffice.org`. There, we click on DOWNLOAD.

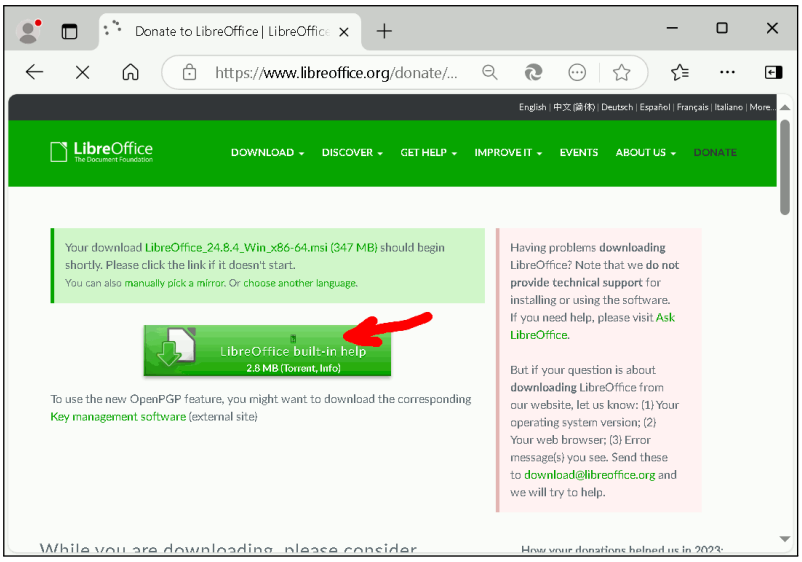


(3.3.2) In the menu that opens, we click on Download LibreOffice.

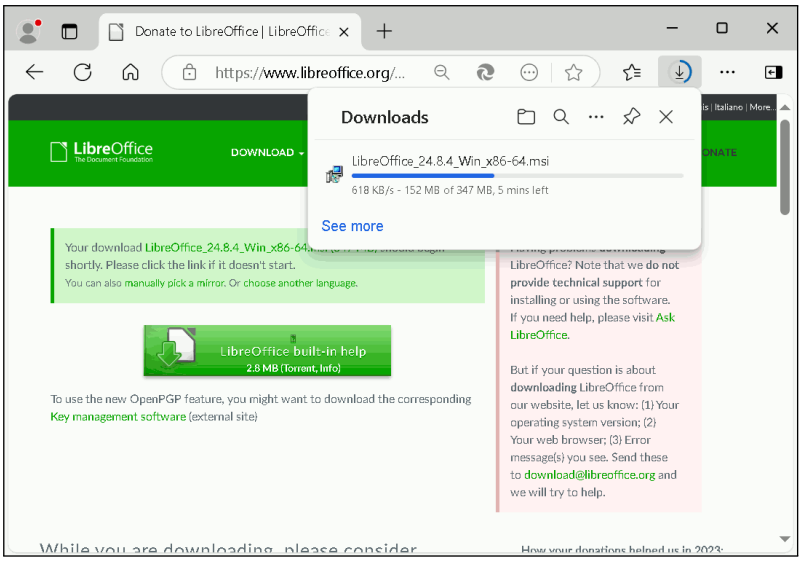


(3.3.3) This takes us to the download page, where we click DOWNLOAD. (We can leave the operator system at the default Windows (64-bit) setting.)

Figure 3.3: Installing LibreOffice under Microsoft `Windows`.



(3.3.4) This takes us to yet another download page, where we click the big button for downloading LibreOffice.



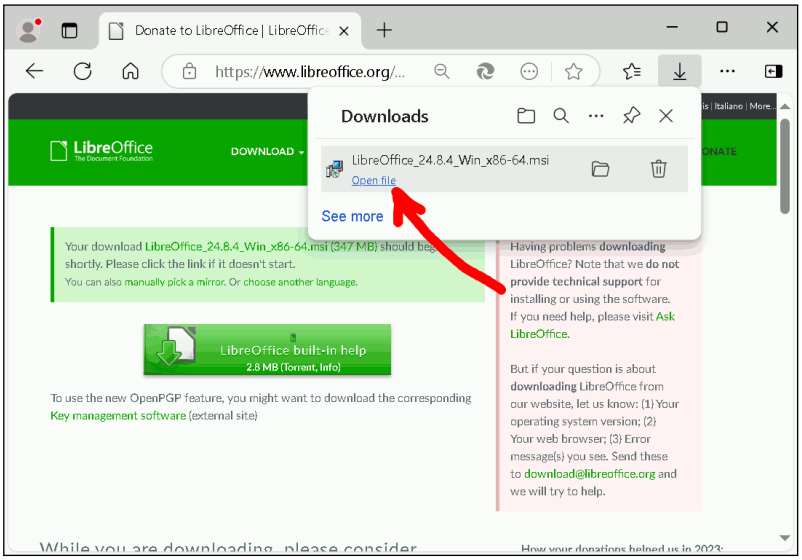
(3.3.5) The download starts.

Figure 3.3: Installing LibreOffice under Microsoft `Windows`.

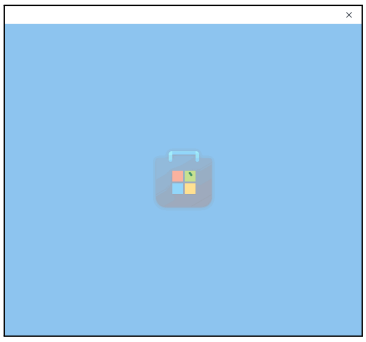
Under some circumstances, e.g., if you have the Acrobat Reader installed, it may be necessary that the installer does some complex updating. It is best to keep the option *Do not close applications. A reboot will be required to complete the setup.* I also tried the other option, but that leads to errors down the line. It is best to leave this at the default setting. So we just click OK in Figure 3.3.16. On your system, maybe this screen does not appear. Maybe you do not have Adobe Acrobat installed, maybe you have a different version or setup. Thus, maybe you will never see this screen and, as a result, do not need to reboot later. If so, good for you. If you see the screen, then just accept that you will need to reboot eventually.

Either way, the installation continues in Figure 3.3.17. And eventually, it is completed and we click Finish in Figure 3.3.18. At this stage, if you did see the screen from Figure 3.3.16, it becomes necessary to reboot. If so, you should close all other programs and click Yes, as shown in Figure 3.3.19 Then, the restart screen appears in Figure 3.3.20.

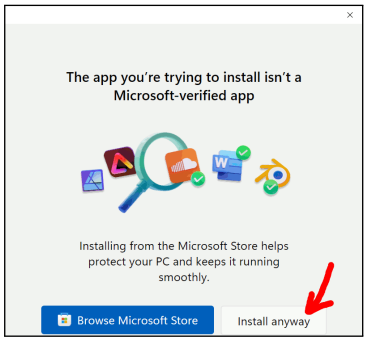
Regardless whether you needed to reboot or not, we can now find a LibreOffice icon on the desktop, as shown in Figure 3.3.21. We click on it. The LibreOffice splash screen appears in Figure 3.3.22. We may get asked to set LibreOffice as default program to open some file types. In order to not mess with your existing system configuration, we un-check the *Perform check on startup* box and click on Cancel



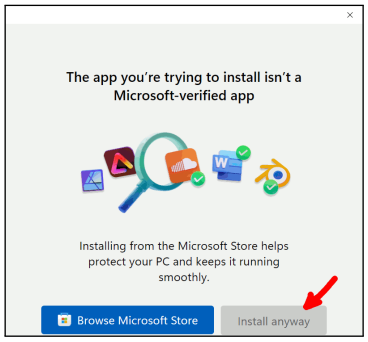
(3.3.6) After the download completes, we click Open file.



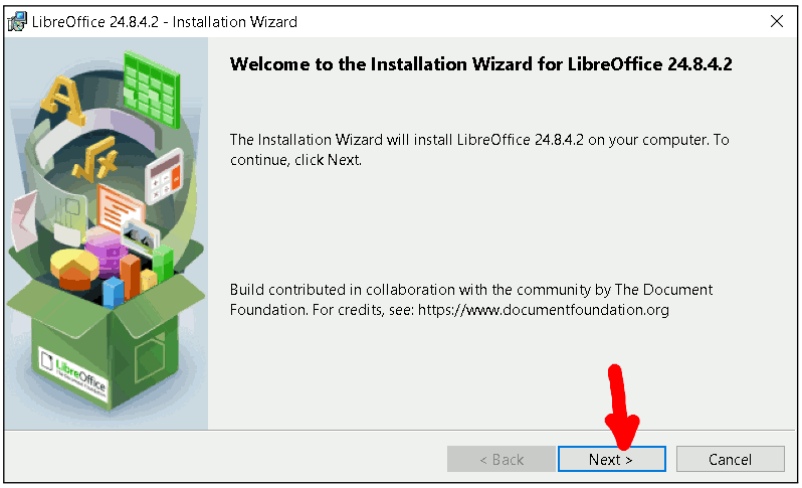
(3.3.7) A Windows loading screen appears.



(3.3.8) We get asked whether we really want to install this program. . .

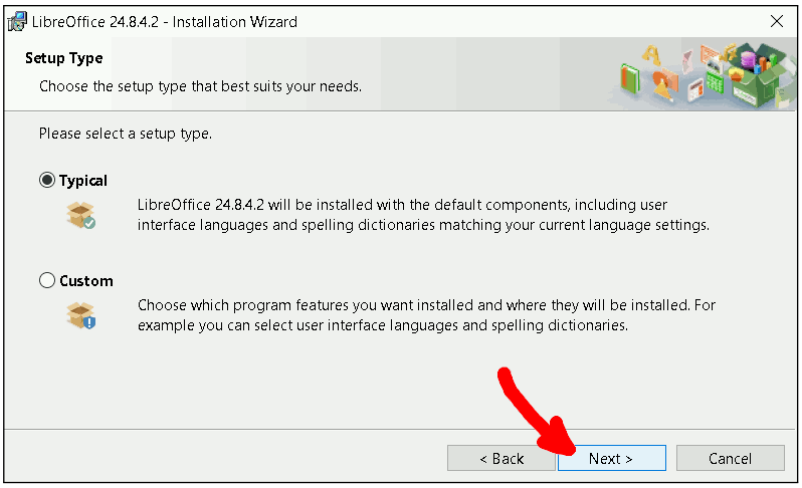


(3.3.9) . . . and we click Install anyway.

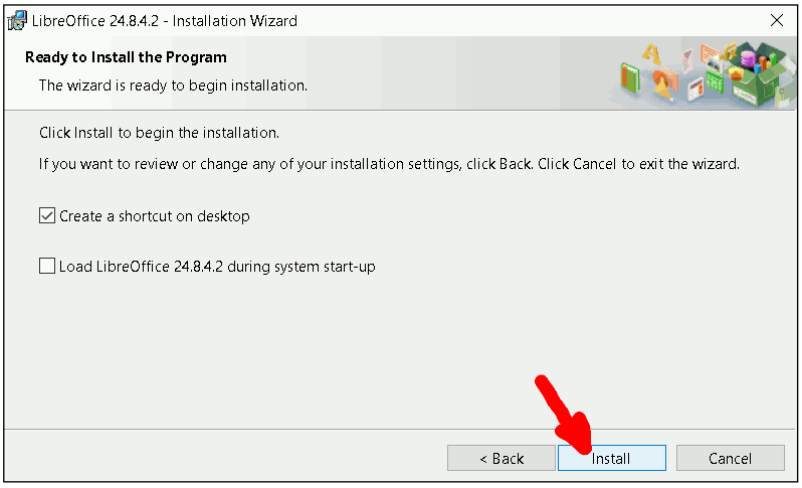


(3.3.10) The installation wizard window appears and we click Next.

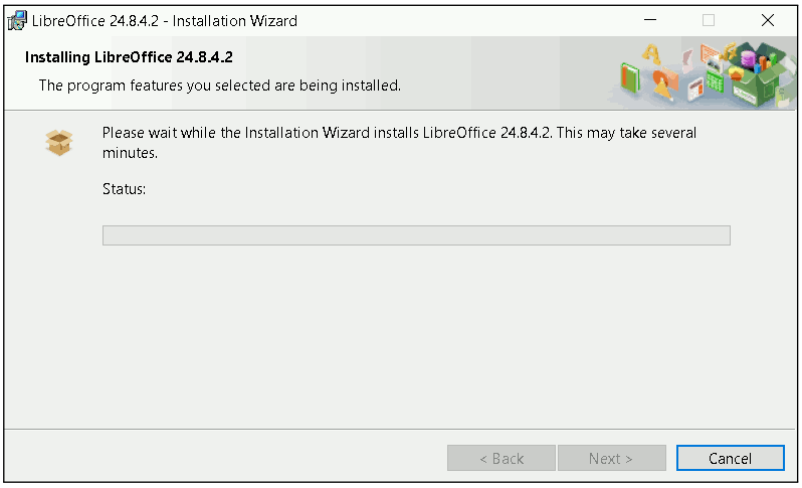
Figure 3.3: Installing LibreOffice under Microsoft Windows.



(3.3.11) We are totally fine with a "typical" installation and click Next.

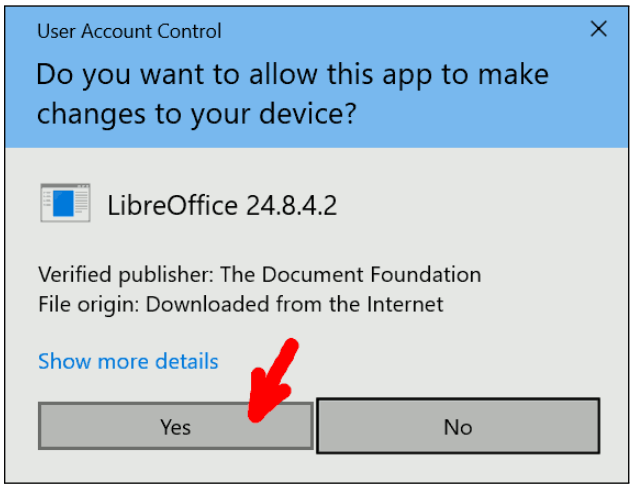


(3.3.12) We are also OK with a shortcut on our desktop and click Install.

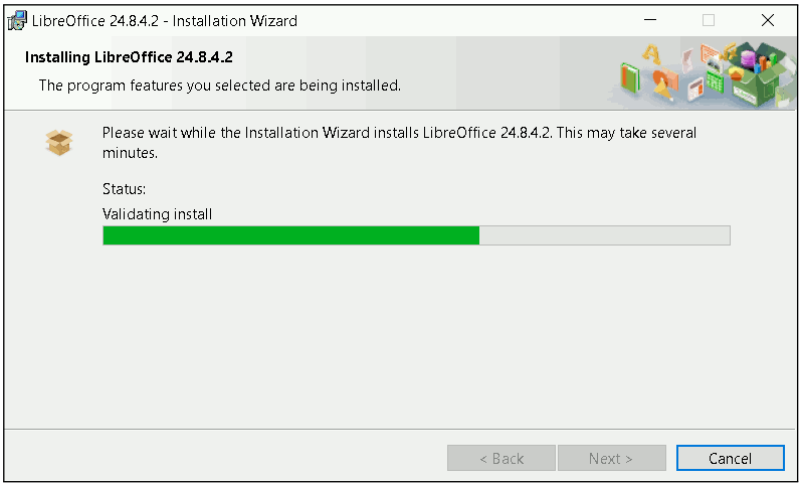


(3.3.13) The installation begins.

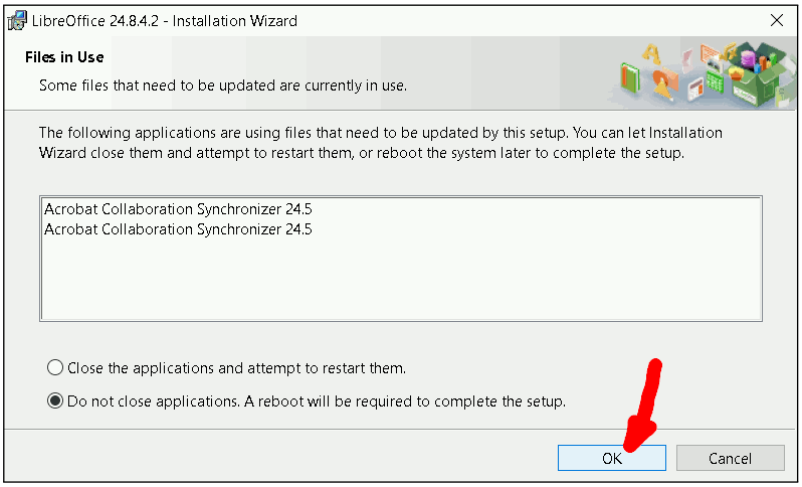
Figure 3.3: Installing LibreOffice under Microsoft `Windows`.



(3.3.14) `Windows` asks us whether we would like to permit the installer to make changes to our system. Yes, we are, so we click Yes.

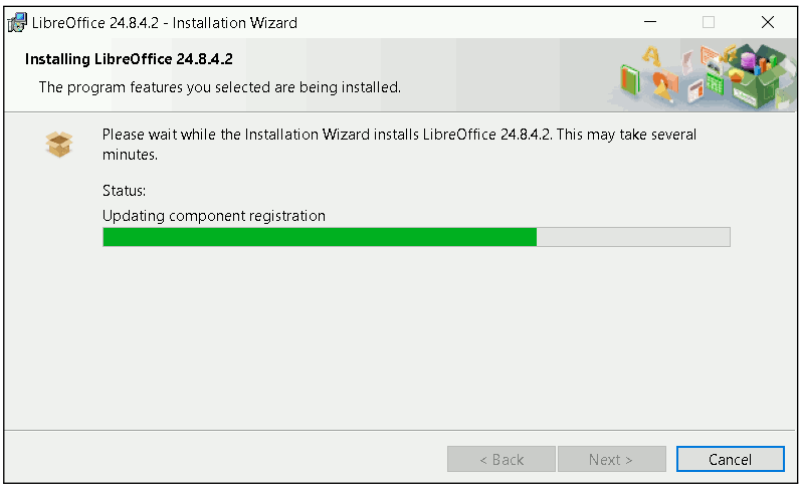


(3.3.15) The installation continues.

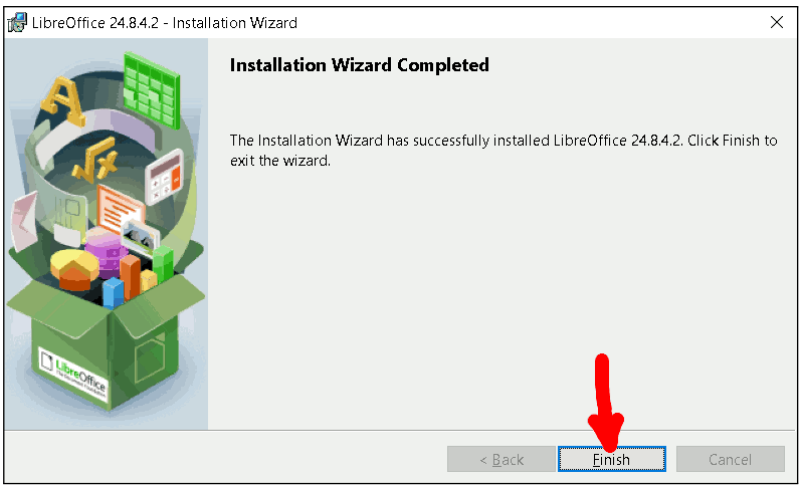


(3.3.16) Under some circumstances, e.g., if you have the Acrobat Reader installed, it may be necessary that the installer does some complex updating. It is best to keep the option *Do not close applications. A reboot will be required to complete the setup.* So we just click OK. On your system, maybe this screen does not appear.

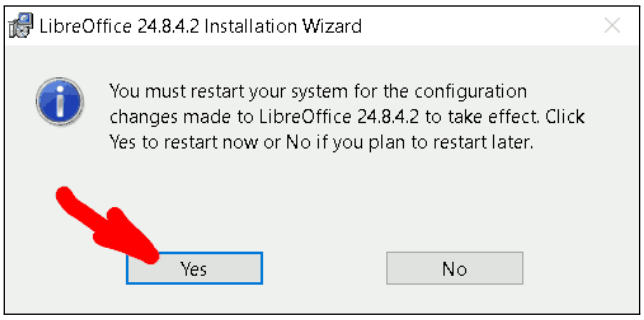
Figure 3.3: Installing LibreOffice under Microsoft `Windows`.



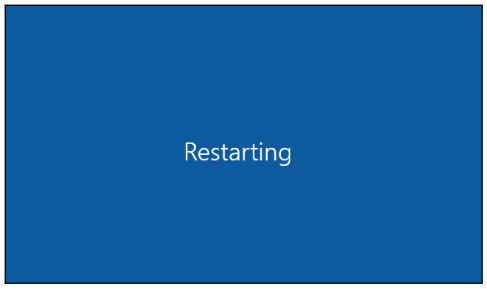
(3.3.17) The installation continues.



(3.3.18) The installation is completed. We click Finish.

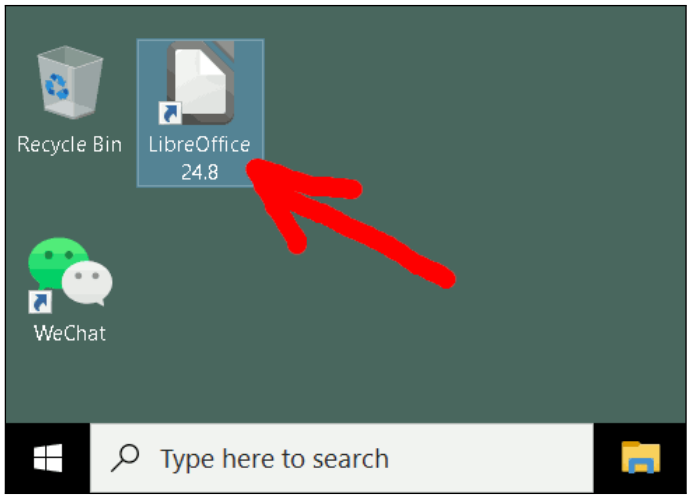


(3.3.19) It may be necessary to reboot (see Figure 3.3.16). If so, close all other programs and click Yes.



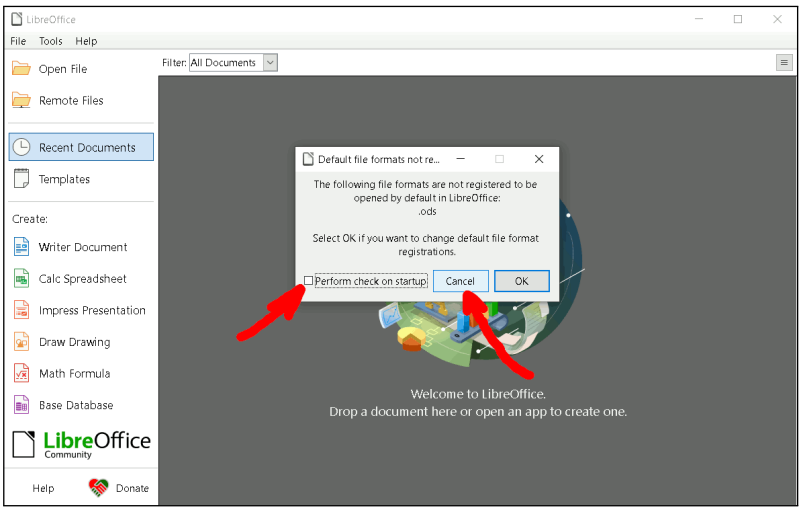
(3.3.20) The restart screen appears

Figure 3.3: Installing LibreOffice under Microsoft `Windows`.



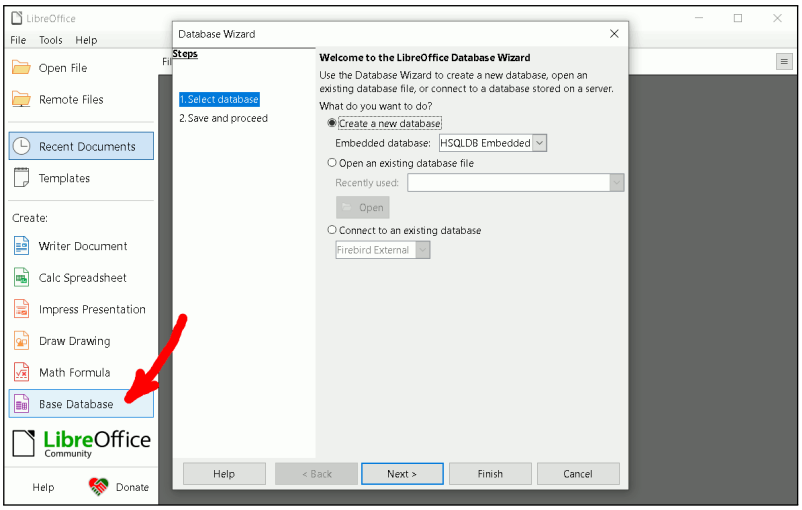
(3.3.21) We can now find a LibreOffice icon on the desktop and click on it.

(3.3.22) The LibreOffice splash screen appears.



(3.3.23) We may get asked to set LibreOffice as default program to open some file types. In order to not mess with your existing system configuration, we un-check the *Perform check on startup* box and click Cancel.

Figure 3.3: Installing LibreOffice under Microsoft `Windows`.



(3.3.24) We click on the Base Database icon in the menu bar on the left-hand side and arrive in the LibreOffice Base welcome screen. We are done here for now and close the program.

Figure 3.3: Installing LibreOffice under Microsoft Windows.

in Figure 3.3.23.

We now click on the Base Database icon in the menu bar on the left-hand side and arrive in the LibreOffice Base welcome screen in Figure 3.3.24. We are done here for now and close the program. We have downloaded and installed LibreOffice Base and can use it for our experiments later.

Chapter 4

Installing Python, PyCharm, and Psycopg

With the software we installed so far, we have two options to work with databases: We can use a command line client (like `psql`) or we can use a convenient GUI (like LibreOffice Base). Nevertheless, the maybe most likely method to work with a database is programmatically. Very often, high-level functionality and logic is implemented in program code instead of SQL. Applications can form a middle tier or top tier of an enterprise software architecture.

Therefore, in this book, we will also learn how to access and work with a database from normal program code. If you are reading this book as part of the *Databases* course [147] at our Hefei University 合肥大学, then you may be aware that I am also teaching a course *Programming with Python* [148]. Then it will not be a surprise that we will use Python [83, 104, 148] as the programming language of choice for accessing a database. The book [148] that accompanies the *Programming with Python* course is *also* free and open source. It too comes with examples available in a GitHub repository.

If you want to install the Python programming language on your system, you can follow the instructions given in [148]. In [148], we also recommend using PyCharm as Integrated Development Environment (IDE) for writing code. Of course, we also describe how to install that IDE. We will not reproduce these installation instructions here. You can look them up in [148]. In that book, we also discuss how to work with Git repositories and how to even download the sources of our book right here from `https://github.com/thomasWeise/databasesCode`. We even describe how to install the required packages to run this code!

Our example codes for accessing PostgreSQL databases from Python do require one package: `psycopg` [144]. This library provides a bridge between the Python programming language and PostgreSQL. It implements the Python DB API 2.0 specification [105], which means that code written using `psycopg` for accessing PostgreSQL may be reusable with another library and DBMS, given a few necessary changes. Either way, if you follow the instructions in *Programming with Python* [148] in the chapter on cloning Git repositories using PyCharm, you will directly learn how to install this library.

4.1 Installing Psycopg

Since the installation of the programming language Python and the IDE PyCharm is already covered in [148], we will not reproduce this information here. The download of our examples and the setup of the required `psycopg` library in a virtual environment in PyCharm is *also* covered in [148]. For the sake of completeness, we will therefore only briefly iterate over the installation of `psycopg` using pip at this stage. It is assumed, however, that you have read [148] at least to a point where you know what pip is and what virtual environments are. We also cover the installation only under Ubuntu Linux, as the installation steps under Microsoft Windows will be analogous (see again [148]).

The most common way to use external packages in Python is to install them into virtual environments. A virtual environment is essentially something like a directory hosting a stand-alone Python installation separate from the Python setup of the computer. This allows you to have different versions of different libraries installed (in different virtual environments).

Imagine that you have one program that requires the Python package NumPy in version 2.0 or above to run. You also need to use another application, which can only run with NumPy *below* version 2.0. This is a very common scenario. It would be impossible to realize, because you can only have one version of NumPy installed in your system. However, you can use both applications if each is installed into its own virtual environment. Because each virtual environment can have its own version of NumPy.

```
tweise@weise-laptop: /tmp
tweise@weise-laptop:/tmp$ mkdir .venv
tweise@weise-laptop:/tmp$
```

(4.1.1) We first open a terminal using Ctrl+Alt+T. We navigate to the directory where our programming will take place. We now create a directory `.venv` to host a new virtual environment by typing `mkdir .venv` and hitting Enter.

```
tweise@weise-laptop: /tmp
tweise@weise-laptop:/tmp$ mkdir .venv
tweise@weise-laptop:/tmp$ python3 -m venv --upgrade-deps .venv
```

(4.1.2) We now set up a new and empty virtual environment in this directory by writing `python3 -m venv --upgrade-deps .venv` and hitting Enter.

```
tweise@weise-laptop: /tmp
tweise@weise-laptop:/tmp$ mkdir .venv
tweise@weise-laptop:/tmp$ python3 -m venv --upgrade-deps .venv
tweise@weise-laptop:/tmp$ source .venv/bin/activate
```

(4.1.3) The virtual environment has been created. We now activate it by writing `source .venv/bin/activate` and hitting Enter.

```
tweise@weise-laptop: /tmp
tweise@weise-laptop:/tmp$ mkdir .venv
tweise@weise-laptop:/tmp$ python3 -m venv --upgrade-deps .venv
tweise@weise-laptop:/tmp$ source .venv/bin/activate
(.venv) tweise@weise-laptop:/tmp$
```

(4.1.4) The virtual environment `.venv` is now active, which we can see by the changed prompt.

```
tweise@weise-laptop: /tmp
tweise@weise-laptop:/tmp$ source .venv/bin/activate
(.venv) tweise@weise-laptop:/tmp$ pip install psycopg
Collecting psycopg
  Using cached psycopg-3.2.4-py3-none-any.whl.metadata (4.3 kB)
Collecting typing-extensions>=4.6 (from psycopg)
  Using cached typing_extensions-4.12.2-py3-none-any.whl.metadata (3.0 kB)
Using cached psycopg-3.2.4-py3-none-any.whl (198 kB)
Using cached typing_extensions-4.12.2-py3-none-any.whl (37 kB)
Installing collected packages: typing-extensions, psycopg
Successfully installed psycopg-3.2.4 typing-extensions-4.12.2
(.venv) tweise@weise-laptop:/tmp$
```

(4.1.5) We install `psycopg` into this environment by writing `pip install psycopg` and hitting Enter.

```
tweise@weise-laptop: /tmp
(.venv) tweise@weise-laptop:/tmp$ pip install psycopg
Collecting psycopg
  Using cached psycopg-3.2.4-py3-none-any.whl.metadata (4.3 kB)
Collecting typing-extensions>=4.6 (from psycopg)
  Using cached typing_extensions-4.12.2-py3-none-any.whl.metadata (3.0 kB)
Using cached psycopg-3.2.4-py3-none-any.whl (198 kB)
Using cached typing_extensions-4.12.2-py3-none-any.whl (37 kB)
Installing collected packages: typing-extensions, psycopg
Successfully installed psycopg-3.2.4 typing-extensions-4.12.2
(.venv) tweise@weise-laptop:/tmp$ deactivate
tweise@weise-laptop:/tmp$
```

(4.1.6) Once we have finished programming, we can deactivate the virtual environment by writing `deactivate` and hitting Enter. The prompt changes back to normal.

Figure 4.1: Installing the Python package `psycopg` into a virtual environment under Ubuntu Linux using pip in a Bash shell terminal.

Therefore, we will explore this route of installing packages, and we will do so under Ubuntu Linux in the Bash terminal in Figure 4.1. For Microsoft Windows, you can find a similar procedure discussed in [148]. The commands there are almost the same.

We begin by open a Bash terminal using Ctrl+Alt+T. Then we navigate to the directory where our programming will take place. In my example, I chose the temporary directory `/tmp` because I will delete everything once I am done taking screenshots. You would choose a more sensible location. Inside this directory, we now create a new directory `.venv` to host a new virtual environment. In the Bash shell, we can do this by typing the command `mkdir .venv` and hitting Enter in Figure 4.1.1

After this empty new directory is created, we can instruct Python to prepare it for use as a virtual environment in Figure 4.1.2. We set up a new and empty virtual environment in this directory by writing `python3 -m venv --upgrade-deps .venv` and hitting Enter. The new virtual environment has been created. We now activate it by writing `source .venv/bin/activate` and hitting Enter in Figure 4.1.3. Once a virtual environment is active, all package installations will go into that environment. Also, if we run a Python program, it will search for installed packages in this environment. And our virtual environment `.venv` is now active, which we can see by the changed prompt in Figure 4.1.4. Notice that the environment is only active in the current terminal. If you open another terminal using Ctrl+Alt+T, our virtual environment is not active in it (but we can activate it exactly as shown in Figure 4.1.3).

We install `psycopg` into this environment by writing `pip install psycopg` and hitting Enter. This invokes the `pip` installer, which looks the package up in PyPI and downloads it. In Figure 4.1.5, you see that it downloads `psycopg` in version `3.2.4`. When you do the same thing, you will probably get a newer version installed.

If we run a Python program that uses `psycopg`, like Listing 8.1, it will find this version of the package in our virtual environment. Once we have finished programming and running programs, we can deactivate the virtual environment by writing `deactivate` and hitting Enter. The prompt changes back to normal in Figure 4.1.6. Whenever we need `psycopg` again, we would activate the virtual environment again as shown in Figure 4.1.3. Of course, we can also install more packages into this environment. And we can also create more environments if we want to, in the same way as discussed here.

Part II

A Simple Example: The Factory Database

Chapter 5

Creating a User and the Data

Listing 5.1: Using SQL to create a the user `boss` with password `superboss123`. (stored in file `create_user.sql`; output in Listing 5.2)

```sql
/* In this example, we create a new user for our database. */

-- On PostgreSQL, there is a table ‘pg_catalog.pg_user‘ with all users.
-- We print the column ‘username‘ with the user names.
SELECT username FROM pg_catalog.pg_user;

-- Create the user 'boss'.
-- He will be the owner of the database that we will create.
CREATE USER boss WITH ENCRYPTED PASSWORD 'superboss123';

-- Now there is a new user: 'boss'.
SELECT username FROM pg_catalog.pg_user;
```

Listing 5.2: The standard output stream (stdout) resulting from the SQL statements in `create_user.sql` given in Listing 5.1.

```
$ psql "postgres://postgres:XXX@localhost" -v ON_ERROR_STOP=1 -ebf
    ↪ create_user.sql
 username
----------
 postgres
(1 row)

CREATE ROLE
 username
----------
 postgres
 boss
(2 rows)

# psql 16.6 succeeded with exit code 0.
```

Listing 5.3: Using SQL to create a database for user `boss`. (stored in file `create_database.sql`; output in Listing 5.4)

```sql
/* In this example, we create a new database named 'factory'. */

-- On PostgreSQL, there is a table `pg_databases` listing all databases.
-- We print the column `datname` with the names of the databases.
SELECT datname FROM pg_database;

-- Create the database 'factory', owned by user 'boss'.
CREATE DATABASE factory OWNER boss;

-- Store a comment about the purpose of our database.
COMMENT ON DATABASE factory is
    'This database holds all data about our factory and products.';

-- Now there is a new database in the list, namely 'factory'.
SELECT datname FROM pg_database;
```

Listing 5.4: The stdout resulting from the SQL statements in `create_database.sql` given in Listing 5.3.

```
$ psql "postgres://postgres:XXX@localhost" -v ON_ERROR_STOP=1 -ebf
    ↪ create_database.sql
  datname
-----------
 postgres
 template1
 template0
(3 rows)

CREATE DATABASE
COMMENT
  datname
-----------
 postgres
 factory
 template1
 template0
(4 rows)

# psql 16.6 succeeded with exit code 0.
```

Chapter 6

Creating Tables and Filling them with Data

Listing 6.1: Creating the table `product` to store the products we produce and sell. (stored in file `create_table_product.sql`; output in Listing 6.2)

```sql
/* We create the new table 'product' in our factory database. */

-- List all tables of the user 'boss' in database 'factory'
-- There are no tables yet.
SELECT tablename FROM pg_catalog.pg_tables
    WHERE tableowner='boss';

-- The table 'product' stores all the produces that we can produce.
-- Each row of this table identifies one such product.
CREATE TABLE product (
    id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,  -- must exist, must be unique
    price DECIMAL(10, 2) NOT NULL,  -- price (RMB): 10 digits, 2 after .
    weight INT NOT NULL,  -- the weight of the product, in grams
    width INT NOT NULL,  -- the width of the product, in mm
    height INT NOT NULL,  -- the height of the product, in mm
    depth INT NOT NULL  -- the depth of the product, in mm
);

-- List all tables of the user 'boss' in database 'factory'
-- Now we see the table 'product'.
SELECT tablename FROM pg_catalog.pg_tables
    WHERE tableowner='boss';
```

Listing 6.2: The stdout resulting from the SQL statements in `create_table_product.sql` given in Listing 6.1.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf create_table_product.sql
 tablename
-----------
(0 rows)

CREATE TABLE
 tablename
-----------
 product
(1 row)

# psql 16.6 succeeded with exit code 0.
```

Listing 6.3: Storing some products in the table `product`. (stored in file `insert_into_table_product.sql`; output in Listing 6.4)

```
/* Store some data into the table 'product'. */

-- Print all the contents from table 'product': Nothing.
SELECT * from product;

-- Insert 11 products into our table.
INSERT INTO product (name, price, weight, width, height, depth)
VALUES ('Shoe, Size 36', 150.99, 1300, 350, 250, 130),
       ('Shoe, Size 37', 152.99, 1325, 350, 250, 130),
       ('Shoe, Size 38', 154.99, 1350, 350, 250, 130),
       ('Shoe, Size 39', 156.99, 1375, 350, 250, 130),
       ('Shoe, Size 40', 158.99, 1400, 350, 250, 130),
       ('Shoe, Size 41', 160.99, 1425, 350, 250, 130),
       ('Shoe, Size 42', 162.99, 1450, 350, 250, 130),
       ('Shoe, Size 43', 164.99, 1475, 350, 250, 130),
       ('Small Purse', 100, 500, 350, 250, 130),
       ('Medium Purse', 120, 750, 400, 300, 200),
       ('Large Purse', 150, 1500, 600, 4000, 250);

-- Now there are 11 rows.
SELECT * from product;
```

Listing 6.4: The stdout resulting from the SQL statements in `insert_into_table_product.sql` given in Listing 6.3.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf insert_into_table_product.sql
 id | name | price | weight | width | height | depth
----+------+-------+--------+-------+--------+-------
(0 rows)

INSERT 0 11
 id |      name       | price  | weight | width | height | depth
----+-----------------+--------+--------+-------+--------+-------
  1 | Shoe, Size 36   | 150.99 |   1300 |   350 |    250 |   130
  2 | Shoe, Size 37   | 152.99 |   1325 |   350 |    250 |   130
  3 | Shoe, Size 38   | 154.99 |   1350 |   350 |    250 |   130
  4 | Shoe, Size 39   | 156.99 |   1375 |   350 |    250 |   130
  5 | Shoe, Size 40   | 158.99 |   1400 |   350 |    250 |   130
  6 | Shoe, Size 41   | 160.99 |   1425 |   350 |    250 |   130
  7 | Shoe, Size 42   | 162.99 |   1450 |   350 |    250 |   130
  8 | Shoe, Size 43   | 164.99 |   1475 |   350 |    250 |   130
  9 | Small Purse     | 100.00 |    500 |   350 |    250 |   130
 10 | Medium Purse    | 120.00 |    750 |   400 |    300 |   200
 11 | Large Purse     | 150.00 |   1500 |   600 |   4000 |   250
(11 rows)

# psql 16.6 succeeded with exit code 0.
```

Listing 6.5: Creating the table `customer` to store the information about our factory's customers. (stored in file `create_table_customer.sql`; output in Listing 6.6)

```sql
/* We create the new table 'customer' in our factory database. */

-- List all tables of the user 'boss' in database 'factory'
-- There is only the table 'product'.
SELECT tablename FROM pg_catalog.pg_tables
    WHERE tableowner='boss';

-- The table 'customer' stores all the customers that we have.
-- Each row of this table identifies one such a customer.
CREATE TABLE customer (
    id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,  -- must exist, must be unique
    mobile CHAR(20) NOT NULL UNIQUE,  -- the mobile phone number
    address VARCHAR(255) NOT NULL  -- the address
);

-- List all tables of the user 'boss' in database 'customer'
-- Now we see the table 'customer'.
SELECT tablename FROM pg_catalog.pg_tables
    WHERE tableowner='boss';
```

Listing 6.6: The `stdout` resulting from the SQL statements in `create_table_customer.sql` given in Listing 6.5.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf create_table_customer.sql
 tablename
-----------
 product
(1 row)

CREATE TABLE
 tablename
-----------
 product
 customer
(2 rows)

# psql 16.6 succeeded with exit code 0.
```

Listing 6.7: Storing some customer records in the table `customer`. (stored in file `insert_into_table_customer.sql`; output in Listing 6.8)

```sql
/* Store some data into the table 'customer'. */

-- Print all the contents from table 'customer': Nothing.
SELECT * from customer;

-- Insert 4 customers into our table.
INSERT INTO customer (name, mobile, address)
VALUES ('Bibbo', '99999999999', 'Hefei, Jinxiu Dadao 99'),
       ('Bebbo', '55555555555', 'Rathaus, Chemnitz, Germany'),
       ('Bebba', '33333333333', 'Times Square, NY, USA'),
       ('Bobbo', '44444444444', 'Eiffel Tower, Paris, France');

-- Now there are 4 rows.
SELECT * from customer;
```

Listing 6.8: The stdout resulting from the SQL statements in `insert_into_table_customer.sql` given in Listing 6.7.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf insert_into_table_customer.sql
 id | name | mobile | address
----+------+--------+---------
(0 rows)

INSERT 0 4
 id | name  |        mobile        |            address
----+-------+----------------------+-----------------------------
  1 | Bibbo | 99999999999          | Hefei, Jinxiu Dadao 99
  2 | Bebbo | 55555555555          | Rathaus, Chemnitz, Germany
  3 | Bebba | 33333333333          | Times Square, NY, USA
  4 | Bobbo | 44444444444          | Eiffel Tower, Paris, France
(4 rows)

# psql 16.6 succeeded with exit code 0.
```

Listing 6.9: Creating the table `demand` to store the orders of our customers. (stored in file `create_table_demand.sql`; output in Listing 6.10)

```
/* We create the new table 'demand' in our factory database. */

-- List all tables of the user 'boss' in database 'factory'
-- There are two tables: 'customer' and 'product'.
SELECT tablename FROM pg_catalog.pg_tables
    WHERE tableowner='boss';

-- The table 'demand' stores all the customer orders.
CREATE TABLE demand (
    id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    customer INT NOT NULL REFERENCES customer(id),
    product INT NOT NULL REFERENCES product(id),
    amount INT NOT NULL
);

-- List all tables of the user 'boss' in database 'factory'
-- Now we see the table 'demand'.
SELECT tablename FROM pg_catalog.pg_tables
    WHERE tableowner='boss';
```

Listing 6.10: The stdout resulting from the SQL statements in `create_table_demand.sql` given in Listing 6.9.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf create_table_demand.sql
 tablename
-----------
 product
 customer
(2 rows)

CREATE TABLE
 tablename
-----------
 product
 customer
 demand
(3 rows)

# psql 16.6 succeeded with exit code 0.
```

Listing 6.11: Storing some order records in the table `demand`. (stored in file `insert_into_table_demand.sql`; output in Listing 6.12)

```sql
/* Store some data into the table 'demand'. */

-- Print all the contents from table 'demand': Nothing.
SELECT * from demand;

-- Insert 7orders into our table.
INSERT INTO demand (customer, product, amount)
VALUES (1, 7, 12), (2, 3, 2), (3, 2, 7), (2, 5, 7), (1, 5, 3),
       (2, 6, 4), (3, 11, 10), (2, 3, 6);

-- Now there are 7 rows.
SELECT * from demand;
```

Listing 6.12: The stdout resulting from the SQL statements in `insert_into_table_demand.sql` given in Listing 6.11.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf insert_into_table_demand.sql
 id | customer | product | amount
----+----------+---------+--------
(0 rows)

INSERT 0 8
 id | customer | product | amount
----+----------+---------+--------
  1 |        1 |       7 |     12
  2 |        2 |       3 |      2
  3 |        3 |       2 |      7
  4 |        2 |       5 |      7
  5 |        1 |       5 |      3
  6 |        2 |       6 |      4
  7 |        3 |      11 |     10
  8 |        2 |       3 |      6
(8 rows)

# psql 16.6 succeeded with exit code 0.
```

Chapter 7

Selections and Views

Listing 7.1: Get the per-customer demands. (stored in file `select_customer_demand.sql`; output in Listing 7.2)

```sql
/* Get the number of demands per customer. */

-- List all demand IDs for each customer.
-- LEFT JOIN: 'Bobbo' also appears once, with a NULL demand.
SELECT customer.name as customer, demand.id as demand_id FROM
    customer LEFT JOIN demand ON (customer.id = demand.customer)
    ORDER BY customer.name;

-- Now we count the demands.
SELECT customer.name as customer, COUNT(demand.id) as demands FROM
    customer LEFT JOIN demand ON (customer.id = demand.customer)
    GROUP BY customer.name ORDER BY customer.name;
```

Listing 7.2: The stdout resulting from the SQL statements in `select_customer_demand.sql` given in Listing 7.1.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf select_customer_demand.sql
 customer | demand_id
----------+-----------
 Bebba    |         7
 Bebba    |         3
 Bebbo    |         8
 Bebbo    |         4
 Bebbo    |         2
 Bebbo    |         6
 Bibbo    |         1
 Bibbo    |         5
 Bobbo    |
(9 rows)

 customer | demands
----------+---------
 Bebba    |       2
 Bebbo    |       4
 Bibbo    |       2
 Bobbo    |       0
(4 rows)

# psql 16.6 succeeded with exit code 0.
```

Listing 7.3: Creating a view, i.e., a stored SQL query that can be treated like a table, to list the demands in human-readable form. (stored in file `create_view_sale.sql`; output in Listing 7.4)

```sql
/* Create a view showing the sales information */

-- We combine the three tables customer, product, and demand.
-- Basically, for each row in 'demand', we find the corresponding rows
-- in the 'customer' and 'product' tables (via the INNER JOIN).
-- This gives us one long row with all the information for one 'demand'.
-- We now choose only some elements of this long row and rename them.
-- We extract the name of the customer and refer to it "customer".
-- We extract the name of the product and refer to it as "product".
-- We also print the "amount" from each customer demand and the price.
CREATE VIEW sale AS
    SELECT customer.name AS customer, product.name AS product,
           product.price AS price, demand.amount AS amount
    FROM demand INNER JOIN customer ON (customer.id = demand.customer)
                INNER JOIN product ON (product.id = demand.product)
    ORDER BY customer.name, product, amount;

-- We can use the view as if it was a table!
SELECT * from sale;
```

Listing 7.4: The stdout resulting from the SQL statements in `create_view_sale.sql` given in Listing 7.3.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf create_view_sale.sql
CREATE VIEW
 customer |     product     | price  | amount
----------+-----------------+--------+--------
 Bebba    | Large Purse     | 150.00 |     10
 Bebba    | Shoe, Size 37   | 152.99 |      7
 Bebbo    | Shoe, Size 38   | 154.99 |      2
 Bebbo    | Shoe, Size 38   | 154.99 |      6
 Bebbo    | Shoe, Size 40   | 158.99 |      7
 Bebbo    | Shoe, Size 41   | 160.99 |      4
 Bibbo    | Shoe, Size 40   | 158.99 |      3
 Bibbo    | Shoe, Size 42   | 162.99 |     12
(8 rows)

# psql 16.6 succeeded with exit code 0.
```

Listing 7.5: Compute the per-customer and per-product sales based on the view `sale`. (stored in file `select_sale.sql`; output in Listing 7.6)

```
/* Extract some information from our database using the view 'sale'. */

-- Get the total sales per customer.
SELECT customer, SUM(amount * price) AS per_customer_sale
    FROM sale GROUP BY customer ORDER BY customer;

-- Get the total sales per product.
SELECT product, SUM(amount) AS amount,
    SUM(amount * price) AS per_product_sale
    FROM sale GROUP BY product ORDER BY product;
```

Listing 7.6: The stdout resulting from the SQL statements in `select_sale.sql` given in Listing 7.5.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf select_sale.sql
 customer | per_customer_sale
----------+-------------------
 Bebba    |           2570.93
 Bebbo    |           2996.81
 Bibbo    |           2432.85
(3 rows)

    product     | amount | per_product_sale
----------------+--------+------------------
 Large Purse    |     10 |          1500.00
 Shoe, Size 37  |      7 |          1070.93
 Shoe, Size 38  |      8 |          1239.92
 Shoe, Size 40  |     10 |          1589.90
 Shoe, Size 41  |      4 |           643.96
 Shoe, Size 42  |     12 |          1955.88
(6 rows)

# psql 16.6 succeeded with exit code 0.
```

Chapter 8

Connecting from Python

Listing 8.1: A Python program connecting to our factory database, using `INSERT` to add some records, and executing a `SELECT` statement. (stored in file `connect_insert_and_select.py`; output in Listing 8.2)

```python
"""Connect to our factory database, insert and select some records."""

from psycopg import connect
from psycopg.rows import dict_row

# Connect to the database and create a cursor to interact with the db:
with (connect("postgres://boss:superboss123@localhost/factory") as conn,
      conn.cursor(row_factory=dict_row) as cur):  # SELECT returns dicts
    print("Executing a single INSERT statement.")
    cur.execute(  # Insert one new demand record.
        "INSERT INTO demand (customer, product, amount) VALUES (%s,%s,%s)",
        (3, 4, 5))

    print("Executing a several INSERT statements at once.")
    cur.executemany(  # Insert three new demand records.
        "INSERT INTO demand (customer, product, amount) VALUES (%s,%s,%s)",
        ((3, 5, 2), (2, 7, 1), (1, 10, 5)))

    print("Now performing a SELECT request for customer Bebbo (id 2).")
    cur.execute("SELECT * FROM demand WHERE customer=%s", (2, ))
    for record in cur:  # Iterate over the records in the cursor.
        print(record)  # Print them as-is.

print("All done.")
```

↓ `python3 connect_insert_and_select.py` ↓

Listing 8.2: The stdout of the program `connect_insert_and_select.py` given in Listing 8.1.

```
Executing a single INSERT statement.
Executing a several INSERT statements at once.
Now performing a SELECT request for customer Bebbo (id 2).
{'id': 2, 'customer': 2, 'product': 3, 'amount': 2}
{'id': 4, 'customer': 2, 'product': 5, 'amount': 7}
{'id': 6, 'customer': 2, 'product': 6, 'amount': 4}
{'id': 8, 'customer': 2, 'product': 3, 'amount': 6}
{'id': 11, 'customer': 2, 'product': 7, 'amount': 1}
All done.
```

Listing 8.3: Repeating the query from Listing 7.5 after inserting the records using our Python program from Listing 8.1. (stored in file `select_sale_2.sql`; output in Listing 8.4)

```
/* Extract again information from our database using the view 'sale'. */

-- Get the total sales per customer again.
SELECT customer, SUM(amount * price) AS per_customer_sale
    FROM sale GROUP BY customer ORDER BY customer;

-- Get the total sales per product again.
SELECT product, SUM(amount) AS amount,
    SUM(amount * price) AS per_product_sale
    FROM sale GROUP BY product ORDER BY product;
```

Listing 8.4: The stdout resulting from the SQL statements in `select_sale_2.sql` given in Listing 8.3.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf select_sale_2.sql
 customer | per_customer_sale
----------+-------------------
 Bebba    |           3673.86
 Bebbo    |           3159.80
 Bibbo    |           3032.85
(3 rows)

    product     | amount | per_product_sale
----------------+--------+------------------
 Large Purse    |     10 |          1500.00
 Medium Purse   |      5 |           600.00
 Shoe, Size 37  |      7 |          1070.93
 Shoe, Size 38  |      8 |          1239.92
 Shoe, Size 39  |      5 |           784.95
 Shoe, Size 40  |     12 |          1907.88
 Shoe, Size 41  |      4 |           643.96
 Shoe, Size 42  |     13 |          2118.87
(8 rows)

# psql 16.6 succeeded with exit code 0.
```

Chapter 9

Cleanup After the Example

Listing 9.1: User `boss` deletes all tables and views inside the database, in the inverse order of their creation. (stored in file `cleanup_inside_database.sql`; output in Listing 9.2)

```sql
/* Cleanup after the example: Delete all Tables and Views. */

-- This must be run inside the database, by user 'boss'.

-- Delete the views.
DROP VIEW IF EXISTS sale;

-- Delete the tables, in the inverse order of creation.
DROP TABLE IF EXISTS demand;
DROP TABLE IF EXISTS product;
DROP TABLE IF EXISTS customer;
```

Listing 9.2: The `stdout` resulting from the SQL statements in `cleanup_inside_database.sql` given in Listing 9.1.

```
$ psql "postgres://boss:superboss123@localhost/factory" -v ON_ERROR_STOP=1
    ↪ -ebf cleanup_inside_database.sql
DROP VIEW
DROP TABLE
DROP TABLE
DROP TABLE
# psql 16.6 succeeded with exit code 0.
```

Listing 9.3: Delete the database `factory` and the user `boss`. This must be executed by the administrator account `postgres`. (stored in file `cleanup_database_and_user.sql`; output in Listing 9.4)

```sql
/* Cleanup after the example: Delete the Database and User */

-- This must be run by the administrative user 'postgres'.

-- If the database 'factory' exists, we delete it.
DROP DATABASE IF EXISTS factory;

-- If the user 'boss' already exists, we delete it.
-- We can only delete the user after all objects associated with it,
-- e.g., the databases, have been deleted.
DROP USER IF EXISTS boss;
```

Listing 9.4: The stdout resulting from the SQL statements in `cleanup_database_and_user.sql` given in Listing 9.3.

```
$ psql "postgres://postgres:XXX@localhost" -v ON_ERROR_STOP=1 -ebf
    ↪ cleanup_database_and_user.sql
DROP DATABASE
DROP ROLE
# psql 16.6 succeeded with exit code 0.
```

Backmatter

Glossary

Bash is a the shell used under `Ubuntu Linux`, i.e., the program that "runs" in the terminal and interprets your commands, allowing you to start and interact with other programs [27, 109, 155]. Learn more at `https://www.gnu.org/software/bash`.

client In a client-server architecture, the client is a device or process that requests a service from the server. It initiates the communication with the server, sends a request, and receives the response with the result of the request. Typical examples for clients are web browsers in the internet as well as clients for database management systems, such as `psql`.

client-server architecture is a system design where a central server receives requests from one or multiple clients [22, 60, 112, 116, 120]. These requests and responses are usually sent over network connections. A typical example for such a system is the World Wide Web (WWW), where web servers host websites and make them available to web browsers, the clients. Another typical example is the structure of database software, where a central server, the database management system, offers access to the database to the different clients. Here, the client can be some terminal software shipping with the DBMS, such as `psql`, or the different applications that access the databases.

CSV A very common and simple text format for exchanging tabular or matrix data is Comma-Separated Values (CSV) [124]. Each row in the text file represents one row in the table or matrix. The elements in the row are separated by a fixed delimiter, usually a comma (","), sometimes a semicolon (";"). `Python` offers some out-of-the-box CSV support in the `csv` module [45].

database A database (DB) is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases* [147].

database administrator A database administrator (DBA) is the person or group responsible for the effective use of database technology in an organization or enterprise.

database management system A database management system (DBMS) is the software layer located between the user or application and the database. The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the database [154].

database system A database system (DBS) is the combination of a database and a the corresponding database management system, i.e., basically, an installation of a DBMS on a computer together with one (or multiple) databases.

Git Git is a distributed Version Control Systems (Version Control System) which allows multiple users to work on the same code while preserving the history of the code changes [127, 138].

GitHub GitHub is a website where software projects can be hosted and managed via the Git Version Control System [114, 138]. Learn more at `https://github.com`.

Integrated Development Environment An Integrated Development Environment (IDE) is a program that allows the user do multiple different activities required for software development in one single system. It often offers functionality such as editing source code, debugging, testing, or interaction with a distributed version control system. For this course, we recommend using `PyCharm`.

JavaScript JavaScript is the predominant programming language used in websites to develop interactive contents for display in browsers [59].

JSON JavaScript Object Notation (JSON) is a data interchange format [28, 134] based on JavaScript [59] syntax.

LAMP Stack A system setup for web applications: Linux, Apache (a webserver), MySQL, and the server-side scripting language PHP [32, 80].

LibreOffice LibreOffice is on open source office suite [1, 68, 123] which can be considered as a good and free alternative to Microsoft Office [103].

LibreOffice Base is a database management systems that can work on stand-alone files but also connect to other popular relational databases [62, 123]. It is part of LibreOffice [1, 68, 123] and has functionality that is comparable to Microsoft Access [18, 37, 140].

Linux is the leading open source operating system, i.e., a free alternative for Windows [16, 77, 126, 136, 143]. We recommend using it for this course, for software development, and for research. Learn more at https://www.linux.org. Its variant Ubuntu is particularly easy to use and install.

locale A locale corresponds basically to a selection of country or culture for a system or application [90]. The locale then determines a set of country- or culture-dependent settings, such as the text representation for money or dates, or what decimal separators are used.A good example is, for example, that American English readers will interpret the date 7/4/2000 as July 4th, 2000, whereas British English readers will read this as the 7th of April, 2000 [50]. It is therefore important that software printing dates knows whether it is running on an American or British English PC.

MariaDB An open source relational database management system that has forked off from MySQL [6, 7, 17, 58, 117]. See https://mariadb.org for more information.

Matplotlib is a Python package for plotting diagrams and charts [84, 85, 89, 113]. Learn more at at https://matplotlib.org [85].

MySQL An open source relational database management system [25, 58, 119, 132, 150]. MySQL is famous for its use in the LAMP Stack. See https://www.mysql.com for more information.

NumPy is a fundamental package for scientific computing with Python, which offers efficient array datastructures [55, 76, 89]. Learn more at https://numpy.org [131].

pip is the standard tool to install Python software packages from the PyPI repository [56, 87]. To install a package `thepackage` hosted on PyPI, type `pip install thepackage` into the terminal. Learn more at https://packaging.python.org/installing.

port A port in networking is a software-defined number associated to a network protocol that receives or transmits communication for a specific service [101]. In the client-server architecture, the server listens for incoming communication connections at a specific port. The clients connect to the network address and port number of the server to establish such connections. Port numbers range from 0 to 65535, where the port numbers from 0 to 1023 are so-called well-known ports corresponding to the most common services, e.g., Hyper Text Transfer Protocol (HTTP), the protocol underpinning of the WWW, uses normally port 80.

PostgreSQL An open source object-relational database management system [61, 63, 110, 132].See https://postgresql.org for more information.

psql is the client program used to access the PostgreSQL DBMS server.

psycopg or, more exactly, `psycopg 3`, is the most popular PostgreSQL adapter for Python, implementing the Python DB API 2.0 specification [105]. Learn more at https://www.psycopg.org [144].

PyCharm is the convenient Python Integrated Development Environment that we recommend for this course [142, 151, 153]. It comes in a free community edition, so it can be downloaded and used at no cost. Learn more at https://www.jetbrains.com/pycharm.

PyPI The Python Package Index (PyPI) is an online repository that provides the software packages that you can install with pip [24, 135, 141]. Learn more at https://pypi.org.

Python The Python programming language [83, 104, 148], i.e., what you will learn about in our book [148]. Learn more at https://python.org.

relational database A relational database is a database that organizes data into rows (tuples, records) and columns (attributes), which collectively form tables (relations) where the data points are related to each other [39, 74, 75, 128, 130, 147, 149].

SciPy is a Python library for scientific computing [89, 145]. Learn more at https://scipy.org.

server In a client-server architecture, the server is a process that fulfills the requests of the clients. It usually waits for incoming communication carring the requests from the clients. For each request, it takes the necessary actions, performs the required computations, and then sends a response with the result of the request. Typical examples for servers are web servers [32] in the internet as well as database management systems. It is also common to refer to the computer running the server processes as server as well, i.e., to call it the "server computer [102]."

SQLite is an relational database management system which runs as in-process library that works directly on files as opposed to the client-server model used by other common database management systems. It is the most wide-spread SQL-based database in use today, installed in nearly every smartphone, computer, web browser, television, and automobile [33, 67, 152]. Learn more at https://sqlite.org/.

standard error stream The standard error stream (stderr) is one of the three pre-defined streams of a console process (together with the standard input stream and the standard output stream) [92]. It is the text stream to which the process writes information about errors and exceptions. If an uncaught `Exception` is raised in Python and the program terminates, then this information is written to stderr. If you run a program in a terminal, then the text that a process writes to its stderr appears in the console.

standard input stream The standard input stream (stdin) is one of the three pre-defined streams of a console process (together with the standard output stream and the standard error stream) [92]. It is the text stream from which the process reads its input text, if any. The Python instruction `input` reads from this stream. If you run a program in a terminal, then the text that you type into the terminal while the process is running appears in this stream.

standard output stream The standard output stream (stdout) is one of the three pre-defined streams of a console process (together with the standard input stream and the standard error stream) [92]. It is the text stream to which the process writes its normal output. The `print` instruction of Python writes text to this stream. If you run a program in a terminal, then the text that a process writes to its stdout appears in the console.

sudo In order to perform administrative tasks such as installing new software under Linux, root (or "super") user privileges as needed [38]. A normal user can execute a program in the terminal as super user by pre-pending `sudo`, often referred to as "super user do." This requires the root password.

terminal A terminal is a text-based window where you can enter commands and execute them [16, 38]. Knowing what a terminal is and how to use it is very essential in any programming- or system administration-related task. If you want to open a terminal under Windows, you can press [⊞]+[R], type in `cmd`, and hit [↵]. Under Ubuntu Linux, [Ctrl]+[Alt]+[T] opens a terminal, which then runs a Bash shell inside.

Ubuntu is a variant of the open source operating system Linux [38, 80]. We recommend that you use this operating system to follow this class, for software development, and for research. Learn more at https://ubuntu.com. If you are in China, you can download it from https://mirrors.ustc.edu.cn/ubuntu-releases.

Version Control System A Version Control Systems (Version Control System) is a software which allows you to manage and preserve the historical development of your program code [138]. A distributed Version Control System allows multiple users to work on the same code and upload their changes to the server, which then preserves the change history. The most popular distributed Version Control System is Git.

virtual environment A virtual environment is a directory that contains a local Python installation [108, 146]. It comes with its own package installation directory. Multiple different virtual environments can be installed on a system. This allows different applications to use different versions of the same packages without conflict, because we can simply install these applications into different virtual environments.

Windows Microsoft Windows is a commerical proprietary operating system [26]. It is widely spread, but we recommend using a Linux variant such as Ubuntu for software development and for our course. Learn more at https://www.microsoft.com/windows.

XML The Extensible Markup Language (XML) is a text-based language for storing and transporting of data [29, 41, 99]. It allows you to define elements in the form `<myElement myAttr="x">...text..</myElement>`. Different from CSV, elements in XML can be hierarchically nested, like `<a><b><c>test</c></b><b>bla</b></a>`, and thus easily represent tree structures. XML is one of most-used data interchange formats. To process XML in Python, use the `defusedxml` library [79], as it protects against several security issues.

YAML YAML Ain't Markup Language™ (YAML) is a human-friendly data serialization language for all programming languages [40, 57, 99]. It is widely used for configuration files in the DevOps environment. See https://yaml.org for more information.

Acronyms

Version Control System Version Control Systems, see Version Control System

API an Application Programming Interface is a set of rules or protocols that enables one software application or component to use or communicate with another [69]

CSV Comma-Separated Values, see, e.g., [45, 124] and CSV

DB database, see database

DBA database administrator, see database administrator

DBMS database management system, see database management system

DBS database system = DB + DBMS, see database system

GUI graphical user interface

HR human resources department

HTTP Hyper Text Transfer Protocol, the protocol linking web browsers to web servers in the WWW [20, 21, 64, 65, 71]

IDE Integrated Development Environment, see Integrated Development Environment

JSON JavaScript Object Notation, see JSON and [134]

OOP object-oriented programming

OS Operating System, the system that runs your computer, see, e.g., `Linux` and `Windows`

SQL Structured Query Language, see [33]

`stderr` see standard error stream and [92]

`stdin` see standard input stream and [92]

`stdout` see standard output stream and [92]

WWW World Wide Web [20, 54]

XML Extensible Markup Language, see XML and [29, 41, 99]

YAML YAML Ain't Markup Language™, see YAML and [40, 57, 99]

SQL Commands

ALTER
    ROLE, 17, 18
    USER, 17, 18

FROM, 30

INSERT, 59

PASSWORD, 17

SELECT, 30, 59

Bibliography

[1] *LibreOffice – The Document Foundation*. Berlin, Germany: The Document Foundation, 2024. URL: https://www.libreoffice.org (visited on 2024-12-12) (cit. on pp. 8, 32, 65).

[2] *Python 3 Documentation*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3 (visited on 2024-07-05).

[3] *A Timeline of Database History & Database Management*. Boston, MA, USA: Quickbase, Inc., Feb. 18, 2022–Aug. 25, 2023. URL: https://www.quickbase.com/articles/timeline-of-database-history (visited on 2025-01-11) (cit. on p. 6).

[4] Database Management Systems ANSI/X3/SPARC Study Group. *Framework Report on Database Management Systems*. Montvale, NJ, USA: American Federation of Information Processing Societies (AFIPS) Press, 1978. Also published as [137] (cit. on pp. 5, 79).

[5] David Ascher, ed. *Python Cookbook*. 1st ed. Sebastopol, CA, USA: O'Reilly Media, Inc., July 2002. ISBN: 978-0-596-00167-4 (cit. on p. 11).

[6] Adam Aspin and Karine Aspin. *Query Answers with MariaDB – Volume I: Introduction to SQL Queries*. Tetras Publishing, Oct. 2018. ISBN: 978-1-9996172-4-0. See also [7] (cit. on pp. 8, 65, 70).

[7] Adam Aspin and Karine Aspin. *Query Answers with MariaDB – Volume II: In-Depth Querying*. Tetras Publishing, Oct. 2018. ISBN: 978-1-9996172-5-7. See also [6] (cit. on pp. 8, 65, 70).

[8] Charles William Bachman. "Software for Random Access Processing". *Datamation* 9(4):36–41, Apr. 1965. Chicago, IL, USA: Technical Publishing Co. and Boston, MA, USA: Cahners Publishing Company. ISSN: 0011-6963 (cit. on p. 6).

[9] Charles William Bachman. "The Origin of the Integrated Data Store (IDS): The First Direct-Access DBMS". *IEEE Annals of the History of Computing* 31(4):42–54, Oct.–Dec. 2009. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1058-6180. doi:10.1109/MAHC.2009.110. URL: https://tschwarz.mscs.mu.edu/Classes/DB23/HW/bachmanIDS.pdf (visited on 2025-01-08) (cit. on pp. 6, 7).

[10] Paul Baran. *A Briefing on the Distributed Adaptive Message-Block Network*. Tech. rep. P-3127. Santa Monica, CA, USA: The RAND Corporation, Apr. 1965. URL: https://www.rand.org/content/dam/rand/pubs/papers/2008/P3127.pdf (visited on 2025-01-20) (cit. on p. 7).

[11] Paul Baran. *On a Distributed Command and Control System Configuration*. Memorandum RM-2632. Santa Monica, CA, USA: The RAND Corporation, Dec. 31, 1960. URL: https://www.rand.org/content/dam/rand/pubs/research_memoranda/2009/RM2632.pdf (visited on 2025-01-20) (cit. on p. 7).

[12] Paul Baran. *On Distributed Communications Networks*. Tech. rep. P-2626. Santa Monica, CA, USA: The RAND Corporation, Sept. 1962. URL: https://www.rand.org/content/dam/rand/pubs/papers/2005/P2626.pdf (visited on 2025-01-20) (cit. on p. 7).

[13] Paul Baran. *On Distributed Communications: I. Introduction to Distributed Communications Networks*. Memorandum RM-3042-PR. Santa Monica, CA, USA: The RAND Corporation, Aug. 1964. URL: https://www.rand.org/content/dam/rand/pubs/research_memoranda/2006/RM3420.pdf (visited on 2025-01-20) (cit. on p. 7).

[14] Paul Baran. *On Distributed Communications: V. History, Alternative Approaches, and Comparisons*. Memorandum RM-3097-PR. Santa Monica, CA, USA: The RAND Corporation, Aug. 1964. URL: https://www.rand.org/content/dam/rand/pubs/research_memoranda/2008/RM3097.pdf (visited on 2025-01-20) (cit. on p. 7).

[15] G. A. Barnard III and Louis Fein. "Organization and Retrieval of Records Generated in a Large-Scale Engineering Project". In: *Papers and Discussions Presented at the 1958 Eastern Joint Computer Conference. Modern Computers: Objectives, Designs, Applications (AIEE-ACM-IRE) 1958 (Eastern)*. Dec. 3–5, 1958, Philadelphia, PA, USA. Ed. by John M. Broomall. New York, NY, USA: Association for Computing Machinery (ACM), pp. 59–63. ISBN: 978-1-4503-7866-6. doi:10.1145/1458043.1458058 (cit. on p. 6).

[16] Daniel J. Barrett. *Efficient Linux at the Command Line*. Sebastopol, CA, USA: O'Reilly Media, Inc., Feb. 2022. ISBN: 978-1-0981-1340-7 (cit. on pp. 11, 65, 66).

[17] Daniel Bartholomew. *Learning the MariaDB Ecosystem: Enterprise-level Features for Scalability and Availability*. New York, NY, USA: Apress Media, LLC, Oct. 2019. ISBN: 978-1-4842-5514-8 (cit. on pp. 8, 65).

[18] Ben Beitler. *Hands-On Microsoft Access 2019*. Birmingham, England, UK: Packt Publishing Ltd, Mar. 2020. ISBN: 978-1-83898-747-3 (cit. on pp. 9, 32, 65).

[19] Uri Berman, Jackie Berman, and Bob Patrick. *The Birth of IMS/360*. Tech. rep. 102762458. Mountain View, CA, USA: Computer History Museum (CHM), Apr. 2007. URL: https://archive.computerhistory.org/resources/access/text/2016/12/102762458-05-01-acc.pdf (visited on 2025-01-08) (cit. on pp. 6, 7).

[20] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...* Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt (visited on 2025-02-05) (cit. on p. 68).

[21] Tim Berners-Lee, Roy T. Fielding, and Henrik Frystyk Nielsen. *Hypertext Transfer Protocol -- HTTP/1.0*. Request for Comments (RFC) 1945. Wilmington, DE, USA: Internet Engineering Task Force (IETF), May 1996. URL: https://www.ietf.org/rfc/rfc1945.txt (visited on 2025-02-05) (cit. on p. 68).

[22] Alex Berson. *Client/Server Architecture*. 2nd ed. Computer Communications Series. New York, NY, USA: McGraw-Hill. New York, NY, USA: McGraw-Hill: McGraw-Hill, Mar. 29, 1996. ISBN: 978-0-07-005664-0 (cit. on pp. 8, 64).

[23] Bernard Obeng Boateng. *Data Modeling with Microsoft Excel*. Birmingham, England, UK: Packt Publishing Ltd, Nov. 2023. ISBN: 978-1-80324-028-2 (cit. on p. 1).

[24] Ethan Bommarito and Michael Bommarito. *An Empirical Analysis of the Python Package Index (PyPI)*. arXiv.org: Computing Research Repository (CoRR) abs/1907.11073. Ithaca, NY, USA: Cornell Universiy Library, July 26, 2019. doi:10.48550/arXiv.1907.11073. URL: https://arxiv.org/abs/1907.11073 (visited on 2024-08-17). arXiv:1907.11073v2 [cs.SE] 26 Jul 2019 (cit. on p. 65).

[25] Silvia Botros and Jeremy Tinley. *High Performance MySQL*. 4th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Nov. 2021. ISBN: 978-1-4920-8051-0 (cit. on pp. 8, 65).

[26] Ed Bott. *Windows 11 Inside Out*. Hoboken, NJ, USA: Microsoft Press, Pearson Education, Inc., Feb. 2023. ISBN: 978-0-13-769132-6 (cit. on pp. 11, 67).

[27] Ron Brash and Ganesh Naik. *Bash Cookbook*. Birmingham, England, UK: Packt Publishing Ltd, July 2018. ISBN: 978-1-78862-936-2 (cit. on p. 64).

[28] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. Request for Comments (RFC) 8259. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 2017. URL: https://www.ietf.org/rfc/rfc8259.txt (visited on 2025-02-05) (cit. on pp. 1, 65).

[29] Tim Bray, Jean Paoli, C. M. Sperberg-McQueen, and Eve Maler, eds. *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. W3C Recommendation. Wakefield, MA, USA: World Wide Web Consortium (W3C). Wakefield, MA, USA: World Wide Web Consortium (W3C): World Wide Web Consortium (W3C), Nov. 26, 2008–Feb. 7, 2013. URL: http://www.w3.org/TR/2008/REC-xml-20081126 (visited on 2024-12-15) (cit. on pp. 1, 67, 68).

[30] Thomas Burns, Elizabeth N. Fong, David Jefferson, Richard Knox, Leo Mark, Christopher Reedy, Louis Reich, Nick Roussopoulos, and Walter Truszkowski. "Reference Model for DBMS Standardization, Database Architecture Framework Task Group (DAFTG) of the ANSI/X3/SPARC Database System Study Group". *ACM SIGMOD Record* 15(1):19–58, May 1985–Mar. 1986. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5808. doi:10.1145/16342.16343 (cit. on p. 5).

[31] Rudd H. Canaday, R.D. Harrison, Evan L. Ivie, J.L. Ryder, and L.A. Wehr. "A Back-End Computer for Data Base Management". *Communications of the ACM (CACM)* 17(10):575–582, Oct. 1974. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/355620.361172 (cit. on p. 7).

[32] Jason Cannon. *High Availability for the LAMP Stack*. Shelter Island, NY, USA: Manning Publications, June 2022 (cit. on pp. 8, 65, 66).

[33] Donald D. Chamberlin. "50 Years of Queries". *Communications of the ACM (CACM)* 67(8):110–121, Aug. 2024. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/3649887. URL: https://cacm.acm.org/research/50-years-of-queries (visited on 2025-01-09) (cit. on pp. 6, 8, 66, 68).

[34] Donald D. Chamberlin, Morton M. Astrahan, Kapali P. Eswaran, Patricia P. Griffiths, Raymond A. Lorie, James W. Mehl, Phyllis Reisner, and Bradford W. Wade. "SEQUEL 2: A Unified Approach to Data Definition, Manipulation, and Control". *IBM Journal of Research and Development* 20(6):560–575, Nov. 1976. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and Armonk, NY, USA: International Business Machines Corporation (IBM). ISSN: 0018-8646. doi:10.1147/RD.206.0560 (cit. on p. 8).

[35] Donald D. Chamberlin and Raymond F. Boyce. "SEQUEL: A Structured English Query Language". In: *Proceedings of 1974 ACM SIGFIDET (now SIGMOD) Workshop on Data Description, Access and Control*. May 1–3, 1974, Ann Arbor, MI, USA. Ed. by Gene Altshuler, Randall Rustin, and Bernard D. Plagman. Vol. 1. New York, NY, USA: Association for Computing Machinery (ACM), May 1974, pp. 249–264. ISBN: 978-1-4503-7415-6. doi:10.1145/800296.811515 (cit. on p. 8).

[36] Lois Mai Chan and Joan S. Mitchell. *Dewey Decimal Classification: Principles and Application*. Dublin, OH, USA: Ohio College Library Center (OCLC), Jan. 2003. ISBN: 978-0-910608-72-5 (cit. on p. 6).

[37] Christmas, FL, USA: Simon Sez IT. *Microsoft Access 2021 – Beginner to Advanced*. Birmingham, England, UK: Packt Publishing Ltd, Aug. 2023. ISBN: 978-1-83546-911-8 (cit. on pp. 9, 32, 65).

[38] David Clinton and Christopher Negus. *Ubuntu Linux Bible*. 10th ed. Bible Series. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd.: John Wiley and Sons Ltd., Nov. 10, 2020. ISBN: 978-1-119-72233-5 (cit. on pp. 11, 66).

[39] Edgar Frank "Ted" Codd. "A Relational Model of Data for Large Shared Data Banks". *Communications of the ACM (CACM)* 13(6):377–387, June 1970. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/362384.362685. URL: https://www.seas.upenn.edu/~zives/03f/cis550/codd.pdf (visited on 2025-01-05) (cit. on pp. 2, 7, 8, 66).

[40] Coding Gears and Train Your Brain. *YAML Fundamentals for DevOps, Cloud and IaC Engineers*. Birmingham, England, UK: Packt Publishing Ltd, Mar. 2022. ISBN: 978-1-80324-243-9 (cit. on pp. 1, 67, 68).

[41] Timothy W. Cole and Myung-Ja K. Han. *XML for Catalogers and Metadata Librarians (Third Millennium Cataloging)*. 1st ed. Dublin, OH, USA: Libraries Unlimited, May 23, 2013. ISBN: 978-1-59884-519-8 (cit. on pp. 1, 67, 68).

[42] *Connections: Local Networks*. Mountain View, CA, USA: Computer History Museum (CHM), 1996–2025. URL: https://www.computerhistory.org/revolution/networking/19/381 (visited on 2025-01-21) (cit. on p. 7).

[43] Fernando J. Corbató, Marjorie Merwin-Daggett, Robert C. Daley, R. J. Creasy, J. D. Hellwig, Richard H. Orenstein, and L. K. Korn. *The Compatible Time-Sharing System: A Programmer's Guide*. Cambridge, MA, USA: Massachusetts Institute of Technology (MIT) Computation Center and MIT Press, 1963–1964. URL: https://www.ibiblio.org/apollo/Documents/CTSS_ProgrammersGuide.pdf (visited on 2025-01-08). Second Printing May, 1964 (cit. on p. 6).

[44] Fernando J. Corbató and Victor A. Vyssotsky. "Introduction and Overview of the Multics System". In: *Proceedings of the 1965 Fall Joint Computer Conference (AFIPS'1965, Fall, Part 1)*. Nov. 30–Dec. 1, 1965, Las Vegas, NV, USA. Ed. by Robert W. Rector. New York, NY, USA: Association for Computing Machinery (ACM), Nov.–Dec. 1965, pp. 185–196. ISBN: 978-1-4503-7885-7. doi:10.1145/1463891.1463912. URL: https://www.multicians.org/fjcc1.html (visited on 2025-01-08) (cit. on p. 6).

[45] "csv – CSV File Reading and Writing". In: *Python 3 Documentation*. Vol. The Python Standard Library. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/library/csv.html (visited on 2024-11-14) (cit. on pp. 64, 68).

[46] *Cuneiform Tablets: From the Reign of Gudea of Lagash to Shalmanassar III*. Washington, DC, USA: Library of Congress. URL: https://www.loc.gov/collections/cuneiform-tablets (visited on 2025-01-08). See also [98, 139] (cit. on pp. 6, 76, 79).

[47] Robert C. Daley and Peter G. Neumann. "A General-Purpose File System for Secondary Storage". In: *Proceedings of the 1965 Fall Joint Computer Conference (AFIPS'1965, Fall, Part 1)*. Nov. 30–Dec. 1, 1965, Las Vegas, NV, USA. Ed. by Robert W. Rector. New York, NY, USA: Association for Computing Machinery (ACM), Nov.–Dec. 1965, pp. 213–229. ISBN: 978-1-4503-7885-7. doi:10.1145/1463891.1463915. URL: https://www.multicians.org/fjcc4.html (visited on 2025-01-08) (cit. on p. 6).

[48] *Database Language SQL*. Tech. rep. ANSI X3.135-1986. Washington, DC, USA: American National Standards Institute (ANSI), 1986 (cit. on p. 8).

[49] *Database Language SQL*. International Standard ISO 9075-1987. Geneva, Switzerland: International Organization for Standardization (ISO), 1987 (cit. on p. 8).

[50] *Dates*. Cambridge, England, UK: Cambridge University Press & Assessment, 2025. URL: https://dictionary.cambridge.org/grammar/british-grammar/dates (visited on 2025-01-21) (cit. on p. 65).

[51] Donald W. Davies. *Proposal for a Digital Communication Network*. Tech. rep. London, England, UK: National Physical Laboratory (NPL), June 1966. URL: https://www.dcs.gla.ac.uk/~wpc/grcs/Davies05.pdf (visited on 2025-01-21) (cit. on p. 7).

[52] Donald W. Davies. *Proposal for the Development of a National Communications Service for On-Line Data Processing*. Tech. rep. London, England, UK: National Physical Laboratory (NPL), Dec. 8, 1965 (cit. on p. 7).

[53] Donald W. Davies. *Remote On-line Data Processing and its Communication Needs*. Tech. rep. London, England, UK: National Physical Laboratory (NPL), Nov. 10, 1965 (cit. on p. 7).

[54] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: 978-0-13-299045-5 (cit. on p. 68).

[55] Justin Dennison, Cherokee Boose, and Peter van Rysdam. *Intro to NumPy*. Centennial, CO, USA: ACI Learning. Birmingham, England, UK: Packt Publishing Ltd, June 2024. ISBN: 978-1-83620-863-1 (cit. on p. 65).

[56] pip Developers. *pip Documentation v24.3.1*. Beaverton, OR, USA: Python Software Foundation (PSF), Oct. 27, 2024. URL: https://pip.pypa.io (visited on 2024-12-25) (cit. on p. 65).

[57] Ingy döt Net, Tina Müller, Pantelis Antoniou, Eemeli Aro, Thomas Smith, Oren Ben-Kiki, and Clark C. Evans. *YAML Ain't Markup Language (YAML™) version 1.2*. Revision 1.2.2. Seattle, WA, USA: YAML Language Development Team, Oct. 1, 2021. URL: https://yaml.org/spec/1.2.2 (visited on 2025-01-05) (cit. on pp. 1, 67, 68).

[58] Russell J.T. Dyer. *Learning MySQL and MariaDB*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2015. ISBN: 978-1-4493-6290-4 (cit. on pp. 8, 65).

[59] *ECMAScript Language Specification*. Standard ECMA-262, 3rd Edition. Geneva, Switzerland: Ecma International, Dec. 1999. URL: https://ecma-international.org/wp-content/uploads/ECMA-262_3rd_edition_december_1999.pdf (visited on 2024-12-15) (cit. on pp. 64, 65).

[60] The Editors of Encyclopaedia Britannica, Gloria Lotha, Aakanksha Gaur, Erik Gregersen, Swati Chopra, and William L. Hosch. "Client-Server Architecture". In: *Encyclopaedia Britannica*. Ed. by The Editors of Encyclopaedia Britannica. Chicago, IL, USA: Encyclopædia Britannica, Inc., Jan. 3, 2025. URL: https://www.britannica.com/technology/client-server-architecture (visited on 2025-01-20) (cit. on pp. 8, 64).

[61] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on pp. 8, 11, 12, 65).

[62] Steve Fanning, Vasudev Narayanan, flywire, Olivier Hallot, Jean Hollis Weber, Jenna Sargent, Pulkit Krishna, Dan Lewis, Peter Schofield, Jochen Schiffers, Robert Großkopf, Jost Lange, Martin Fox, Hazel Russman, Steve Schwettman, Alain Romedenne, Andrew Pitonyak, Jean-Pierre Ledure, Drew Jensen, and Randolph Gam. *Base Guide 7.3. Revision 1. Based on LibreOffice 7.3 Community*. Berlin, Germany: The Document Foundation, Aug. 2022. URL: https://books.libreoffice.org/en/BG73/BG73-BaseGuide.pdf (visited on 2025-01-13) (cit. on pp. 8, 11, 32, 65).

[63] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-564-1 (cit. on pp. 8, 11, 12, 65).

[64] Roy T. Fielding, Jim Gettys, Jeffrey C. Mogul, Henrik Frystyk Nielsen, and Tim Berners-Lee. *Hypertext Transfer Protocol -- HTTP/1.1*. Request for Comments (RFC) 2068. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Jan. 1997. URL: https://www.ietf.org/rfc/rfc2068.txt (visited on 2025-02-05) (cit. on p. 68).

[65] Roy T. Fielding, Mark Nottingham, and Julian F. Reschke. *HTTP Semantics*. Request for Comments (RFC) 9110. Wilmington, DE, USA: Internet Engineering Task Force (IETF), June 2022. URL: https://www.ietf.org/rfc/rfc9110.txt (visited on 2025-02-05) (cit. on p. 68).

[66] Keith D. Foote. *A Brief History of Database Management*. Studio City, CA, USA: Dataversity Digital LLC, Oct. 25, 2021. URL: https://www.dataversity.net/brief-history-database-management (visited on 2025-01-11) (cit. on p. 6).

[67] Kevin P. Gaffney, Martin Prammer, Laurence C. Brasfield, D. Richard Hipp, Dan R. Kennedy, and Jignesh M. Patel. "SQLite: Past, Present, and Future". 15(12):3535–3547, Aug. 2022. doi:10.14778/3554821.3554842. URL: https://www.vldb.org/pvldb/vol15/p3535-gaffney.pdf (visited on 2025-01-12). All papers in this issue were presented at the *48th International Conference on Very Large Data Bases (VLDB 2022)*, Sept. 5-9, 2022, hybrid/Sydney, NSW, Australia (cit. on pp. 8, 66).

[68] Jonas Gamalielsson and Björn Lundell. "Long-Term Sustainability of Open Source Software Communities beyond a Fork: A Case Study of LibreOffice". In: *Proceedings of the 8th IFIP WG 2.13 International Conference on Open Source Systems: Long-Term Sustainability OSS'12*. Sept. 10–13, 2012, Hammamet, Tunisia. Ed. by Imed Hammouda, Björn Lundell, Tommi Mikkonen, and Walt Scacchi. Vol. 378. IFIP Advances in Information and Communication Technology (IFIP-AICT). Heidelberg, Baden-Württemberg, Germany: Springer-Verlag GmbH Germany. Heidelberg, Baden-Württemberg, Germany: Springer-Verlag GmbH Germany: Springer-Verlag GmbH Germany, 2012, pp. 29–47. ISSN: 1868-4238. ISBN: 978-3-642-33441-2. doi:10.1007/978-3-642-33442-9_3 (cit. on pp. 8, 32, 65).

[69] Michael Goodwin. *What is an API?* Armonk, NY, USA: International Business Machines Corporation (IBM), Apr. 9, 2024. URL: https://www.ibm.com/topics/api (visited on 2024-12-12) (cit. on p. 68).

[70] Dawn Griffiths. *Excel Cookbook – Receipts for Mastering Microsoft Excel*. Sebastopol, CA, USA: O'Reilly Media, Inc., May 2024. ISBN: 978-1-0981-4332-9 (cit. on p. 1).

[71] Ilya Grigorik. *HTTP Protocols*. Sebastopol, CA, USA: O'Reilly Media, Inc., Dec. 2017. ISBN: 978-1-4920-3046-1 (cit. on p. 68).

[72] Donald J. Haderle and Cynthia M. Saracco. "The History and Growth of IBM's DB2". *IEEE Annals of the History of Computing* 35(2):54–66. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1058-6180. doi:10.1109/MAHC.2012.55 (cit. on p. 8).

[73] Thomas Haigh. "How Charles Bachman Invented the DBMS, A Foundation of Our Digital World". *Communications of the ACM (CACM)* 59(7):25–30, July 2016. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/2935880. URL: https://cacm.acm.org/opinion/how-charles-bachman-invented-the-dbms-a-foundation-of-our-digital-world (visited on 2024-01-09) (cit. on p. 6).

[74] Terry Halpin and Tony Morgan. *Information Modeling and Relational Databases*. 3rd ed. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, July 2024. ISBN: 978-0-443-23791-1 (cit. on pp. 2, 66).

[75] Jan L. Harrington. *Relational Database Design and Implementation*. 4th ed. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, Apr. 2016. ISBN: 978-0-12-849902-3 (cit. on pp. 2, 66).

[76] Charles R. Harris, K. Jarrod Millman, Stéfan van der Walt, Ralf Gommers, Pauli "pv" Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. "Array programming with NumPy". *Nature* 585:357–362, 2020. London, England, UK: Springer Nature Limited. ISSN: 0028-0836. doi:10.1038/S41586-020-2649-2 (cit. on p. 65).

[77] Michael Hausenblas. *Learning Modern Linux*. Sebastopol, CA, USA: O'Reilly Media, Inc., Apr. 2022. ISBN: 978-1-0981-0894-6 (cit. on pp. 11, 65).

[78] Lars Heide. "Shaping a Technology: American Punched Card Systems 1880-1914". *IEEE Annals of the History of Computing* 19(4):28–41, Oct.–Dec. 1997. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1058-6180. doi:10.1109/85.627897 (cit. on p. 6).

[79] Christian Heimes. *defusedxml 0.7.1: XML Bomb Protection for Python stdlib Modules*. Mar. 8, 2021. URL: https://pypi.org/project/defusedxml (visited on 2024-12-15) (cit. on p. 67).

[80] Matthew Helmke. *Ubuntu Linux Unleashed 2021 Edition*. 14th ed. Reading, MA, USA: Addison-Wesley Professional, Aug. 2020. ISBN: 978-0-13-668539-5 (cit. on pp. 8, 11, 65, 66).

[81] Herman Hollerith. *Apparatus for Compiling Statistics*. United States Patent US-0395783-A. Washington, DC, USA: United States Patent Office, Sept. 23, 1884–Jan. 8, 1889. URL: https://ppubs.uspto.gov/pubwebapp/static/pages/ppubsbasic.html (visited on 2025-01-07) (cit. on p. 6).

[82] Herman Hollerith. *Machine for Tabulating Statistics*. United States Patent 526,130. Washington, DC, USA: United States Patent Office, Aug. 20, 1892–Sept. 18, 1894. URL: https://ppubs.uspto.gov/pubwebapp/static/pages/ppubsbasic.html (visited on 2025-01-07) (cit. on p. 6).

[83] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer. Cham, Switzerland: Springer: Springer, 2023. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on pp. 11, 43, 66).

[84] John D. Hunter. "Matplotlib: A 2D Graphics Environment". *Computing in Science & Engineering* 9(3):90–95, May–June 2007. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1521-9615. doi:10.1109/MCSE.2007.55 (cit. on p. 65).

[85] John D. Hunter, Darren Dale, Eric Firing, Michael Droettboom, and The Matplotlib Development Team. *Matplotlib: Visualization with Python*. Austin, TX, USA: NumFOCUS, Inc., 2012–2025. URL: https://matplotlib.org (visited on 2025-02-02) (cit. on p. 65).

[86] *Information Technology – Database Languages – SQL – Part 1: Framework (SQL/Framework), Part 1*. International Standard ISO/IEC 9075-1:2023(E), Sixth Edition, (ANSI X3.135). Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), June 2023. URL: https://standards.iso.org/ittf/PubliclyAvailableStandards/ISO_IEC_9075-1_2023_ed_6_-_id_76583_Publication_PDF_(en).zip (visited on 2025-01-08). Consists of several parts, see https://modern-sql.com/standard for information where to obtain them. (Cit. on p. 8).

[87] "Installing Python Modules". In: *Python 3 Documentation*. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. URL: https://docs.python.org/3/installing (visited on 2024-08-17) (cit. on p. 65).

[88] Jay E. Israel, James G. Mitchell, and Howard E. Sturgis. *Separating Data From Function in a Distributed File System*. Blue and White Series CSL-78-5. Palo Alto, CA, USA: Xerox Palo Alto Research Center (PARC), Sept. 1978 (cit. on p. 8).

[89] Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with NumPy, SciPy and Matplotlib*. New York, NY, USA: Apress Media, LLC, Dec. 2018. ISBN: 978-1-4842-4246-9 (cit. on pp. 65, 66).

[90] "Locale". In: *POSIX.1-2024: The Open Group Base Specifications Issue 8, IEEE Std 1003.1™-2024 Edition*. Ed. by Andrew Josey. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and San Francisco, CA, USA: The Open Group, Aug. 8, 2024. Chap. XBD 7. URL: https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap07.html (visited on 2025-01-21) (cit. on p. 65).

[91] Andrew Josey, ed. *POSIX.1-2024: The Open Group Base Specifications Issue 8, IEEE Std 1003.1™-2024 Edition*. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and San Francisco, CA, USA: The Open Group, Aug. 8, 2024. URL: https://pubs.opengroup.org/onlinepubs/9799919799 (visited on 2024-10-30).

[92] "stderr, stdin, stdout – Standard I/O Streams". In: *POSIX.1-2024: The Open Group Base Specifications Issue 8, IEEE Std 1003.1™-2024 Edition*. Ed. by Andrew Josey. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and San Francisco, CA, USA: The Open Group, Aug. 8, 2024. URL: https://pubs.opengroup.org/onlinepubs/9799919799/functions/stdin.html (visited on 2024-10-30) (cit. on pp. 66, 68).

[93] Tom Kilburn, R. Bruce Payne, and David J. Howarth. "The Atlas Supervisor". In: *Proceedings of the 1961 Eastern Joint Computer Conference: Computers – Key to Total Systems Control, (AFIPS'1961, Eastern)*. Dec. 12–14, 1961, Washington, DC, USA. Ed. by Willis H. Ware. New York, NY, USA: Association for Computing Machinery (ACM), Dec. 1961, pp. 279–294. ISBN: 978-1-4503-7873-4. doi:10.1145/1460764.1460786. URL: https://www.chilton-computing.org.uk/acl/technology/atlas/p019.htm (visited on 2025-01-08) (cit. on p. 6).

[94] Won Kim. "Relational Database Systems". *ACM Computing Surveys (CSUR)* 11(3):187–211, Sept. 1979. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0360-0300. doi:10.1145/356778.356780 (cit. on p. 7).

[95] Barbara Klein, Richard Alan Long, Kenneth Ray Blackman, Diane Lynne Goff, Stephen P. Nathan, Moira McFadden Lanyi, Margaret M. Wilson, John Butterweck, and Sandra L. Sherrill. *Introduction to IMS: Your Complete Guide to IBM Information Management System*. 2nd ed. Indianapolis, IN, USA: IBM Press, Mar. 13, 2012. ISBN: 978-0-13-288687-1 (cit. on p. 6).

[96] Bernd Klein. *Einführung in Python 3 – Für Ein- und Umsteiger*. 3., überarbeitete. München, Bayern, Germany: Carl Hanser Verlag GmbH & Co. KG, 2018. ISBN: 978-3-446-45208-4. doi:10.3139/9783446453876 (cit. on p. 11).

[97] Leonard Kleinrock. *Information Flow in Large Communication Nets, Ph.D. Thesis Proposal*. Cambridge, MA, USA: Massachusetts Institute of Technology (MIT), May 31, 1961. URL: https://www.lk.cs.ucla.edu/data/files/Kleinrock/Information%20Flow%20in%20Large%20Communication%20Nets.pdf (visited on 2025-01-20) (cit. on p. 7).

[98] Leah Knobel and Neely Tucker. "It's As If It Was Written in... Clay. For 4,200 Years", Nov. 22, 2021. Washington, DC, USA: Library of Congress. ISSN: 2836-9459. URL: https://blogs.loc.gov/loc/2021/11/its-as-if-it-was-written-in-clay-for-4200-years (visited on 2025-01-08). See also [46] (cit. on pp. 6, 73).

[99] Katie Kodes. *Intro to XML, JSON, & YAML*. London, England, UK: Payhip, 2019–Sept. 4, 2020 (cit. on pp. 1, 67, 68).

[100] Tim Kraska and Michael Cafarella. *6.5830/6.5831: Database Systems*. Cambridge, MA, USA: Massachusetts Institute of Technology (MIT), Aut. 2024. URL: https://dsg.csail.mit.edu/6.5830 (visited on 2025-01-08) (cit. on p. 7).

[101] James F. Kurose and Keith Ross. *Computer Networking: A Top Down Approach*. 8th ed. Hoboken, NJ, USA: Pearson Education, Inc., Mar. 30, 2020. ISBN: 978-0-13-668155-7 (cit. on pp. 7, 8, 65).

[102] Jay LaCroix. *Mastering Ubuntu Server*. 4th ed. Birmingham, England, UK: Packt Publishing Ltd, Sept. 2022. ISBN: 978-1-80323-424-3 (cit. on p. 66).

[103] Joan Lambert and Curtis Frye. *Microsoft Office Step by Step (Office 2021 and Microsoft 365)*. Hoboken, NJ, USA: Microsoft Press, Pearson Education, Inc., June 2022. ISBN: 978-0-13-754493-6 (cit. on p. 65).

[104] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer. Cham, Switzerland: Springer: Springer, 2015. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (cit. on pp. 11, 43, 66).

[105] Marc-André Lemburg. *Python Database API Specification v2.0*. Python Enhancement Proposal (PEP) 249. Beaverton, OR, USA: Python Software Foundation (PSF), Apr. 12, 1999. URL: https://peps.python.org/pep-0249 (visited on 2025-02-02) (cit. on pp. 43, 65).

[106] Joseph Carl Robnett "Lick" Licklider. *MEMORANDUM FOR: Members and Affiliates of the Intergalactic Computer Network*. Washington, DC, USA: Advanced Research Projects Agency (ARPA), Apr. 23, 1963. URL: https://worrydream.com/refs/Licklider_1963_-_Members_and_Affiliates_of_the_Intergalactic_Computer_Network.pdf (visited on 2025-01-20) (cit. on p. 7).

[107] Stuart Macfarlane. *A Brief History of Databases*. Claymont, DE, USA: Sutori Website Administrator/HSTRY LTD., Nov. 1, 2024. URL: https://www.sutori.com/en/story/a-brief-history-of-databases (visited on 2025-01-10) (cit. on p. 6).

[108] Carl Meyer. *Python Virtual Environments*. Python Enhancement Proposal (PEP) 405. Beaverton, OR, USA: Python Software Foundation (PSF), June 13, 2011–May 24, 2012. URL: https://peps.python.org/pep-0405 (visited on 2024-12-25) (cit. on p. 67).

[109] Cameron Newham and Bill Rosenblatt. *Learning the Bash Shell – Unix Shell Programming: Covers Bash 3.0*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., 2005. ISBN: 978-0-596-00965-6 (cit. on p. 64).

[110] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: 978-1-4919-6336-4 (cit. on pp. 8, 11, 12, 65).

[111] Richard H. Orenstein and Robert C. Daley. *TO: Computation Center Staff. LDEDT and BPEDT, the CTSS Disk Editors*. Tech. rep. CC-208. Cambridge, MA, USA: Massachusetts Institute of Technology (MIT) Computation Center, May 9, 1963. URL: https://people.csail.mit.edu/saltzer/CTSS-Documents/CC-Memos/CC-208.pdf (visited on 2025-01-08) (cit. on p. 6).

[112] Robert Orfali, Dan Harkey, and Jeri Edwards. *Client/Server Survival Guide*. 3rd ed. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Jan. 25, 1999. ISBN: 978-0-471-31615-2 (cit. on pp. 8, 64).

[113] Ashwin Pajankar. *Hands-on Matplotlib: Learn Plotting and Visualizations with Python 3*. New York, NY, USA: Apress Media, LLC, Nov. 2021. ISBN: 978-1-4842-7410-1 (cit. on p. 65).

[114] Yasset Pérez-Riverol, Laurent Gatto, Rui Wang, Timo Sachsenberg, Julian Uszkoreit, Felipe da Veiga Leprevost, Christian Fufezan, Tobias Ternent, Stephen J. Eglen, Daniel S. Katz, Tom J. Pollard, Alexander Konovalov, Robert M. Flight, Kai Blin, and Juan Antonio Vizcaíno. "Ten Simple Rules for Taking Advantage of Git and GitHub". *PLOS Computational Biology* 12(7), July 14, 2016. San Francisco, CA, USA: Public Library of Science (PLOS). ISSN: 1553-7358. doi:10.1371/JOURNAL.PCBI.1004947 (cit. on p. 64).

[115] *RAMAC: The first random-access disk drive revolutionized how businesses use computers and set the stage for everything from space flight to e-commerce*. IBM Heritage. Armonk, NY, USA: International Business Machines Corporation (IBM). Armonk, NY, USA: International Business Machines Corporation (IBM): International Business Machines Corporation (IBM). URL: https://www.ibm.com/history/ramac (visited on 2025-01-09) (cit. on p. 6).

[116] Abhishek Ratan, Eric Chou, Pradeeban Kathiravelu, and Dr. M.O. Faruque Sarker. *Python Network Programming*. Birmingham, England, UK: Packt Publishing Ltd, Jan. 2019. ISBN: 978-1-78883-546-6 (cit. on pp. 8, 64).

[117] Federico Razzoli. *Mastering MariaDB*. Birmingham, England, UK: Packt Publishing Ltd, Sept. 2014. ISBN: 978-1-78398-154-0 (cit. on pp. 8, 65).

[118] Robert W. Rector, ed. *Proceedings of the 1965 Fall Joint Computer Conference (AFIPS'1965, Fall, Part 1)*. Nov. 30–Dec. 1, 1965, Las Vegas, NV, USA. New York, NY, USA: Association for Computing Machinery (ACM), Nov.–Dec. 1965. ISBN: 978-1-4503-7885-7. doi:10.1145/1463891.

[119] Mike Reichardt, Michael Gundall, and Hans D. Schotten. "Benchmarking the Operation Times of NoSQL and MySQL Databases for Python Clients". In: *Proceedings of the 47th Annual Conference of the IEEE Industrial Electronics Society (IECON'21*. Oct. 13–15, 2021, Toronto, ON, Canada. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), Oct. 2021, pp. 1–8. ISSN: 2577-1647. ISBN: 978-1-6654-3554-3. doi:10.1109/IECON48115.2021.9589382 (cit. on pp. 8, 65).

[120] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA, USA: O'Reilly Media, Inc., Jan. 2020. ISBN: 978-1-4920-4345-4 (cit. on pp. 8, 64).

[121] Lawrence G. Roberts. "The ARPANET & Computer Networks". In: *Proceedings of the ACM Conference on The History of Personal Workstations (HPW'86)*. Jan. 9–10, 1986, Palo Alto, CA, USA. New York, NY, USA: Association for Computing Machinery (ACM), Jan. 1986, pp. 51–58. ISBN: 978-0-89791-176-4. doi:10.1145/12178.12182 (cit. on p. 7).

[122] Heinz Schweppe and Manuel Scholz. "Introduction". In: *Einführung in die Datenbanksysteme. Datenbanken für die Bioinformatik*. Berlin, Germany: Freie Universität Berlin, Apr. 12–Oct. 11, 2005. URL: https://www.inf.fu-berlin.de/lehre/SS05/19517-V/FolienEtc/dbs045-01-Intro-2.pdf (visited on 2025-01-08) (cit. on p. 5).

[123] Winfried Seimert. *LibreOffice 7.3 – Praxiswissen für Ein- und Umsteiger*. Blaufelden, Schwäbisch Hall, Baden-Württemberg, Germany: mitp Verlags GmbH & Co. KG, Apr. 2022. ISBN: 978-3-7475-0504-5 (cit. on pp. 8, 11, 32, 65).

[124] Yakov Shafranovich. *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. Request for Comments (RFC) 4180. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Oct. 2005. URL: https://www.ietf.org/rfc/rfc4180.txt (visited on 2025-02-05) (cit. on pp. 1, 64, 68).

[125] Yuriy Shamshin. "The History of Databases". In: *Databases*. Riga, Latvia: ISMA University of Applied Sciences, May 2, 2024. URL: https://dbs.academy.lv/lection/dbs_LS02ENa_hist.pdf (visited on 2025-01-11) (cit. on p. 6).

[126] Ellen Siever, Stephen Figgins, Robert Love, and Arnold Robbins. *Linux in a Nutshell*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Sept. 2009. ISBN: 978-0-596-15448-6 (cit. on p. 65).

[127] Anna Skoulikari. *Learning Git*. Sebastopol, CA, USA: O'Reilly Media, Inc., May 2023. ISBN: 978-1-0981-3391-7 (cit. on p. 64).

[128] John Miles Smith and Philip Yen-Tang Chang. "Optimizing the Performance of a Relational Algebra Database Interface". *Communications of the ACM (CACM)* 18(10):568–579, Oct. 1975. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/361020.361025 (cit. on pp. 2, 66).

[129] Michael Stonebraker, ed. *The INGRES Papers: Anatomy of a Relational Database System*. Reading, MA, USA: Addison-Wesley Professional, 1986. ISBN: 978-0-201-07185-6 (cit. on p. 8).

[130] Allen Taylor. *Introducing SQL and Relational Databases*. New York, NY, USA: Apress Media, LLC, Sept. 2018. ISBN: 978-1-4842-3841-7 (cit. on pp. 2, 66).

[131] NumPy Team. *NumPy*. San Francisco, CA, USA: GitHub Inc and Austin, TX, USA: NumFOCUS, Inc. URL: https://numpy.org (visited on 2025-02-02) (cit. on p. 65).

[132] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: 978-1-80323-347-5 (cit. on pp. 8, 11, 12, 65).

[133] *The IBM Punched Card: The paper on-ramp to the Information Age once held most of the world's data*. IBM Heritage. Armonk, NY, USA: International Business Machines Corporation (IBM). Armonk, NY, USA: International Business Machines Corporation (IBM): International Business Machines Corporation (IBM). URL: https://www.ibm.com/history/punched-card (visited on 2025-01-08) (cit. on p. 6).

[134] *The JSON Data Interchange Syntax*. Standard ECMA-404, 2nd Edition. Geneva, Switzerland: Ecma International, Dec. 2017. URL: https://ecma-international.org/publications-and-standards/standards/ecma-404 (visited on 2024-12-15) (cit. on pp. 1, 65, 68).

[135] *The Python Package Index (PyPI)*. Beaverton, OR, USA: Python Software Foundation (PSF), 2024. URL: https://pypi.org (visited on 2024-08-17) (cit. on p. 65).

[136] Linus Torvalds. "The Linux Edge". *Communications of the ACM (CACM)* 42(4):38–39, Apr. 1999. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0001-0782. doi:10.1145/299157.299165 (cit. on pp. 11, 65).

[137] "The ANSI/X3/SPARC DBMS Framework Report of the Study Group on Database Management Systems". *Information Systems: Databases: Their Creation, Management and Utilization* 3(3):173–191. Ed. by Dennis Tsichritzis and Anthony Klug, 1978. Oxford, Oxfordshire, England, UK: Pergamon Press Ltd., now Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0306-4379. Also published as [4] (cit. on pp. 5, 70).

[138] Mariot Tsitoara. *Beginning Git and GitHub: Version Control, Project Management and Teamwork for the New Developer*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0215-7 (cit. on pp. 64, 66).

[139] Christina Tyler. "Clay Tablets Reveal Accounting Answers". *The Gazette* 10(36), Oct. 1, 1999. URL: https://www.loc.gov/collections/cuneiform-tablets/articles-and-essays/clay-tablets-reveal-accounting-answers (visited on 2025-01-08). Also part of [46] (cit. on pp. 6, 73).

[140] Laurie A. Ulrich and Ken Cook. *Access For Dummies*. Hoboken, NJ, USA: For Dummies, Dec. 2021. ISBN: 978-1-119-82908-9 (cit. on pp. 9, 32, 65).

[141] Marat Valiev, Bogdan Vasilescu, and James D. Herbsleb. "Ecosystem-Level Determinants of Sustained Activity in Open-Source Projects: A Case Study of the PyPI Ecosystem". In: *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT FSE'18)*. Nov. 4–9, 2018, Lake Buena Vista, FL, USA. Ed. by Gary T. Leavens, Alessandro F. Garcia, and Corina S. Păsăreanu. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 644–655. ISBN: 978-1-4503-5573-5. doi:10.1145/3236024.3236062 (cit. on p. 65).

[142] Bruce M. Van Horn II and Quan Nguyen. *Hands-On Application Development with PyCharm*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-235-0 (cit. on p. 65).

[143] Sander van Vugt. *Linux Fundamentals*. 2nd ed. Hoboken, NJ, USA: Pearson IT Certification, June 2022. ISBN: 978-0-13-792931-3 (cit. on p. 65).

[144] Daniele "dvarrazzo" Varrazzo, Federico "fogzot" Di Gregorio, and Jason "jerickso" Erickson. *Psycopg*. London, England, UK: The Psycopg Team, 2010–2023. URL: https://www.psycopg.org (visited on 2025-02-02) (cit. on pp. 11, 43, 65).

[145] Pauli "pv" Virtanen, Ralf Gommers, Travis E. Oliphant, Matt "mdhaber" Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, CJ Carey, Ilhan "ilayn" Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregos, Paul van Mulbregt, and SciPy 1.0 Contributors. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". *Nature Methods* 17:261–272, Mar. 2, 2020. London, England, UK: Springer Nature Limited. ISSN: 1548-7091. doi:10.1038/s41592-019-0686-2. URL: http://arxiv.org/abs/1907.10121 (visited on 2024-06-26). See also arXiv:1907.10121v1 [cs.MS] 23 Jul 2019. (Cit. on p. 66).

[146] "Virtual Environments and Packages". In: *Python 3 Documentation*. Vol. The Python Tutorial. Beaverton, OR, USA: Python Software Foundation (PSF), 2001–2024. Chap. 12. URL: https://docs.python.org/3/tutorial/venv.html (visited on 2024-12-24) (cit. on p. 67).

[147] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), Institute of Applied Optimization (应用优化研究所, IAO), 2025. URL: https://thomasweise.github.io/databases (visited on 2025-01-05) (cit. on pp. ii, 43, 64, 66).

[148] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), Institute of Applied Optimization (应用优化研究所, IAO), 2024–2025. URL: https://thomasweise.github.io/programmingWithPython (visited on 2025-01-05) (cit. on pp. 2, 11, 43, 45, 66).

[149] *What is a relational database?* Armonk, NY, USA: International Business Machines Corporation (IBM), Oct. 20, 2021–Dec. 12, 2024. URL: https://www.ibm.com/think/topics/relational-databases (visited on 2025-01-05) (cit. on pp. 2, 66).

[150] Ulf Michael "Monty" Widenius, David Axmark, and Uppsala, Sweden: MySQL AB. *MySQL Reference Manual – Documentation from the Source*. Sebastopol, CA, USA: O'Reilly Media, Inc., July 9, 2002. ISBN: 978-0-596-00265-7 (cit. on pp. 8, 65).

[151] Kevin Wilson. *Python Made Easy*. Birmingham, England, UK: Packt Publishing Ltd, Aug. 2024. ISBN: 978-1-83664-615-0 (cit. on p. 65).

[152] Marianne Winslett and Vanessa Braganholo. "Richard Hipp Speaks Out on SQLite". *ACM SIGMOD Record* 48(2):39–46, June 2019. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5808. doi:10.1145/3377330.3377338 (cit. on pp. 8, 66).

[153] Martin Yanev. *PyCharm Productivity and Debugging Techniques*. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2022. ISBN: 978-1-83763-244-2 (cit. on p. 65).

[154] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: https://www.techtarget.com/searchdatamanagement/definition/database-management-system (visited on 2025-01-11) (cit. on pp. 6, 64).

[155] Giorgio Zarrelli. *Mastering Bash*. Birmingham, England, UK: Packt Publishing Ltd, June 2017. ISBN: 978-1-78439-687-9 (cit. on p. 64).